

信奥沃土集训营

While(!Success) Try++;

函数专题 - 2025 (姓名:)

解题步骤

1. 读题

每道题必须读两遍以上，并针对没有理解的部分，对照样例进行理解，直到找到解决问题暴力（模拟）方法。

2. 算样例

用读题时找到的暴力方法（模拟）计算样例，验证暴力方法的正确性，并体会暴力方法中耗时的操作。

根据数据范围的提示信息，总结归纳出能够 AC 题目的算法。

用想到的 AC 方法重新演算样例，确保正确性。

如果题目提供的样例都演算完了，仍未能找到 AC 方法，时间允许的情况下，自己多出几个样例进行演算，除了可以找方法外，还可以作为测试数据。如果时间不充裕了，就写出暴力解法得部分分。

3. 选数据结构

针对暴力方法或者 AC 方法，选择适用的数据结构，降低时间复杂度。

4. 写步骤

对找到的方法进行归纳总结，提炼出粗略的步骤，指导后续的解题。

5. 伪代码

针对关键的步骤，写出伪代码。

6. 编程与测试

参照步骤和伪代码编写程序，并逐步做细致的测试。

不确定 AC 方法一定正确的时候，先将暴力方法编出来，通过数据规模 n 做分支，或者用对拍程序排错。

空程序模版

```
#include <bits/stdc++.h>

using namespace std;

#define ll long long //整数类型全部用 ll 定义，并做显式赋值，printf 和 scanf 用 %lld
#define el "\n"//用 el 代替 endl，提高速度，避免加速无效

const ll N=2e5+5;

ll n=0,a[N]={};//数组尽量定义成全局，最大数量不能超过 500 万
//vector<ll> v;

void solve()
{
    ll sum=0;

    cin>>n;

    //v.resize(n);

    for(ll i=0;i<n;i++)
    {
        cin>>a[i];

        sum+=a[i];

        //cerr 测试输出到控制台，提交代码时删除，限于文件读写的场景下
        cerr<<a[i]<<" "<<sum<<el;
    }

    cout<<sum<<el;//结果输出到文件
}

void pre(){} //预处理自定义函数
void init(){} //多测数据初始化自定义函数
int main()//main 函数返回式必须是 int
{
    ios::sync_with_stdio(0);//加速，10 万个以上数据时启用，也可以默认启用，
    cin.tie(0);cout.tie(0);//启用此代码禁止 cin 和 scanf 混用。

    //freopen("***.in","r",stdin);

    //freopen("***.out","w",stdout);
```

```
int T=1;//cin>>T;//多测时，取消注释

pre();

while(T--){

    init();

    solve();

}

return 0;

}
```

运算符运算优先级和结合方向性

优先级	运算符	描述	结合性
1	::	作用域解析运算符	左到右
2	() [] . -> ++ --	后缀运算符	左到右
3	++ -- + - ! ~ * & sizeof new delete typeid	一元运算符	右到左
4	*/ %	乘除取模	左到右
5	+ -	加法和减法	左到右
6	<< >>	左移和右移	左到右
7	< <= > >=	关系运算符	左到右
8	== !=	相等运算符	左到右
9	&	按位与	左到右
10	^	按位异或	左到右
11		按位或	左到右
12	&&	逻辑与	左到右
13		逻辑或	左到右
14	? :	条件运算符（三目运算符）	右到左
15	= += -= *= /= %= <<= >>= &= ^= =	赋值运算符	右到左
16	,	逗号运算符	左到右

运算符注意点

>比==运算级别高

算术运算的运算结果的数据类型与两个操作数范围较大的类型一致，char 类型按照 int 类型记。

如：int a;float f;

cout<<typeid(a+f).name(); 输出 f，即结果是 float 类型

又如：int a;char c;

cout<<typeid(a+c).name(); 输出 i，即结果是 int 类型

又如：char a,c;

cout<<typeid(a+c).name(); 输出 i，即结果是 int 类型，因为 char 类型按照 int 类型记

以上规则，导致/（除号）与数学÷差异较大，即当/两边都是整数的时候是数学上的整除，否则才跟数学上的小数除法一致。

%两边必须是整数，且结果的正负号只与被除数一致。

关系运算符的运算结果，逻辑运算符两边的运算数和运算结果都是 bool 类型，条件表达式的运算结果也是 bool 类型。

逻辑与、逻辑或有逻辑短路，如删除 string s 的前导零 if(s.size()>1&& s[0]!='0') 两个条件不要颠倒，以免 s 为空时取 s[0]导致错误。

复合运算法需要先赋值。++在前先自增再赋值，++在后先赋值再自增。

+=：string 的+=运算建议不用，容易出错。

数据类型及 string 类

char(a)：将 a 强制转化成以 a 为 ASCII 码的字符，'A':65 'a':97 '0':48 大写+32==小写

int(s.find("aa",0)) //强制转换成 int 类型

(long long)(t) //强制转换成 long long 类型，

unsigned：定义无符号整数，第一个二进制位不表示正负号，相当于只有非负数，所以非负数数范围扩大一倍。

unsigned int a = -20; printf("%d", a); 输出-20，为啥又成负数了？因为%d 是带符号整数的占位符，应该用%u。

unsigned long long a=9223372036854775807+1; printf("%llu",a);

int 转 bool 规则，非零转为 1，零转为 0。所以 if(5) 等同于 if(1)，while(x) 等同于 while(x!=0)

string 转整型：stoi：string 转 int；stoll：string 转 long long。如 long long t=stoll("123456");

整型转 string：to_string() 参数不能超过 unsigned long long 范围 string s=to_string(1234567);

整数类型选择时，除了要参考题目给的数据范围外，还需要注意运算过程，如果两个数的数据范围是 1e9，但需要求着两个的数的最小公倍数，中间要算乘法，所以可能会爆 int。

一维数组

数组最好定义在全局，总容量一般要控制在 500 万范围内，绝对不能超过 1000 万个 int，因为开不出来这么多连续的空间，也就是说如果数组容量需要 1000 万，那么这个数组应该不需要建。但如果没有找到不建数组的方法，那么超过 500 万个建议用 vector 实现。如果必须把数组定义在函数内部私有，数组容量不能超过 10 万。避免，因数组容量太大开不出来，导致直接 0 分。

在一些题目中，乍看上去需要数组，但仔细思考之后，可以只在输入的时候处理一遍，后面就不用了，这种时候尽量不要定义数组。

如果你找到的方法需要创建数组，而且根据题意数组容量超过 1 千万，那么可以肯定你的方法不够好。

如果题目需要输入 $1e9$ 以上的数据，但你找到的方法不需要对这些 $1e9$ 个数据进行输入，因为只输入就会超时，所以可以不用输入这 $1e9$ 个元素直接输出结果。

二维数组（矩阵）

下标一般从 1,1 开始更加方便。

主对角线： $i=j=0$ ，与主对角线平行的方向 $i-j$ 是固定值；副对角线： $i+j=n+1$ ，与副对角线平行的方向 $i+j$ 是固定值；

注意行与行、列与列的递推关系。

矩阵的转置、左右对称、上下对称、对角线对称、中心点对称的实现方法。

拐角矩阵和斜角矩阵的实现方法。

C 语言格式化输入输出函数 `cstdio` 头文件

`scanf("%c",&c)`：输入一个字符到变量 `c`，输入到变量或者数组元素必须加取地址符。如果输入到字符数组不用加，因为数组名本身就是个地址，如 `char str[105]; scanf("%s",str);`

`scanf("%1d",&d)`：按照场宽 1 分隔数据并输入到整数变量 `d`

注意：`scanf` 不合适输入到 `string` 类型。

`printf("%5.2lf\n",f)`：场宽 5 并保留两位小数输出变量 `f`，然后换行

`printf("%4.2s",s.c_str())`：场宽 4，输出 `string s` 的前两个字符

占位符：`int %d`；`long long %lld`；`float %f`；`double %lf`；`char %c`；`char c[] %s`；

格式：

`%-5.2lf`：靠左对齐，场宽 5，四舍五入保留两位小数

`%05d`：场宽 5，不足在前面补 0

`sprintf`：输出到字符数组中

```
char c[100];
```

```
sprintf(c,"%X",123232);
```

```
cout<<c; //字符数组 c 里存储了 123232 的十六进制字符串。
```

`stringstream`：字符串数据流，可以方便的进行数据类型的转换，如下：

```
stringstream ss;
```

```
ss<<"00123 456 "<<789; 将字符串放入数据流中
```

```
int a,b;
```

```
string s;
```

```
ss>>a>>b>>s; //把数据流中的数据输入到变量 a、b 和 s，a 的值是 123，b 的值是 456，s 的值是“789”。
```

sscanf: 从字符串中格式化输入数据

如以下代码可以将 string s="FF" 转成 10 进制数值

```
string s="FF";  
int a;  
sscanf(s.c_str(),"%X",&a);
```

其它输入函数

getchar(): 每次读取任意一个字符, 含特殊字符

cin.getline(s,100,'\$'): C 语言中, 输入一整行到字符数组 s 中, 最长 100 个字符, 遇到 '\$' 停止

getline(cin,s): C++ 语言中, 输入一整行到 string s 中

注意 cin 与 getline 衔接时, 中间需要用 getchar 取走 cin 留下的 '\n'

特殊输入 1:

```
//输入 123+456, 数值与符号紧挨着的情况  
int a,b; char c;  
//cin>>a>>c>>b; ok  
scanf("%d%c%d",&a,&c,&b);  
cout<<a<<" "<<c<<" "<<b<<endl;  
printf("%d %c %d\n",a,c,b);
```

特殊输入 2:

```
//输入 ab=12  
char ca[100];  
int a;  
//cin>>s>>c>>a;//错误  
scanf("%2s=%d",ca,&a);  
cout<<ca<<" "<<a<<endl;
```

特殊输入 3:

```
//不停输入直到结束  
while(cin>>a[n++])  
while(~scanf("%d %d",&a,&b))
```

数学 cmath 头文件

已有 double x;

fabs(x): 返回小数 x 的绝对值, double 类型

sqrt(x): 返回 x 的算术平方根 (非负数), double 类型

`pow(x,y)` : 返回 x 的 y 次方, `double` 类型

`ceil(x)` : 返回 x 向上取整的整数, `double` 类型

`floor(x)` : 返回 x 向下取整的整数, `double` 类型

`round(x)` : 返回 x 四舍五入取整的整数, `printf` 中的保留多少位小数也是用的四舍五入, `double` 类型

`int a=x;` : 去尾法取整

`s=sqrt(p*(p-a)*(p-b)*(p-c))` : 海伦公式计算三角形面积, $p=(a+b+c)/2.0$ 。

两点间距离: `sqrt((x1-x2)*(x1-x2)+(y1-y2)*(y1-y2))`

`log(x)`: 返回底数是自然常数 e 的 x 的对数。`double` 类型。

`log2(x)`: 返回底数是 2 的 x 的对数。`double` 类型。

`log10(x)`: 返回底数是 10 的 x 的对数。`double` 类型。

字符数组 `cstring` 头文件

`strlen(a)` : 返回字符数组 a 的字符个数

`strcmp(a,b)` : 字典序比较字符数组 a 和字符数组 b , 如果 $a>b$ 返回 1, $a==b$ 返回 0, $a<b$ 返回 -1

`strcat(a,b)`: 把 b 接到 a 后面

`string` 类型 `string` 头文件

`string` 类型与数值的转换函数

`string s=to_string(123)`: 将 123 转换为 `string` 类型。

`int t=stoi("101010",0,2)`: 从字符串的开头解析数字, 直到遇到第一个无效字符或者字符串结束。将 2 进制 `string "101010"` 转为 10 进制数值。无第 2、3 参数就是直接把十进制字符串转成 `int` 类型数值。

`int t=stoll("12345")`: 同上, 转成 `long long` 类型, `stoull` 转成 `unsigned long long`。

```
string s = "3.14";  
float f = stof(s);  
double d = stod(s);
```

`string` 类型的成员函数

`s.size()` : 返回 s 的字符数量, 同 `s.length()`, 返回值不是 `int` 类型, 也不是 `long long` 类型, 不要直接将 `s.size()` 作为 `min` 或者 `max` 的参数。当参数是函数返回值时, 最好不要用 `max` 和 `min`。如

`int t=max(s.size(),5);`

`s.begin()` : s 第一个元素的地址

`s.end()` : s 最后一个元素后面的地址

`s.insert(pos,"fsfsdfs")` : 在 `s` 的 `pos` 下标前插入字符串 $O(n)$

`s.find("faf",1)` : 从 `s` 的下标是 1 的位置开始查找"faf", 返回的值强转成 `int` 后, 如果没有找到就返回-1, 找到了就返回找到位置的下标。`s.find("faf")` 从头查找"faf" $O(n)$

`s.rfind("fa")`: 从尾到头找"fa", 返回开始位置的下标。

`s.erase(pos,n)` : 把 `s` 从 `pos` 下标开始删除 `n` 个字符, 直接改变 `s` 的内容, 无返回值。`s.erase(2)` 从下标为 2 的位置删除到结尾 $O(1)\sim O(n)$

`s.substr(pos,n)` : 返回 `s` 从 `pos` 下标开始连续取 `n` 个字符的字符串。`s.substr(1)`从下标为 1 的位置取到结尾 $O(n)$

`s.replace(pos,n,"fssdfs")` : 把 `s` 从 `pos` 下标开始取 `n` 个字符, 替换成"fssdfs", 直接改变 `s`, 无返回值。 $O(n)$

`printf("%s",s.c_str())` : `c_str()`返回 `string` 类型变量 `s` 的字符组成的字符数组。

数组初始化（赋值）

`memset(a,0,sizeof(a))` : 从 `a` 数组的首字节开始讲 `a` 数组的每个字节赋值为 0, 其效果能将 `a` 数组中的每个元素设置为 0。只有 0 和-1 好使。

如: `memset(a,-1,sizeof(a))` //`a` 数组的每个元素都赋值为-1

算法类 algorithm 头文件

`abs(x)` : 返回整数 `x` 的绝对值, `double` 类型

`min(a,b)` : 返回 `a` 与 `b` 中较小者, `double` 类型, `a` 和 `b` 必须是相同数据类型

`max(a,b)` : 返回 `a` 与 `b` 中较大者, `double` 类型, `a` 和 `b` 必须是相同数据类型

`__gcd(a,b)`: 计算整数 `a` 与 `b` 的最大公因数

`fill(a,a+n,x)` : 将 `a` 数组从下标为 0 的元素开始到最后一个元素全部赋值为 `x` $O(n)$

`sort(a,a+n,cmp)` : 将 `a` 数组从 `a[0]`到 `a[n-1]`排序, 默认从小到大排序, 如果指定了 `cmp` 比较函数, 就按照 `cmp` 比较函数排序。 $O(n\log n)$ 。`bool cmp(int a,int b){return a>b;}`//>表示从大到小排序

`sort(s.begin(),s.end())` : 将字符串 `s` 的字符按照从小到大排序, 若从大到小排序, 可以自己写 `cmp` 函数, 也可以用系统的 `less` 和 `greater` 比较器 如: `sort(s.begin(),s.end(),greater<char>())` $O(n\log n)$ 注意标出的小括号。

`stable_sort(a,a+n,cmp)` : `sort` 使用快速排序算法, 不稳定排序, `stable_sort` 使用归并排序, 稳定排序 $O(n\log n)$

`unique(a,a+n,cmp)`: 将有序数组 `a`, 按照 `cmp` 的排序规则去重, 返回最后一个不重复元素之后的指针或者迭代器, 经常后面-`a` 转化成不重复的元素数量。

`reverse(a,a+n)` : 将 `a` 数组从 `a[0]`到 `a[n-1]`反向, 前面的到后面, 后面的到前面。 $O(n)$

`reverse(s.begin(),s.end())` : 将字符串 `s` 的每个字符反向, 前面的到后面, 后面的到前面 $O(n)$

`binary_search(a,a+n,x)` : 在 `a` 数组中二分查找 `x`, 找到了就返回 1, 找不到就返回 0。 $O(\log n)$

`lower_bound(a,a+n,x)-a` : 找左侧边界: 就是在找到 `x` 之后继续往左找, 当重复执行结束后, `left` 是数组 `a` 中二分查找第一个 `>=x` 的位置, 如果找到了就返回找到位置的地址, 找不到就返回数组最后一个元素之后的地址 (这个地址是隔壁家的)。返回的是地址, 这个地址减 `a` 就得到了元素的下标, 类型是 `int`。 $O(\log n)$

`upper_bound(a,a+n,x)-a` : 找右侧边界后面: 就是在找到 `x` 之后继续往右找, 当重复执行结束后, `left` 就是在数组 `a` 中二分查找第一个 `>x` 的位置, 如果找到了就返回找到位置的地址, 找不到就返回数组最后一个元素之后的地址 (这个地址是隔壁家的)。返回的是地址, `-a` 就得到了元素的下标, 类型是 `int`。 $O(\log n)$

`nth_element(a,a+k-1,a+n)`: 返回第 `k` 小的元素。

`swap(a, b)` : 交换 `a` 与 `b` 的值, `a` 与 `b` 的数据类型必须一致

`while(next_permutation(a,a+n))` : 把数组 `a` 从 `a[0]` 到 `a[n-1]` 改变为下一个排列内容。

`prev_permutation` 是上一个排列内容。下 (上) 一个排列的规律:

1. 从右到左找第一个变小 (大) 的位置, 下标为 `x`
2. 从右到左找第一个大 (小) 于 `a[x]` 的位置, 下标为 `y`, 交换 `a[x]` 和 `a[y]`
3. 从下标 `x+1` 开始到结束的部分, 从小到大 (从大到小) 排序

`accumulate(a,a+n,0)`: 用于计算给定范围内元素的累加和 (或自定义操作的结果), 其参数如下:

`min_element(a,a+n)`: 返回 `a` 数组最小值的指针, `int minx=*min(a,a+n)`; `max_element` 返回最大值指针, 两者类似。

STL 及 STL 成员函数:

STL:

`vector`, 向量, 尾部快速增删, 元素数量可变, 随机访问迭代器, 支持下标。常用以下几种定义方式, 其它容易的定义方式类似:

`vector<int> v`; 只定义一个 `vector`, 无元素

`vector<int> v(5)`; 定义一个 `vector`, 含 5 个元素

`vector<int> v(100,5)`; 定义一个 `vector`, 含 100 个元素, 每个元素值都是 5

`vector<int> v(a,a+n)`; 定义一个 `vector`, 将 `a` 数组中的 `n` 个元素加入到 `vector` 中

`vector<int> v[105]`; 定义一个 `vector`, 含 105 行, 每行是一个 `vector<int>`, 相当于 105 行的二维数组, 每行的列数可以变。

`vector<vector<int>> v`; 定义一个 `vector`, 行数和每行的列数都可以变。

`vector<pair<int,int>> v`; 定义一个 `vector`, 每个元素是一个数对。

`deque`, 双端队列, 头和尾快速增删, 元素数量可变, 随机访问迭代器, 支持下标, 可以用来模拟队列和栈 (用来解决 `stack` 和 `queue` 不能遍历的问题)

`list`, 双向链表, 任何位置可以快速增删, 但查询较慢, 双向迭代器

`set`, 集合, 去重, 排序, 查询时间 $O(\log n)$.`count(x)` 查询有多少个元素 `x`, 无返回 0

`multiset`, 不去重, 排序, 查询时间 $O(\log n)$

`unordered_set`, 去重, 不排序, 查询时间 $O(1)$

`unordered_multiset`, 不去重, 不排序, 查询时间 $O(1)$

`map`, 映射, 去重, 排序, 查询时间 $O(\log n)$

`multimap`, 不去重, 排序, 查询时间 $O(\log n)$

`unordered_map`, 去重, 不排序, 查询时间 $O(1)$

`unordered_multimap`, 不去重, 不排序, 查询时间 $O(1)$

`stack`, 栈, 快速先进后出, 不支持迭代器, 不能遍历

`queue`, 队列, 快速先进先出, 不支持迭代器, 不能遍历

`priority_queue`, 优先队列, 适用于边维护数据边取最值, 对顶堆可以解决取前 10% 的分数线。
默认使用 `less` 比较器形成大顶堆。

`priority_queue<ll, vector<ll>, greater<ll>()> pq`; 用 `greater` 比较器定义一个小顶堆

`bitset`, 位图, 位图(`BitSet`)是一种高效的数据结构, 用于紧凑地存储和操作二进制位(0 和 1)的集合。它通过将多个位打包存储在单个字节或字中来实现空间的高效利用。位图的核心思想是:

使用**位(bit)**而不是字节(byte)来存储信息, 注意 `bs[0]`是二进制个位, 下标是跟数组倒着。

每个位代表一个布尔值(0 或 1)

通过位运算(AND, OR, XOR 等)高效操作

`array`, 同数组

`pair`, 数据对, 相当于`{first, second}`的结构体。`pair`定义的变量, 可以直接进行比较运算, 先比第一个元素, 第一个元素相等的再比较第二个元素。

STL 成员函数:

`.push(a)`: 加入数据到容器

`.pop()`: 删除容器中的头部数据

`.push_back(a)`: 在容器尾部添加数据 同功能: `emplace_back`

`.pop_back()`: 从容器尾部删除数据

`.push_front(a)`: 在容器头部添加数据 同功能: `emplace_front`

`.pop_front()`: 从容器头部删除数据

`.empty()`: 容器是否为空

`.size()`: 容器内元素数量

`.begin()`: 返回第一个元素的迭代器

`.end()`: 返回最后一个元素后面一个单位的的迭代器

`.rbegin()`: 返回最后一个元素的迭代器

`.rend()`: 返回第一个元素前面一个单位的迭代器

.insert(): 插入数据

.erase(): 删除数据

.assign(v,x): 向 vector 中增加 v 个元素 x

遍历方法:

```
void solve(){
    //对于 vector 和 deque, 支持随机访问迭代器的容器, 可以用以下四种方法
    vector<int> v(5,10);
    for(auto p:v) cout<<p<<" ";
    cout<<el;
    for(int i=0;i<v.size();i++) cout<<v[i]<<" ";
    cout<<el;
    for(auto it=v.begin();it<v.end();it++) cout<<*it<<" ";
    cout<<el;
    for(vector<int>::iterator it=v.begin();it<v.end();it++) cout<<*it<<" ";
    cout<<el;

    //对于 list、set、map, 支持双向迭代器, 用以下几种方法, 顺便演示结构体和 pair 的
    用法
    set<pair<int,int>> s;
    s.insert({5,10});
    for(auto p:s) cout<<p.first<<":"<<p.second<<el;
    for(auto it=s.begin();it!=s.end();it++){
        cout<<it->first<<" "<<it->second<<el;
    }
}
```

自定义函数

1. 参数传递方式

值传递

void myswap(int a,int b)//不改变实参

指针传递值

void myswap(int *a,int *b)//改变实参

引用传递

`void myswap(int &a,int &b)//改变实参`

思考什么时候需要一起用指针和引用？`void add(node *&root,int x)` 需要在自定义函数中改变指针赋值的地址时需要用引用。

形参默认值，如 `int toInt(string x="123")` 在调用函数时可以不传实参，如 `cout<<toInt("12345");`。

2. 函数返回值

无返回值 `void`，可以不写 `return`

其它类型的返回值，必须写 `return`，在有选择分支时需要格外注意。

```
int f(int i){
    while(i>=5){
        if(i%10==5)return 1;i/=10;
    }
    //此处必须有 return 0;
    return 0;
}
```

3. 函数重载

普通函数重载：函数名相同，但参数区类型或顺序不同。

```
void print(int num) {
    cout << "Integer number: " << num << std::endl;
}

void print(double num) {
    cout << "Double number: " << num << std::endl;
}

int main() {
    print(5);
    print(3.14);
    return 0;
}
```

结构体运算符重载

```

struct node{
    int num;
    bool operator < (const node b) const{
        return num>b.num;
    }
};

```

4. 构造函数

```

struct TreeNode{
    char element;
    TreeNode* leftChild;
    TreeNode* rightChild;
    TreeNode(char ch, TreeNode* l, TreeNode* r) {
        element = ch;
        leftChild = l;
        rightChild = r;
    }
    TreeNode() {
        element = '0';
        leftChild = nullptr;
        rightChild = nullptr;
    }
};

TreeNode tn=new TreeNode('a',&lc,&rc);

```

常用的自定义函数

1. 判断 x 是否是质数

以下代码可以在 1s 内判断 2500000000000000 以内的一个整数是不是质数。

```

bool prime(ll x){

```

```

if(x<2) return 0;
for(;; i=2;i*i<=x;i++) if(x%i==0) return 0; //注意 i 的数据类型
return 1;
}

```

//改进后，将 1, 2, 3, 4 以及 $6n+2, 6n+3, 6n+4$ 的情况优先做掉，先确认或者排除

```

bool isprime(int n){
    if(n<=1||n==4)
        return 0;
    if(n==2||n==3)
        return 1;
    if(n%6!=1&& n%6!=5)
        return 0 ;
    for(;; i=5;i*i<=n;i+=6) {
        if(n%i==0||n%(i+2)==0)
            return 0 ;
    }
    return 1;
}

```

2. 判断 x 是否回文数

```

bool hws(int x){
    int y=x,t=0;
    while(y!=0){
        t=t*10+y%10;
        y/=10;
    }
    return (x==t);
}

```

5. 求整数 a 与 b 的最大公因数

```

int gcd(int a,int b){
    while(a%b!=0){
        int t=a%b;a=b;b=t;  //或者 a%=b,swap(a,b);
    }
    return b;
}

int gcd2(int a, int b) {
    return b == 0 ? a : gcd2(b, a % b); //return a%b==0?b:gcd2(b,a%b);
}

```

6. 求 a 与 b 的最小公倍数

注意 $a*b$ 可能爆 int。

```

ll lcm(ll a,ll b){
    return a/gcd(a,b)*b; //先除再乘，防止爆ll
}

```

7. 打前三大擂台

打擂台时务必要注意擂台变量的初始值。

```

#include<iostream>
using namespace std;

int main(){
    int a,maxn1=INT_MIN,maxn2=INT_MIN,maxn3=INT_MIN;
    for(int i=1;i<=10;i++){
        cin>>a;
        if(a>maxn1){
            maxn3=maxn2;
            maxn2=maxn1;
            maxn1=a;
        }else if(a>maxn2){
            maxn3=maxn2;

```

```

        maxn2=a;
    }else if(a>maxn3){
        maxn3=a;
    }
}

cout<<maxn1<<" "<<maxn2<<" "<<maxn3;
return 0;
}

```

8. 比较字符串 **s** 和 **t** 的数值序大小

```

int scmp(string s,string t){
    if(s==t) return 0;
    else if(s.size()<t.size())||s.size()==t.size()&& s<t) return -1;
    else return 1;
}

```

9. 字符串中删除字符

```

//去前导零
while(s.size()>1&&s[0]=='0') s.erase(0,1);
//删除字符串 s 中的数字字符, 注意 i--
for(int i=0;i<s.size();i++){
    if(s[i]>='0'&&s[i]<='9') s.erase(i,1),i--;
}

```

10. 判断整数 **x** 是否是平方数

```

bool pingFang(int x){
    int t=round(sqrt(x));
    return (t*t==x);
}

```

11. 判断整数 **x** 是否是整数的 **y** 次幂

```
bool miShu(int x,int y){
    int t=round(pow(x,1.0/y));
    return (pow(t,y)==x);
}
```

12. sort 中使用的自定义比较函数 cmp

```
bool cmp(int a,int b){//注意此处的数据类型，特别是结构体比较的时候
    return a<b;    <从小到大排序，>从大到小排序
}

bool cmp(node a,node b){
    if(a.zf!=b.zf) return a.zf>b.zf;//先按总分排名，从大到小
    if(a.yw!=b.yw) return a.yw>b.yw;//再按语文排名，从大到小
    return a.xm<b.xm;//再按姓名字典序从小到大排序
}
```

13. 把整数转换成 string

```
//方法一：系统函数 to_string
string s=to_string(12345678910121314); //参数不能大于 unsigned long long 范围

//方法二：
string toString(int x){
    string res;
    bool flag=0;
    if(x<0) flag=1,x=-x,res.erase(0,1);
    while(x){
        res=char(x%10+'0')+res;
        x/=10;
    }
    if(flag) res='-'+res;
    return res;
}
```

```
//方法三:
string toString(int x){
    stringstream ss;
    ss<<x;
    string res;
    ss>>res;
    return res;
}
```

14. 把 int x 转二进制。ps: do while 兼顾 x 为 0 的情况

```
string toEr(int x){
    string res;
    do{
        res=char(x%2+'0')+res;
        x/=2;
    }while(x);
    return res;
}

//方法二: bitset, 需要注意前导零
bitset<32> b(x);
string res=b.to_string();
while(res[0]=='0'&&res.size()>1) res.erase(0,1);
```

15. 把 int x 转成 y 进制。ps: do while 兼顾 x 为 0 的情况

```
string toYjinzhi(int x,int y){
    string res,s="0123456789ABCDEFGHIJKLMNPOQRSTUVWXYZ";
    do{
        res=s[x%y]+res;
        x/=y;
    }while(x);
}
```

```
    return res;
}
```

16. 十六进制 string x 转二进制

```
string HtoEr(string x){
    string res;
    map<char,string> m;//建立十六进制位与四位二进制位的字典
    m.insert({'0',"0000"});
    m.insert({'1',"0001"});
    m.insert({'2',"0010"});
    m.insert({'3',"0011"});
    m.insert({'4',"0100"});
    m.insert({'5',"0101"});
    m.insert({'6',"0110"});
    m.insert({'7',"0111"});
    m.insert({'8',"1000"});
    m.insert({'9',"1001"});
    m.insert({'A',"1010"});
    m.insert({'B',"1011"});
    m.insert({'C',"1100"});
    m.insert({'D',"1101"});
    m.insert({'E',"1110"});
    m.insert({'F',"1111"});
    for(int i=0;i<x.size();i++){
        res=res+m[x[i]];
    }
    while(res.size()>1&&res[0]=='0') res.erase(0,1);//去前导零
    return res;
}

string s="FF";
int a=stoi(s,0,16);
bitset<32> b(a);
```

```
s=b.to_string();  
while(s[0]!='0'&& s.size()>1) s.erase(0,1);  
cout<<s;
```

17. string 转整数

```
//方法一  
int toInt(string x){  
    int res=0;  
    for(int i=0;i<x.size();i++){  
        res=res*10+x[i]-'0';  
    }  
    return res;  
}  
  
//方法二：用 stringstream（略）  
stringstream ss;  
ss<<x;  
int a;  
ss>>a;  
  
//方法三：用 stoi 或者 stoll  
int a=stoi(x);
```

18. 二进制中 1 的数量

```
int count_bits(int n) {  
    int count = 0;  
    while (n) {  
        n &= (n - 1);  
        count++;  
    }  
  
    //cnt+=n&1;这样也可以  
    //n>>1;
```

```

    }

    return count;
}

```

19. 求组合数 $C(n,k)$

```

// 组合数公式法 O(n)
long long binomialCoeff(int n, int k) {
    if (k > n - k) k = n - k;
    long long res = 1;

    for (int i = 0; i < k; ++i) {
        res *= (n - i);
        res /= (i + 1);
    }

    return res;
}

```

组合数还有一个递推公式： $C(n,k)=C(n-1,k-1)+C(n-1,k)$ ，放在二维数组中理解，那么就是 $a[i][j]=a[i-1][j-1]+a[i-1][j]$ ，当前格子的值是左上角的值+正上方的值，而且 $C(n,0)$ 和 $C(n,n)$ 也是 1，这个规律跟杨辉三角一致。所以用计算杨辉三角和可以算出多个 $C(n,k)$ 的值。

```

ll Cnk[2005][2005];
void solve() {
    ll n,k,p;//C(n,k)对 p 求余
    cin>>n>>k>>p;
    for(int i=0;i<=n;i++){
        for(int j=0;j<=i;j++){
            if(j==0||i==j) Cnk[i][j]=1;
            else Cnk[i][j]=Cnk[i-1][j-1]+Cnk[i-1][j];
            cout<<Cnk[i][j]<<" ";
        }
        cout<<el;
    }
    cout<<Cnk[n][k];
}

```

```
    return;
}
```

20. 递归函数模版

```
void 递归函数(参数列表)
{
    if(结束条件)//满足后执行下列语句，结束当前函数并返回
    {
        //执行函数体，结束函数前该干的事情
    }
    else{
        //执行函数体，满足当前操作
        //改变参数，进行下次操作
    }
}
```

如斐波那契数列函数

```
int feb(int x){
    if(x<2) return 1;
    else return feb(x-1)+feb(x-2);
}
```

以上代码存在大量的重复计算，可以做记忆化优化

```
int a[N]={0};
int feb(int x){
    if(x<2) return 1;
    if(!a[x]) a[x]=feb(x-1)+feb(x-2);
    return a[x];
}
```

能用递推不用递归，超过 10000 层不要用递归改用栈模拟。

21. 一维数组去重

```
int n=10,a[15]={14,2,3,4,5,5,5,6,7,8};
```

```
//去重
```

```
//方法一：排序后，第一个元素一定不跟前面重复，依次将后面的元素与前面的元素做对比
```

```
//如果不相等就输出后面的元素。
```

```
sort(a,a+n);
cout<<a[0]<<" ";
for(int i=1;i<n;i++) if(a[i]!=a[i-1]) cout<<a[i]<<" ";
cout<<endl;
```

```
//方法二： set
```

```
unordered_set<int> st;
for(int i=0;i<n;i++) st.insert(a[i]);
for(auto it=st.begin();it!=st.end();it++) cout<<*it<<" ";
cout<<endl;
```

```
//方法三：排序后用 unique(a,a+n) 将不重复的元素前移，重复的元素后移，并返回非重复元素后面的指针。
```

```
//-a 之后就是第一个重复的元素，从 0 开始的数组的话，-a 得到不重复的元素数量
```

```
sort(a,a+n);
n=unique(a,a+n)-a;
for(int i=0;i<n;i++) cout<<a[i]<<" ";
```

22. 快速幂

整数的快速幂，适用与模数 p 较小时 ($p \cdot p$ 不爆 long long) 时间复杂度 $O(\log n)$

代码块

```
ll qPow(ll a,ll b,ll p){
    ll res=1;
    while(b){
        if(b%2==1) res=res*a%p; //if(b&1)
        a=a*a%p;
        b/=2;//b>>1
    }
    return res;
```

```

}

ll fast_pow(ll base, ll power, ll mod) {
    ll result = 1;
    while (power > 0) {
        if (power & 1) {
            result = (result * base) % mod;
        }
        base = (base * base) % mod;
        power >>= 1;
    }
    return result;
}

```

小数的快速幂，含负指数的情况

```

double fast_power(double x, int n) {
    double res = 1.0;
    ll m = n;
    if (m < 0) {
        x = 1 / x;
        m = -m;
    }
    while (m > 0) {
        if (m % 2 == 1) {
            res *= x;
        }
        x *= x;
        m /= 2;
    }
    return res;
}

```

23. 安全模乘函数（龟速乘）

安全模乘函数避免模 p (p 很大, 如 $1e18+3$) 计算过程中爆 long long 的情况, 即 $(a\%p)*(a\%p)$ 会爆 long long 的情况。

```
ll p=1e18+3;
ll safe_cheng(ll a, ll b) {
    ll res = 0;
    a %= p;
    b %= p;
    while (b > 0) {
        if (b % 2 == 1) res = (res + a) % p;
        a = (a * 2) % p;
        b /= 2;
    }
    return res;
}
```

24. 快速乘

如果要计算 $a*b\%p$, 若 p 很大, $a\%p*(b\%p)$ 的结果可能爆 long long, 又不想用高精精度, 那么可以用以下快速乘法 (来自国家集训队论文) :

既然说到了这里, 就再顺便提及一个技巧, 它用来解决这样一个问题, 如果要求我们模的常数是一个 64bit 整数, 那在做乘法时就没有扩展类型给我们使用了, 这时看上去必须要我们手动编写高精度整数运算的代码了。

幸运的是, 有一个巧妙的方法可以简单解决这个问题:

```
typedef long long ll;
#define MOL 123456789012345LL
inline ll mul_mod_ll(ll a, ll b) {
    ll d = (ll) floor(a * (double) b / MOL + 0.5);
    ll ret = a * b - d * MOL;
    if (ret < 0) ret += MOL;
    return ret;
}
```

首先它使用浮点运算来得到 $a*b/MOL$ 的值, 关键在于第二句, 显然 $a*b-d*MOL$ 中的两个乘法都是可能会溢出的, 不过没关系, 因为我们可以预见其差是一个 64bit 可容纳的正整数, 那么溢出部分的差仅可能为 0 或者 1, 最后一句符号的特判用来处理溢出部分差为 1 的情况。容易注意到, 我们计算 $a*b/MOL$ 时使用了浮点运算, 误差是不可避免的, 故建议不要对太大的 MOL 使用这个算法。(感谢 crazyboy 无私地给我提供这个技巧), [csdn.net/Cyan_rose](https://www.csdn.net/Cyan_rose)

时间复杂度 $O(1)$, 压行之后:

```
ll a,b,mod;
cin>>a>>b>>mod;
```

```
cout<<(a*b-(ll)(round((double)a*b/mod))*mod+mod)%mod;
```

25. 卡特兰数

卡特兰数列

- 递推法: $O(n^2)$ 时间复杂度
- 组合数公式法: $O(n)$ 时间复杂度

```
#include <iostream>
using namespace std;

// 递推法  $O(n^2)$ 
long long catalan(int n) {
    if (n == 0 || n == 1) return 1;

    long long C[n + 1] = {0};
    C[0] = C[1] = 1;

    for (int i = 2; i <= n; ++i) {
        C[i] = 0;
        for (int j = 0; j < i; ++j) {
            C[i] += C[j] * C[i - j - 1];
        }
    }
    return C[n];
}
```

// 组合数公式法 $O(n)$

```
long long binomialCoeff(int n, int k) {
    if (k > n - k) k = n - k;
    long long res = 1;

    for (int i = 0; i < k; ++i) {
        res *= (n - i);
    }
}
```

```

        res /= (i + 1);
    }
    return res;
}

long long catalan_fast(int n) {
    long long c = binomialCoeff(2 * n, n);
    return c / (n + 1);
}

int main() {
    int n = 5;
    cout << "递推法 C(" << n << ") = " << catalan(n) << endl;
    cout << "组合数法 C(" << n << ") = " << catalan_fast(n) << endl;
    return 0;
}

```

26. 高精度

高精度加法

```

string jia(string s,string t){
    int a[10005]={},b[10005]={},c[10005]={};
    //数位对齐：s 和 t 倒过来放到数组里
    for(int i=0;i<s.size();i++) a[1+i]=s[s.size()-1-i]-'0';
    for(int i=0;i<t.size();i++) b[1+i]=t[t.size()-1-i]-'0';
    a[0]=s.size();
    b[0]=t.size();
    //从个位到高位对位加
    int maxn=max(a[0],b[0]),jw=0;
    for(int i=1;i<=maxn;i++)
    {
        c[i]=(a[i]+b[i]+jw)%10;
        jw=(a[i]+b[i]+jw)/10;//满十进一
    }
}

```

```

}

//处理最高位进位的小尾巴
if(jw>0) c[++maxn]=jw;
c[0]=maxn;
while(c[c[0]]==0&& c[0]>1) c[0]--;
//输出到 string
string res;
for(int i=c[0];i>=1;i--) res=res+char(c[i]+'0');
return res;
}

```

高精度减法

```

string jian(string s,string t){
    int a[10005]={},b[10005]={},c[10005]={};
    string res;
    //删除前导零
    while(s.size()>1&&s[0]=='0') s.erase(0,1);//s=s.substr(1);
    while(t.size()>1&&t[0]=='0') t.erase(0,1);
    //如果 s 数值<t 数值，输出负号，交换 s 和 t
    if(s.size()<t.size()||s.size()==t.size()&&s<t)
        res="-",swap(s,t);
    //把字符串倒过来放到数组里，数组的第一个元素里放位数
    for(int i=0;i<s.size();i++) a[i+1]=s[s.size()-1-i]-'0';
    a[0]=s.size();
    for(int i=0;i<t.size();i++) b[i+1]=t[t.size()-1-i]-'0';
    b[0]=t.size();
    //取最大位数
    int maxn=max(a[0],b[0]);
    //对位减
    for(int i=1;i<=maxn;i++)
    {
        c[i]=(a[i]-b[i]);
    }
}

```

```

        if(c[i]<0) a[i+1]--,c[i]+=10;
    }
    c[0]=maxn;
    //删除前导零，除非位数是1
    while(c[c[0]]==0&& c[0]>1) c[0]--;
    //输出到 string
    for(int i=c[0];i>=1;i--) res=res+char(c[i]+'0');
    return res;
}

```

高精度乘法

```

string cheng(string s,string t){
    int a[10005]={},b[10005]={},c[10005]={};
    //数位对齐：s 和 t 倒过来放到数组里
    for(int i=0;i<s.size();i++) a[1+i]=s[s.size()-1-i]-'0';
    for(int i=0;i<t.size();i++) b[1+i]=t[t.size()-1-i]-'0';
    a[0]=s.size();
    b[0]=t.size();
    //从个位到高位对位加
    int maxn=a[0]+b[0];
    //按位交叉乘
    for(int i=1;i<=a[0];i++) for(int j=1;j<=b[0];j++) c[i+j-1]+=a[i]*b[j];
    //处理进位
    for(int i=1;i<=maxn;i++){
        c[i+1]+=c[i]/10;
        c[i]%=10;
    }
    c[0]=maxn;
    while(c[c[0]]==0&& c[0]>1) c[0]--;
    //输出到 string
    string res;
    for(int i=c[0];i>=1;i--) res=res+char(c[i]+'0');
}

```

```
    return res;
}
```

带符号整数的加减乘除

```
void solve(){
    char op;
    string s,t;
    cin>>s;
    while(cin>>op>>t){
        if(op=='+'){
            if(s[0]=='-'&&t[0]=='-') s='-'+jia(s.substr(1),t.substr(1));
            else if(s[0]=='-'&&t[0]!='-') s=jian(t,s.substr(1));
            else if(s[0]!='-'&&t[0]=='-') s=jian(s,t.substr(1));
            else s=jia(s,t);
        }
        else if(op=='-'){
            if(s[0]=='-'&&t[0]=='-') s=jian(t.substr(1),s.substr(1));
            else if(s[0]=='-'&&t[0]!='-') s='-'+jia(s.substr(1),t);
            else if(s[0]!='-'&&t[0]=='-') s=jia(s,t.substr(1));
            else s=jian(s,t);
        }
        else if(op=='*'){
            if(s[0]=='-'&&t[0]=='-') s=cheng(t.substr(1),s.substr(1));
            else if(s[0]=='-'&&t[0]!='-') s='-'+cheng(s.substr(1),t);
            else if(s[0]!='-'&&t[0]=='-') s='-'+cheng(s,t.substr(1));
            else s=cheng(s,t);
        }
        cout<<s<<el;
    }

    return;
}
```

高精度小数除以低精度整数

将被除数分成两段分别除以高精度，先做整数部分，最后的余数要加到小数部分中。小数部分的结果需要按照要求四舍五入保留小数位数，需要多算一位，然后根据多算的这位的数字是否大于等于 5 来决定是否循环进位。为了方便循环进位，将结果保存在整数数组中。四舍五入后需要删除前导零。根据整数部分的长度记住小数点位置，输出的时候相应位置输出小数点。

```
string s1,s2;//s1: 整数部分, s2 小数部分
int t,ys,beichushu,n;//t: 除数, ys: 余数 beichushu: 被除数 n: 保留小数位数
vector<int> res;//保存结果数据, 方便四舍五入是循环进位
void solve(){
    cin>>s1>>t>>n;
    //用小数点拆分被除数
    int f=s1.find(".",0);
    if(f>0) s2=s1.substr(f+1),s1=s1.substr(0,f);
    //整数部分相除
    for(int i=0;i<s1.size();i++){
        beichushu=ys*10+s1[i]-'0';
        ys=beichushu%t;
        res.push_back(beichushu/t);
    }
    //小数部分相除, 注意整数部分的余数, 四舍五入保留 n 位, 需要多算一位小数
    for(int i=0;i<=n;i++){
        if(s2.size(>i) beichushu=ys*10+s2[i]-'0';
        else beichushu=ys*10;
        ys=beichushu%t;
        res.push_back(beichushu/t);
    }
    //四舍五入, 可能影响到整数部分, 所以要放在删除前导零之前。
    if(res[res.size()-1]>=5){
        res[res.size()-2]++;
        for(int i=res.size()-2;i>0;i--){
            if(res[i]>9){
                res[i-1]++;
                res[i]-=10;
            }
        }
    }
}
```

```

        }
        else break;
    }
}
res.pop_back();
int pos=s1.size(),begin=0;
//删除前导零
while(res[begin]==0&&pos-begin>1) begin++;
//正序输出
for(int i=begin;i<res.size();i++) {
    if(i==pos) cout<<'.';
    cout<<res[i];
}
return;
}
/* 3 组样例
输入：1234.5678 9 6
输出：137.174200
输入：9.9999999 10 4
输出：1.0000
输入：9.9999999 10 0
输出：1
*/

```

27. 双指针

双指针（Two Pointers）是一种常用的算法技巧，通过使用两个指针（或索引）在数据结构（如数组、链表）中协同遍历，达到高效解决问题的目的。它通常用于优化时间复杂度，将暴力解法从 $O(n^2)$ 降低到 $O(n)$ 或 $O(n \log n)$ 。

核心思想：

两个指针：一个指针从起点出发（左指针），另一个从终点或特定位置出发（右指针）。

协同移动：根据问题条件，动态调整指针的移动方向（相向、同向或背向）。

终止条件：当指针相遇或满足特定条件时停止。

类型	适用场景	示例问题
相向双指针	有序数组查找	两数之和、三数之和
快慢指针	链表问题	判断链表是否有环、找中点
滑动窗口	子串/子数组	最长无重复子串、最小覆盖子串
逆向双指针	合并数组	合并两个有序数组

链表中点和环的入口：

链表中点：从第一个节点开始，快指针每次走两步，慢指针每次走一步，快指针到达末尾时慢指针在中点。

环检测：从第一个节点开始，快慢指针相遇后，将其中一个指针重置到头部，两指针同速移动，再次相遇点为环入口。

几个双指针的应用场景：

给定一个数组 `nums` 和一个值 `val`，原地移除所有等于 `val` 的元素，返回新数组。

快指针遍历数组，慢指针记录非 `val` 元素的位置。

示例输入：[3,2,2,3], 3

示例输出：2

```
int removeElement(vector<int>& nums, int val) {
    int slow = 0;
    for (int fast = 0; fast < nums.size(); ++fast) {
        if (nums[fast] != val) nums[slow++] = nums[fast];
    }
    return slow;
}
```

给定一个非递减整数数组，返回每个元素的平方后按非递减顺序排序的结果。

双指针策略：左右指针从两端向中间移动，比较平方值后填充结果数组。

示例输入：[-4,-1,0,3,10]

示例输出：[0,1,9,16,100]

```
vector<int> sortedSquares(vector<int>& nums) {
    int n = nums.size(), left = 0, right = n - 1;
    vector<int> res(n);
    for (int i = n - 1; i >= 0; --i) {
        if (abs(nums[left]) > abs(nums[right])) res[i] = nums[left] * nums[left++];
    }
}
```

```

        else res[i] = nums[right] * nums[right--];
    }

    return res;
}

```

判断链表中是否有环。

双指针策略：快慢指针，快指针每次走两步，慢指针走一步，若相遇则有环。

示例输入：head = [3,2,0,-4] (有环)

示例输出：true

```

bool hasCycle(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}

```

给定一个整数数组 height，表示垂直线的高度，

找出两条线与 x 轴组成的容器能容纳最多的水。

双指针策略：左右指针向中间移动，每次移动较矮的一边以寻找更大面积。

示例输入：[1,8,6,2,5,4,8,3,7]

示例输出：49

```

int maxArea(vector<int>& height) {
    int left = 0, right = height.size() - 1, res = 0;
    while (left < right) {
        res = max(res, min(height[left], height[right]) * (right - left));
        if (height[left] < height[right]) left++;
        else right--;
    }
    return res;
}

```

```
}
```

找出数组中所有和为 0 的三元组（不重复）。

双指针策略：排序后固定一个数，用双指针在剩余数组中寻找两数之和。

示例输入：[-1,0,1,2,-1,-4]

示例输出：[[-1,-1,2],[-1,0,1]]

```
vector<vector<int>> threeSum(vector<int>& nums) {
    sort(nums.begin(), nums.end());
    vector<vector<int>> res;
    for (int i = 0; i < nums.size(); ++i) {
        if (i > 0 && nums[i] == nums[i - 1]) continue;
        int left = i + 1, right = nums.size() - 1;
        while (left < right) {
            int sum = nums[i] + nums[left] + nums[right];
            if (sum == 0) {
                res.push_back({nums[i], nums[left], nums[right]});
                while (left < right && nums[left] == nums[left + 1]) left++;
                while (left < right && nums[right] == nums[right - 1]) right--;
                left++; right--;
            } else if (sum < 0) left++;
            else right--;
        }
    }
    return res;
}
```

找出数组中所有和为 target 的四元组（不重复）。

双指针策略：排序后固定两个数，用双指针在剩余数组中寻找两数之和。

示例输入：[1,0,-1,0,-2,2], 0

示例输出：[[-2,-1,1,2],[-2,0,0,2],[-1,0,0,1]]

```
vector<vector<int>> fourSum(vector<int>& nums, int target) {
    sort(nums.begin(), nums.end());
```

```

vector<vector<int>> res;

for (int i = 0; i < nums.size(); ++i) {
    if (i > 0 && nums[i] == nums[i - 1]) continue;
    for (int j = i + 1; j < nums.size(); ++j) {
        if (j > i + 1 && nums[j] == nums[j - 1]) continue;
        int left = j + 1, right = nums.size() - 1;
        while (left < right) {
            long sum = (long)nums[i] + nums[j] + nums[left] + nums[right];
            if (sum == target) {
                res.push_back({nums[i], nums[j], nums[left], nums[right]});
                while (left < right && nums[left] == nums[left + 1]) left++;
                while (left < right && nums[right] == nums[right - 1]) right--;
                left++; right--;
            } else if (sum < target) left++;
            else right--;
        }
    }
}

return res;
}

```

将包含 0、1、2 的数组原地排序。

双指针策略：三指针（左、中、右）分区交换。

示例输入：[2,0,2,1,1,0]

示例输出：[0,0,1,1,2,2]

```

void sortColors(vector<int>& nums) {
    int left = 0, mid = 0, right = nums.size() - 1;
    while (mid <= right) {
        if (nums[mid] == 0) swap(nums[left++], nums[mid++]);
        else if (nums[mid] == 1) mid++;
        else swap(nums[mid], nums[right--]);
    }
}

```

```
}
```

在字符串 `s` 中找出包含字符串 `t` 所有字符的最小子串。

双指针策略：滑动窗口（左右指针维护窗口，哈希表记录字符需求）。

示例输入："`ADOBECODEBANC`", "`ABC`"

示例输出："`BANC`"

```
string minWindow(string s, string t) {
    unordered_map<char, int> need;
    for (char c : t) need[c]++;
    int left = 0, min_len = INT_MAX, start = 0, count = t.size();
    for (int right = 0; right < s.size(); ++right) {
        if (need[s[right]]-- > 0) count--;
        while (count == 0) {
            if (right - left + 1 < min_len) {
                min_len = right - left + 1;
                start = left;
            }
            if (need[s[left++]]++ == 0) count++;
        }
    }
    return min_len == INT_MAX ? "" : s.substr(start, min_len);
}
```

找出数组中满足和 $\geq$ `target` 的最短连续子数组

双指针策略：滑动窗口（右指针扩展，左指针收缩）。

示例输入：[`2,3,1,2,4,3`], `7`

示例输出：`2`

```
int minSubArrayLen(int target, vector<int>& nums) {
    int left = 0, sum = 0, res = INT_MAX;
    for (int right = 0; right < nums.size(); ++right) {
        sum += nums[right];
        while (sum >= target) {
```

```

        res = min(res, right - left + 1);

        sum -= nums[left++];
    }
}

return res == INT_MAX ? 0 : res;
}

```

找出字符串中不含有重复字符的最长子串长度。

双指针策略：滑动窗口（哈希集合记录字符位置）。

示例输入："abcabcbb"

示例输出：3（最长子串 "abc"）

```

int lengthOfLongestSubstring(string s) {
    unordered_map<char, int> last_pos;
    int left = 0, res = 0;
    for (int right = 0; right < s.size(); ++right) {
        if (last_pos.count(s[right])) left = max(left, last_pos[s[right]] + 1);
        last_pos[s[right]] = right;
        res = max(res, right - left + 1);
    }
    return res;
}

```

在字符串 `s` 中找出所有是 `p` 的字母异位词的字串起始索引。

输入格式：s = "cbaebabacd", p = "abc"

输出格式：[0,6]（子串 "cba" 和 "bac"）

双指针策略：滑动窗口（固定长度，哈希表记录字符计数）。

```

vector<int> findAnagrams(string s, string p) {
    vector<int> res, p_count(26, 0), window(26, 0);
    for (char c : p) p_count[c - 'a']++;
    for (int right = 0; right < s.size(); ++right) {
        window[s[right] - 'a']++;
        if (right >= p.size()) window[s[right - p.size()] - 'a']--;
    }
}

```

```

        if (window == p_count) res.push_back(right - p.size() + 1);
    }

    return res;
}

```

计算数组中乘积小于 k 的连续子数组的个数。

输入格式 : `nums = [10,5,2,6], k = 100`

输出格式 : 8 (子数组如 `[10], [5], [2], [6], [10,5], [5,2], [2,6], [5,2,6]`)

双指针策略 : 滑动窗口 (右指针扩展, 左指针收缩乘积)。

```

int numSubarrayProductLessThanK(vector<int>& nums, int k) {
    if (k <= 1) return 0;

    int left = 0, product = 1, res = 0;

    for (int right = 0; right < nums.size(); ++right) {
        product *= nums[right];

        while (product >= k) product /= nums[left++];

        res += right - left + 1;
    }

    return res;
}

```

允许替换 k 次字符后, 返回字符串中最长重复字符的子串长度。

示例输入 : `"ABAB", 2`

示例输出 : 4

双指针策略 : 滑动窗口 (维护窗口内最大重复字符数)。

```

int characterReplacement(string s, int k) {
    vector<int> count(26, 0);

    int left = 0, max_count = 0, res = 0;

    for (int right = 0; right < s.size(); ++right) {
        max_count = max(max_count, ++count[s[right] - 'A']);

        while (right - left + 1 - max_count > k) {
            count[s[left++] - 'A']--;
        }

        res = max(res, right - left + 1);
    }
}

```

```
    }  
    return res;  
}
```

从树序列中收集最多两种水果，返回能收集的最大数量。

输入格式 ： fruits = [1,2,1]

输出格式 ： 3（全部收集）

双指针策略 ： 滑动窗口（哈希表记录水果类型）。

```
int totalFruit(vector<int>& fruits) {  
    unordered_map<int, int> basket;  
    int left = 0, res = 0;  
    for (int right = 0; right < fruits.size(); ++right) {  
        basket[fruits[right]]++;  
        while (basket.size() > 2) {  
            if (--basket[fruits[left]] == 0) basket.erase(fruits[left]);  
            left++;  
        }  
        res = max(res, right - left + 1);  
    }  
    return res;  
}
```

给定数组和滑动窗口大小 k，返回每次窗口中的最大值。

示例输入 ： [1,3,-1,-3,5,3,6,7], 3

示例输出 ： [3,3,5,5,6,7]

双指针策略 ： 单调队列（维护窗口内递减序列）。

```
vector<int> maxSlidingWindow(vector<int>& nums, int k) {  
    deque<int> dq;  
    vector<int> res;  
    for (int right = 0; right < nums.size(); ++right) {  
        while (!dq.empty() && nums[right] >= nums[dq.back()]) dq.pop_back();  
        dq.push_back(right);  
        if (dq.front() <= right - k) dq.pop_front();  
    }  
    for (int i = k - 1; i < nums.size(); i++) res.push_back(nums[dq.front()]);  
    return res;  
}
```

```

        if (right >= k - 1) res.push_back(nums[dq.front()]);
    }

    return res;
}

```

找出数组中需要排序的最短子数组以使整个数组升序。

输入格式 : nums = [2,6,4,8,10,9,15]

输出格式 : 5 (排序 [6,4,8,10,9])

双指针策略 : 左右指针向中间移动, 寻找无序边界。

```

int findUnsortedSubarray(vector<int>& nums) {
    int n = nums.size(), left = 0, right = n - 1;
    while (left < n - 1 && nums[left] <= nums[left + 1]) left++;
    while (right > 0 && nums[right] >= nums[right - 1]) right--;
    if (left >= right) return 0;
    int min_val = *min_element(nums.begin() + left, nums.begin() + right + 1);
    int max_val = *max_element(nums.begin() + left, nums.begin() + right + 1);
    while (left >= 0 && nums[left] > min_val) left--;
    while (right < n && nums[right] < max_val) right++;
    return right - left - 1;
}

```

计算柱子排列后能接的雨水总量。

示例输入 : [0,1,0,2,1,0,1,3,2,1,2,1]

示例输出 : 6

双指针策略 : 左右指针向中间移动, 记录左右最大值计算水量。

```

int trap(vector<int>& height) {
    int left = 0, right = height.size() - 1;
    int left_max = 0, right_max = 0, res = 0;
    while (left < right) {
        if (height[left] < height[right]) {
            left_max = max(left_max, height[left]);
            res += left_max - height[left++];
        } else {

```

```

        right_max = max(right_max, height[right]);
        res += right_max - height[right--];
    }
}

return res;
}

```

在有序不重叠的区间列表中插入新区间

输入格式 : `intervals = [[1,3],[6,9]], newInterval = [2,5]`

输出格式 : `[[1,5],[6,9]]`

双指针策略 : 遍历区间列表, 处理重叠部分

```

vector<vector<int>> insert(vector<vector<int>>& intervals, vector<int>& newInterval) {
    vector<vector<int>> res;
    int i = 0, n = intervals.size();
    while(i < n && intervals[i][1] < newInterval[0]){
        res.push_back(intervals[i++]);
    }
    while(i < n && intervals[i][0] <= newInterval[1]){
        newInterval[0] = min(newInterval[0], intervals[i][0]);
        newInterval[1] = max(newInterval[1], intervals[i][1]);
        i++;
    }
    res.push_back(newInterval);
    while(i < n){
        res.push_back(intervals[i++]);
    }
    return res;
}

```

求两个区间列表的交集

示例输入 : `[[0,2],[5,10],[13,23],[24,25]], [[1,5],[8,12],[15,24],[25,26]]`

示例输出 : `[[1,2],[5,5],[8,10],[15,23],[24,24],[25,25]]`

双指针策略 : 双指针遍历两个列表, 计算交集

```

vector<vector<int>> intervalIntersection(vector<vector<int>>& A,
vector<vector<int>>& B) {
    vector<vector<int>> res;
    int i = 0, j = 0;
    while(i < A.size() && j < B.size()){
        int low = max(A[i][0], B[j][0]);
        int high = min(A[i][1], B[j][1]);
        if(low <= high){
            res.push_back({low, high});
        }
        if(A[i][1] < B[j][1]) i++;
        else j++;
    }
    return res;
}

```

将字符串划分为尽可能多的片段，使每个字母只出现在一个片段中

示例输入 : "ababcbacadefegdehijhklij"

示例输出 : [9,7,8]

双指针策略 : 记录字符最后出现位置，维护区间边界

```

vector<int> partitionLabels(string S) {
    vector<int> last(26, 0);
    for(int i=0; i<S.size(); i++){
        last[S[i]-'a'] = i;
    }
    vector<int> res;
    int start = 0, end = 0;
    for(int i=0; i<S.size(); i++){
        end = max(end, last[S[i]-'a']);
        if(i == end){
            res.push_back(end-start+1);
            start = end + 1;
        }
    }
}

```

```

    }

    return res;
}

```

计算需要的最少会议室数量

示例输入 : [[0,30],[5,10],[15,20]]

示例输出 : 2

双指针策略 : 分别排序开始和结束时间, 统计重叠

```

int minMeetingRooms(vector<vector<int>>& intervals) {
    if(intervals.empty()) return 0;
    vector<int> starts, ends;
    for(auto& interval : intervals){
        starts.push_back(interval[0]);
        ends.push_back(interval[1]);
    }
    sort(starts.begin(), starts.end());
    sort(ends.begin(), ends.end());
    int res = 0, end = 0;
    for(int i=0; i<starts.size(); i++){
        if(starts[i] < ends[end]) res++;
        else end++;
    }
    return res;
}

```

计算数组中恰好包含 k 个奇数的子数组数量

示例输入 : [1,1,2,1,1], 3

示例输出 : 2

双指针策略 : 滑动窗口统计奇数个数

```

int numberOfSubarrays(vector<int>& nums, int k) {
    int res = 0, count = 0, left = 0;
    for(int right=0; right<nums.size(); right++){
        if(nums[right] % 2 == 1) count++;
    }
}

```

```

while(count > k){
    if(nums[left++] % 2 == 1) count--;
}
if(count == k){
    res++;
    int temp = left;
    while(temp < right && nums[temp] % 2 == 0){
        res++;
        temp++;
    }
}
}
return res;
}

```

从数组两端取 k 张牌，求最大点数

输入格式 : cardPoints = [1,2,3,4,5,6,1], k = 3

输出格式 : 12

找最长子数组，其中最大绝对差不超过 limit

输入格式 : nums = [8,2,4,7], limit = 4

输出格式 : 2

双指针策略 : 滑动窗口维护最大最小值

```

int longestSubarray(vector<int>& nums, int limit) {
    deque<int> max_q, min_q;
    int left = 0, res = 0;
    for(int right=0; right<nums.size(); right++){
        while(!max_q.empty() && nums[right] > max_q.back()) max_q.pop_back();
        while(!min_q.empty() && nums[right] < min_q.back()) min_q.pop_back();
        max_q.push_back(nums[right]);
        min_q.push_back(nums[right]);
        while(max_q.front() - min_q.front() > limit){
            if(nums[left] == max_q.front()) max_q.pop_front();
            if(nums[left] == min_q.front()) min_q.pop_front();
        }
        res = max(res, right - left + 1);
    }
    return res;
}

```

```

        left++;
    }

    res = max(res, right-left+1);
}

return res;
}

```

找数组中最长的山脉子数组

示例输入 : [2,1,4,7,3,2,5]

示例输出 : 5

双指针策略 : 中心扩展法找山脉

```

int longestMountain(vector<int>& arr) {
    int res = 0, n = arr.size();
    for(int i=1; i<n-1; i++){
        if(arr[i] > arr[i-1] && arr[i] > arr[i+1]){
            int left = i-1, right = i+1;

            while(left > 0 && arr[left] > arr[left-1]) left--;
            while(right < n-1 && arr[right] > arr[right+1]) right++;

            res = max(res, right-left+1);
        }
    }

    return res;
}

```

找出链表中倒数第 k 个节点

输入格式 : head = [1,2,3,4,5], k = 2

输出格式 : 4

双指针策略 : 快指针先走 k 步, 然后同步移动

```

ListNode* getKthFromEnd(ListNode* head, int k) {
    ListNode *fast = head, *slow = head;

    while(k--) fast = fast->next;

    while(fast){
        fast = fast->next;
    }
}

```

```

        slow = slow->next;
    }

    return slow;
}

```

28. 埃氏筛法

```

bool a[1000005];
int n=1000000;
void prime_aishi(int n){
    a[0]=a[1]=1;//0 和 1 不是质数
    for(ll i=2;i<=n;i++){
        if(a[i]==0){
            for(ll j=i*i;j<=n;j+=i){ //i 和 j 都用 long long 以避免 i*i 时溢出
                a[j]=1;
            }
        }
    }
    for(int i=0;i<=n;i++)
        if(!a[i]) cout<<i<<" ";
}

```

用埃氏筛法思想可以实现求 1-n 的因数和。

29. 欧拉筛法

欧拉筛法除了求 1-n 范围内的质数外，还可以求每个合数的最小质因子、除本身之外的最大因子和最大因子对和。

```

bool a[5000005];
int n=1000000,cnt,primes[5000005];
void prime_oula(int n){
    a[0]=a[1]=1;//0 和 1 不是质数
    for(int i=2;i<=n;i++){
        if(a[i]==0){//如果 i 是质数
            primes[cnt++]=i;

```

```

    }

    for(int j=0;j<cnt&&primes[j]*i<=n;j++){
        a[primes[j]*i]=1;//i 的质数倍都不是质数
        if(i%primes[j]==0) break;//每个合数被最小质因数筛除
    }
}

for(int i=0;i<cnt;i++) cout<<primes[i]<<" ";
}

```

30. 桶排序

//桶排序除了排序外，桶的思想还可以用来计数和验证是否包含所有数字或字符等。

```

#include<bits/stdc++.h>
using namespace std;
string s="wgpjwpfwejfowejfpopiwtptweitpwjffafjapsofqjffhafqwhfqwodwq";
int a[30];//桶数组，下标是每个字符的 ASCII 值，桶数组元素的值是相应字符的数量。
int main(){
    for(int i=0;i<s.size();i++){
        a[s[i]-'a']++;
    }
    for(int i=0;i<26;i++){
        for(int j=0;j<a[i];j++){
            cout<<char(i+'a');
        }
    }
    return 0;
}

```

当放入桶中的数据不是可以作为数组下标的整数时，如不是整数或者数据范围超大，用 map 代替桶数组。

31. 归并排序

归并排序过程中可以求逆序对数量（**离线**）以及对逆序的变型如求每个数前面比它小的数的和等。

```
int a[15]={3,2,5,7,1,9,2,5,4,8},b[15],n=10;
```

```
void mergeSort(int l,int r){
    if(l<r){
        int mid=(l+r)/2;
        mergeSort(l,mid);
        mergeSort(mid+1,r);
        int i=l,j=mid+1,t=l;
        while(i<=mid&&j<=r){
            if(a[i]<=a[j]) b[t++]=a[i++];
            else b[t++]=a[j++];
        }
        while(i<=mid) b[t++]=a[i++];
        while(j<=r) b[t++]=a[j++];
        for(int s=l;s<=r;s++) a[s]=b[s];
    }
}
```

```
int main(){
    mergeSort(0,n-1);
    for(int i=0;i<n;i++) cout<<b[i]<<" ";
    return 0;
}
```

32. 快速排序

```
#include<bits/stdc++.h>
using namespace std;
int a[15]={3,2,5,7,1,9,2,5,4,8},b[15],n=10;
void qSort(int l,int r){
    int i=l,j=r,x=a[l];
    while(i<=j){
        while(a[i]<x) i++;
```

```

        while(a[j]>x) j--;
        if(i<=j) swap(a[i++],a[j--]);
    }
    if(l<j) qSort(l,j);
    if(i<r) qSort(i,r);
}

int main(){
    qSort(0,n-1);
    for(int i=0;i<n;i++) cout<<a[i]<<" ";
    return 0;
}

```

33. 二分

二分左闭右闭

```

#include<bits/stdc++.h>
using namespace std;
int a[15]={1,2,2,2,2,5,6,6,6,9},n=10;
int main(){
    int left=0,right=n-1,mid,x;
    cin>>x;
    while(left<=right){
        mid=(left+right)/2;
        if(x==a[mid]){
            break;
        }else if(x<a[mid])
            right=mid-1;
        else left=mid+1;
    }
    if(left<=right) cout<<"yes";
    else cout<<"no";
    return 0;
}

```

二分左闭右开

```
#include<bits/stdc++.h>
using namespace std;
int a[15]={1,2,2,2,2,5,6,6,6,9},n=10;
int main(){
    int left=0,right=n,mid,x;
    cin>>x;
    while(left<right){
        mid=(left+right)/2;
        if(x==a[mid]){
            break;
        }else if(x<a[mid])
            right=mid;
        else left=mid+1;
    }
    if(left<right) cout<<"yes";
    else cout<<"no";
    return 0;
}
```

等同 `lower_bound`，求 `x` 的左边界，即第一个 `>=x` 的位置。找到 `x` 要继续往左边找，`while` 结束后，`left` 就是第一个 `>=x` 的位置。

```
#include<bits/stdc++.h>
using namespace std;
int a[15]={1,2,2,2,2,5,6,6,6,9},n=10;
int main(){
    int left=0,right=n-1,mid,x;
    cin>>x;
    while(left<=right){
        mid=(left+right)/2;
        if(x==a[mid]){
            right=mid-1;
        }else if(x<a[mid])
            right=mid-1;
        else left=mid+1;
    }
    if(left<right) cout<<"yes";
    else cout<<"no";
    return 0;
}
```

```

        right=mid-1;
    else left=mid+1;
}

cout<<left;

return 0;
}

```

等同 `upper_bound`，求 x 的右边界，即第一个 $>x$ 的位置。找到 x 要继续往右找，`while` 结束后，`left` 是第一个 $>x$ 的位置。

```

#include<bits/stdc++.h>
using namespace std;
int a[15]={1,2,2,2,2,5,6,6,6,9},n=10;
int main(){
    int left=0,right=n-1,mid,x;
    cin>>x;
    while(left<=right){
        mid=(left+right)/2;
        if(x==a[mid]){
            left=mid+1;
        }else if(x<a[mid]){
            right=mid-1;
        }else left=mid+1;
    }
    cout<<left;

    return 0;
}

```

`upper_bound-lower_bound` 的结果是 x 的个数。对于原来就有序的数组，查找 x 的个数的时间复杂度是 $O(\log n)$ 。

34. 三分

三分法可以用于求函数在某个区间范围内的最值，如已知函数 $f(x)=-x^3x-x^3x+2^*x+8$ ，当 x 在 $[-1,2]$ 范围内有一个最大值，求最大值，保留 3 位小数。

```

#include<bits/stdc++.h>

```

```

using namespace std;

double f(double x){
    return -x*x*x-x*x+2*x+8;
}

int main(){
    double left=-1,right=2,lmid,rmid;
    while(right-left>0.0001){
        lmid=left+(right-left)/3;
        rmid=left+(right-left)*2/3;
        if(f(lmid)<f(rmid)) left=lmid;
        else right=rmid;
    }
    printf("%.3lf",left);
    return 0;
}

```

35. 二分答案

二分答案适用于具有单调性的问题。

```

//套路之第一部分：check 函数判 mid
bool check(ll mid){
    for(int i=1;i<=n;i++){
        //通过循环做一些事情，比如
        /*
        统计数量 cnt;
        求和累计 sum;
        符合条件的答案 ans
        */
        return ans<=P;//是否满足条件，大于或者小于，大于等于或者小于等于，等于
    }
}

int main(){
    //套路之第二部分，输入，l 和 r 的确定
    cin>>n>>m;

```

```

for(int i=1;i<=n;i++) cin>>a[i];//输入根据实际情况定夺

l=0;    //确定 left, 有可能 1, 有可能-1, 有可能是指定的值
r=1e9; //确定 right, 有可能是题目直接或者间接指定的 m,p 等信息, 有可能是范围最
大值

//套路之第三部分, 二分主体, 具体情况分两种, 见下方
while(l<=r){//这是左闭右闭的, 可以改成左闭右开的

    ll mid=l+((r-l)>>1);//避免越界

    if(check(mid)){

        ans=mid;//记录最后一个符合 check 的解

        l=mid+1;//根据题意变换区间范围 也可能是 r=mid-1

    }

    else

        r=mid-1;//根据题意变换区间范围 也可能是 l=mid+1

    }

    cout<<ans;

    return 0;

}

```

- 二分主体是求满足条件的**最大, 最多**的情况, 也就是查找最后一个小于等于 (或者说小于) Key 的情况

```

while(l<=r){

    ll mid=l+((r-l)>>1);

    if(check(mid)){

        ans=mid;

        l=mid+1;

    }

    else

        r=mid-1;

}

```

- 二分主体是求满足条件的**最少, 最节约, 恰好够用**等情况, 也就是查找第一个大于等于(或者说大于) key 的情况, 类似于 lower_bound(upper_bound)所要实现的目标的情况

```

while(l<=r){
    ll mid=l+((r-l)>>1);
    if(check(mid)){
        ans=mid;
        r=mid-1;
    }
    else
        l=mid+1;
}

```

- 如果用左闭右开区间，注意要将 r 的初始值、while(l<r)、r=mid 三处一起修改。

二分答案类问题的特点，求最大值的最小值，或者求最小值的最大值。

36. 一维数组前缀和与差分

一维数组前缀和：

$$s[i] = s[i-1] + a[i]$$

计算 i 从 L 到 R 时，a[i] 的累加和：

$$\text{区间和} = s[R] - s[L-1]$$

一维数组差分：

$$d[i] = a[i] - a[i-1]$$

将原数组 i 从 L 到 R 的每一个元素 a[i] 都增加 x

区间修改：d[L] += x; d[R+1] -= x;

37. 二维数组前缀和与差分

二维数组计算前缀和：

$$s[i][j] = s[i-1][j] + s[i][j-1] + a[i][j] - s[i-1][j-1];$$

计算二维区间和（计算原数组从 x1,y1 到 x2,y2 的累加和）：

$$\text{区间和} = s[x2][y2] - s[x1-1][y2] - s[x2][y1-1] + s[x1-1][y1-1]$$

二维数组计算差分：

```
d[i][j]=a[i][j]-a[i-1][j]-a[i][j-1]+a[i-1][j-1];
```

区间修改（原数组从 x_1, y_1 到 x_2, y_2 的每一项都 $+x$ ）:

```
d[x1][y1]+=x; d[x2+1][y2+1]+=x; d[x1][y2+1]-=x; d[x2+1][y1]-=x;
```

38. 最长上升子序列 LIS

最长上升子序列 LIS 与最长不降子序列解法类似，主要有两种：

方法一：动态规划 $O(n^2)$

状态定义： $dp[i]$ 表示最长上升子序列包含有第 i 个元素时的 LIS 长度

状态转移方程： $dp[i]=\max(dp[j]+1) \ a[j]<a[i]$

如果要输出最长上升子序列，可以通过 dp 数组，从后往前分析得到。如下：

代码块

```
#include<bits/stdc++.h>

using namespace std;

#define ll long long

const int N=1e4+5;

int a[N],n,ans;

void solve(){
    cin>>n;
    vector<int> dp(n,1);
    for(int i=0;i<n;i++) cin>>a[i];
    for(int i=1;i<n;i++){
        for(int j=0;j<i;j++){
            if(a[j]<a[i]) dp[i]=max(dp[i],dp[j]+1);
        }
        ans=max(ans,dp[i]);
    }
    cout<<ans;
    return;
}
```

也可以在打擂台时将 $dp[i]$ 更新时的前驱 j 记录到 pre 数组中，然后从 $dp[x]$ 最大的 x 开始，沿 $pre[x]$ 递推得到最长上升子序列的倒序结果。如果题目要求长度相同要字典序更小或者更大的，只需要修改 $pre[i]$ 赋值的条件即可。如下程序输出最长上升子序列字典序最大的结果。

代码块

```
#include<bits/stdc++.h>

using namespace std;

#define ll long long

const int N=1e4+5;

int a[N],n,ans,maxi;

void solve(){

    cin>>n;

    vector<int> dp(n,1);

    vector<int> anslist,pre(n);

    for(int i=0;i<n;i++) cin>>a[i];

    for(int i=1;i<n;i++){

        for(int j=0;j<i;j++){

            if(a[j]<a[i]) {

                if(dp[j]+1>dp[i]) {

                    dp[i]=dp[j]+1;

                    pre[i]=j;

                }

            }

        }

        if(dp[i]>ans) ans=dp[i],maxi=i;

    }

    cout<<ans<<endl;

    for(int i=0;i<ans;i++){

        anslist.push_back(maxi);

        maxi=pre[maxi];

    }

    for(int i=ans-1;i>=0;i--){

        cout<<a[anslist[i]]<<" ";

    }

    return;

}
```

方法二：二分 $O(n\log n)$

贪心+二分思想。分别写出以第 i 个元素结束的字典序最小的最长上升序列，如对于数组：

5 2 4 8 6 9 5

分别是：

5 从左到右找第一个大于等于 2 的换成 2 得到第二个元素结尾的 LIS， $dp[2]$ =换的这个位置是第几个元素=1

2 从左到右找第一个大于等于 4 的换成 4 得到第三个元素结尾的 LIS， $dp[3]$ =换的这个位置是第几个元素=2

2 4 左到右找第一个大于等于 8 的换成 8 得到第四个元素结尾的 LIS， $dp[4]=3$

2 4 8 左到右找第一个大于等于 6 的换成 6 得到第五个元素结尾的 LIS， $dp[5]=4$

2 4 6 找第一个大于等于 x 的位置，就是 `lower_bound`。如果是求最长不降子序列，就用 `upper_bound`。

2 4 6 9

2 4 5

依次类推，结果就是 $dp[0]-dp[1]$ 的最大值，也就是保存此上升序列的数组元素个数，实际上用不到 dp 数组。

注意最终上升序列数组中保存的不一定是正确的最长上升序列。如以上数据结束后是 2 4 5 9，而此上升序列不可能存在。

代码块

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long
const int N=1e4+5;
int a,n,ans;
vector<int>ord;
void solve(){
    cin>>n;
    for(int i=0;i<n;i++){
        cin>>a;
        int t=lower_bound(ord.begin(),ord.end(),a)-ord.begin();
        if(t==ord.size()){ //a 最大，找到 ord.end()了
            ord.push_back(a);
        }
    }
}
```

```

        else{
            ord[t]=a;
        }
    }

    cout<<ord.size()<<endl;

    return;
}

```

39. 最长公共子序列 LCS

最长公共子序列（LCS）是动态规划中的经典问题，用于找到两个序列共有的、相对顺序一致的最长子序列。

与最长公共子串（Longest Common Substring）不同，LCS 不要求连续，只需保持顺序一致。

动态规划解法 ($O(n^2)$)

状态定义

$dp[i][j]$: $X[0..i-1]$ 和 $Y[0..j-1]$ 的 LCS 长度。

转移方程

如果 $X[i-1] == Y[j-1]$ ，则 $dp[i][j] = dp[i-1][j-1] + 1$ 。

否则， $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$ 。

初始化

$dp[0][j] = 0$ （空字符串与任何字符串的 LCS 长度为 0）。

$dp[i][0] = 0$ 。

```

#include <vector>
#include <algorithm>

using namespace std;

int lcs(string X, string Y) {
    int m = X.size(), n = Y.size();
    vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));

    for (int i = 1; i <= m; ++i) {
        for (int j = 1; j <= n; ++j) {

```

```

        if (X[i-1] == Y[j-1]) {
            dp[i][j] = dp[i-1][j-1] + 1;
        } else {
            dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
        }
    }
}
return dp[m][n];
}

```

可以用滚动数组优化：

```

int lcs(string X, string Y) {
    int m = X.size(), n = Y.size();
    vector<int> dp(n + 1, 0);

    for (int i = 1; i <= m; ++i) {
        int prev = 0; // 保存 dp[i-1][j-1]
        for (int j = 1; j <= n; ++j) {
            int temp = dp[j]; // 保存 dp[i-1][j]
            if (X[i-1] == Y[j-1]) {
                dp[j] = prev + 1;
            } else {
                dp[j] = max(dp[j], dp[j-1]);
            }
            prev = temp; // 更新 prev 为 dp[i-1][j]
        }
    }
    return dp[n];
}

```

如果需要输出具体的 LCS，可以反向回溯 dp 表：

```

string getLCS(string X, string Y, vector<vector<int>>& dp) {

```

```

int i = X.size(), j = Y.size();
string lcs;

while (i > 0 && j > 0) {
    if (X[i-1] == Y[j-1]) {
        lcs.push_back(X[i-1]);
        i--, j--;
    } else if (dp[i-1][j] > dp[i][j-1]) {
        i--;
    } else {
        j--;
    }
}

reverse(lcs.begin(), lcs.end());
return lcs;
}

```

变型

最长公共子串 (Longest Common Substring)

- 要求子序列必须连续。
- 修改 dp 定义: $dp[i][j]$ 表示以 $X[i-1]$ 和 $Y[j-1]$ 结尾的最长公共子串长度。
- 转移方程:

```

if (X[i-1] == Y[j-1]) {
    dp[i][j] = dp[i-1][j-1] + 1;
} else {
    dp[i][j] = 0; // 不连续则重置为 0
}

```

40. 离散化

排序去重后, 使用元素的序号代替元素本身。如**区间求计数问题**: 给定学生的考试成绩 (0~100 分), 统计分数在 $[70, 90]$ 的人数。离散化后可使用前缀和和数组加速查询 $O(1)$, 将区间计数问题转化为了区间查询问题。

```

vector<int> discretize(vector<int>& arr) {
    vector<int> sorted = arr;
    sort(sorted.begin(), sorted.end()); // 排序
    sorted.erase(unique(sorted.begin(), sorted.end()), sorted.end()); // 去重

    vector<int> result;
    for (int x : arr) {
        int idx = lower_bound(sorted.begin(), sorted.end(), x) - sorted.begin() + 1; // 从 1
开始
        result.push_back(idx);
    }
    return result;
}

```

也可以用 `set` 排序和去重的特性，和 `unordered_map` 哈希查询 $O(1)$ 性质，建立一个原始值与离散值的映射字典。

```

#define ll long long
#define el '\n'
const ll MAXN=500005;
ll a[MAXN];
set<ll> st; // 用来排序和去重
unordered_map<ll,ll> dic; // 结果字典
ll n;
void solve(){
    cin>>n;
    for(int i=0;i<n;i++){
        cin>>a[i];
        st.insert(a[i]);
    }
    ll rank=1;
    for(ll x:st) dic[x]=rank++;
    for(int i=0;i<n;i++){
        cout<<a[i]<<" "<<dic[a[i]]<<el;
    }
}

```

```
}  
}
```

4.1. 单调栈

维护一个栈，要在把第 i 个元素进栈前，先弹出不具备单调性的栈顶。

求每个元素后面更大的数

也可以变型为求每个元素 x 的左边有多少个连续的元素比 x 小（或大）。每个元素最多进栈 1 次，时间复杂度 $O(n)$

```
int n=10,a[15]={14,2,3,4,5,5,5,6,7,8};  
    //单调栈：求 a[i]后面最先一个大于 a[i]的元素  
vector<int> res(n,-1); //保存结果  
stack<int> stk; //单调递减栈  
for(int i=0;i<n;i++){  
    while(!stk.empty()&&a[stk.top()]<a[i]){ //栈不为空并且栈顶比当前元素小，记录结果并  
        出栈  
        res[stk.top()]=a[i];  
        stk.pop();  
    }  
    stk.push(i);  
}  
for(int i=0;i<n;i++) cout<<res[i]<<" ";
```

求最大矩形面积

分析：通过维护一个**单调递增栈**，快速找到每个柱子右侧第一个比它矮的边界，矩形的高度就是栈顶元素的值，矩形的宽就是**出栈后的**栈顶元素到第一个小于栈顶元素间的元素数量： $i - \text{stk.top()} - 1$ 。打擂台求高 $\times$ 宽的最大值

```
int n=5,a[15]={0,2,4,3,4,3,0}; //数据前后补零，方便计算  
ll maxn=0; //防止爆 int  
vector<int> res(n,-1); //保存结果  
stack<int> stk; //单调递增栈  
stk.push(0);  
for(int i=1;i<=n+1;i++) { //前后补了零，注意循环范围
```

```

while(a[stk.top()]>a[i]){
    ll h=a[stk.top()];
    stk.pop();//注意这句和后面一句的先后顺序
    ll w=i-stk.top()-1;
    maxn=max(maxn,h*w);
}
stk.push(i);
}

cout<<maxn; // 输出

```

42. 单调队列

单调队列是一种高效处理滑动窗口最值问题的数据结构。单调队列是一种特殊的双端队列，它保持队列中元素的单调性（递增或递减）。这种结构可以在 $O(1)$ 时间内获取当前窗口的最值。

- 单调递减队列（求最大值）：队列头部始终是当前窗口的最大值
- 单调递增队列（求最小值）：队列头部始终是当前窗口的最小值

每次新元素加入时，都会从队列尾部开始移除破坏单调性的元素，确保队列始终保持单调性。

双端队列实现细节

- 使用 `deque` 而不是 `queue`，因为需要两端操作
- 存储下标而非值，便于判断元素是否过期
- 比较时通过下标访问实际值

```

#include <bits/stdc++.h>

using namespace std;

vector<int> maxSlidingWindow(vector<int>& nums, int k) {
    if (nums.empty() || k <= 0) return {};

    vector<int> result;
    deque<int> dq; // 存储的是元素下标

    for (int i = 0; i < nums.size(); ++i) {
        // 步骤 1: 移除超出窗口范围的元素
        // 检查队列头部元素是否还在当前窗口内
    }
}

```

```

// 窗口范围是[i-k+1, i], 所以小于 i-k+1 的都过期了
while (!dq.empty() && dq.front() < i - k + 1) {
    dq.pop_front();
}

// 步骤 2: 维护单调递减性质
// 从队列尾部开始, 移除所有小于当前元素的元素
// 因为这些元素不可能再成为窗口最大值了
while (!dq.empty() && nums[dq.back()] < nums[i]) {
    dq.pop_back();
}

// 步骤 3: 将当前元素加入队列
dq.push_back(i);

// 步骤 4: 记录结果 (当窗口形成后)
if (i >= k - 1) {
    result.push_back(nums[dq.front()]);
}
}

return result;
}

// 测试用例
int main() {
    vector<int> nums = {1,3,-1,-3,5,3,6,7};
    int k = 3;
    vector<int> result = maxSlidingWindow(nums, k);

    cout << "滑动窗口最大值: ";
    for (int num : result) {
        cout << num << " ";
    }
}

```

```

    }

    cout << endl;

    return 0;
}

```

43. 并查集函数

```

//初始化
void init(int n){
    for(int i=1;i<=n;i++){
        pre[i]=i;
    }
    return;
}

//寻根
int root(int x){
    if(pre[x]==x) return x;    //我的父亲结点是我自己，我就是自己的父亲结点
    return pre[x]=root(pre[x]); //路径压缩
}

//判断两个结点是否有联系
bool isConnected(int x,int y) {
    return root(x)==root(y);
}

//合并结点
void merge(int x,int y){
    pre[root(x)]=root(y);
}

```

44. KMP

```

#include <bits/stdc++.h>

using namespace std;

```

```

int main() {

    string s = "ababababcdabcbab", t = "abcdabc";

    int n = s.size(), m = t.size();

    // 计算 next 数组，以下是经典算法，还有另外两种写法，自学

    int next[m] = {0}; // next 数组，大小与 t 长度一致
    for (int i = 1, j = 0; i < m; i++) {
        while (j > 0 && t[i] != t[j]) j = next[j-1];
        if (t[i] == t[j]) {
            j++;
            next[i] = j;
        }
    }

    // 打印 next 数组（调试用）
    for (int i = 0; i < m; i++) {
        cout << next[i] << " ";
    }
    cout << endl;

    // KMP 匹配
    for (int i = 0, j = 0; i < n; i++) {
        while (j > 0 && s[i] != t[j]) j = next[j - 1];
        if (s[i] == t[j]) j++;
        if (j == m) {
            cout << "匹配成功，t 在 s 中的起始下标: " << i - m + 1 << endl;
            break;
        }
    }

    return 0;
}

```

45. 字符串哈希

双哈希降低冲突

代码块

```
#include<bits/stdc++.h>

using namespace std;

#define ll long long
#define el "\n"

const ll P1=131,P2=13331,MOD1=1e9+7,MOD2=1e9+9,N=1e5+5;
ll p1[N],p2[N],sh1[N],sh2[N],th1[N],th2[N];

void init(ll n){ //计算数组 p1 和 p2
    p1[0]=p2[0]=1;
    for(int i=1;i<=n;i++){
        p1[i]=(p1[i-1]*P1)%MOD1;
        p2[i]=(p2[i-1]*P2)%MOD2;
    }
}

void set_hash(string s,ll h1[],ll h2[]){ //计算字符串 s 的哈希， 分别放到 sh1 和 sh2
    for(int i=1;i<=s.size();i++){ //实际下标比相应字符下标大 1
        h1[i]=(h1[i-1]*P1+s[i-1])%MOD1;
        h2[i]=(h2[i-1]*P2+s[i-1])%MOD2;
    }
}

pair<ll,ll> get_hash(string &s,ll *h1,ll *h2,ll left,ll right){
    pair<ll,ll> res;
    res.first=(h1[right+1]-h1[left]*p1[right-left+1]%MOD1+MOD1)%MOD1;
    res.second=(h2[right+1]-h2[left]*p2[right-left+1]%MOD2+MOD2)%MOD2;
    return res;
}

void solve(){
    string s="gssssaaAAAAAAwfwwww";
    string t="AAAAA";
```

```

init(max(s.size(),t.size()));//计算数组 p1 和 p2

set_hash(s,sh1,sh2);//计算字符串 s 的哈希，分别放到 sh1 和 sh2

set_hash(t,th1,th2);//计算字符串 t 的哈希，分别放到 th1 和 th2

pair<ll,ll> t_hash=get_hash(t,th1,th2,0,t.size()-1),s_hash;

for(int i=0;i<=s.size()-t.size();i++)
{
    s_hash=get_hash(s,sh1,sh2,i,i+t.size()-1);

    if(s_hash==t_hash){

        cout<<i;

        return;

    }

}

return;
}

int main()
{

    solve();

    return 0;

}

```

46. 回文字符串高效算法

46.1 中心扩展法

中心扩展法的核心思想是：回文串可以从它的中心展开，并且总共有 $2n-1$ 个可能的中心（因为回文长度可能是奇数或偶数）。对于每个中心，我们向两边扩展，直到字符不匹配为止。

算法步骤

遍历所有可能的中心点：

对于长度为 n 的字符串，有 n 个奇数中心(单个字符)和 $n-1$ 个偶数中心(两个字符之间)

总共有 $2n-1$ 个可能的中心

从每个中心向两边扩展：

对于奇数长度回文，中心是一个字符

对于偶数长度回文，中心是两个字符之间

向左右两边扩展，直到字符不匹配或到达字符串边界

记录最长回文子串：

在扩展过程中，记录遇到的最长回文子串

```
#include <bits/stdc++.h>
using namespace std;

pair<int, int> expandAroundCenter(const string& s, int left, int right) {
    while (left >= 0 && right < s.size() && s[left] == s[right]) {
        left--;
        right++;
    }
    return {left + 1, right - 1};
}

string longestPalindrome(string s) {
    if (s.empty()) return "";

    int start = 0, end = 0;
    for (int i = 0; i < s.size(); i++) {
        // 处理奇数长度回文
        auto [left1, right1] = expandAroundCenter(s, i, i);
        // 处理偶数长度回文
        auto [left2, right2] = expandAroundCenter(s, i, i + 1);

        // 更新最长回文
        if (right1 - left1 > end - start) {
            start = left1;
            end = right1;
        }
        if (right2 - left2 > end - start) {
            start = left2;
            end = right2;
        }
    }

    return s.substr(start, end - start + 1);
}
```

```
}
```

46.2 Manacher 算法（马拉车）

马拉车算法是一种用于在线性时间内查找字符串中最长回文子串的高效算法。

马拉车算法的核心思想是利用回文的对称性质，通过维护一个"中心"和"右边界"来避免重复计算。算法通过预处理将字符串转换为统一奇偶长度的形式，然后利用对称性快速计算每个位置的回文半径。

算法步骤

预处理: 在字符串中插入特殊字符(如#)，使所有回文都变为奇数长度

例如: "abba" → "#a#b#b#a#"

初始化:

数组 P[]: 记录以每个字符为中心的最长回文半径

center: 当前中心位置

right: 当前回文的最右边界

主循环:

遍历处理后的字符串

利用对称性快速计算当前位置的初始回文半径

进行中心扩展

更新中心和右边界

```
#include <bits/stdc++.h>

using namespace std;

string longestPalindrome(string s) {
    if (s.empty()) return "";

    // 预处理字符串
    string t = "#";
    for (char c : s) {
        t += c;
        t += '#';
    }

    int n = t.size();
```

```

vector<int> P(n, 0);

int center = 0, right = 0;

int maxLen = 0, maxCenter = 0;

for (int i = 0; i < n; i++) {
    // 利用对称性快速计算初始半径 int mirror = 2 * center - i;
    if (i < right) {
        P[i] = min(right - i, P[mirror]);
    }

    // 尝试扩展 int left = i - (1 + P[i]);
    int r = i + (1 + P[i]);
    while (left >= 0 && r < n && t[left] == t[r]) {
        P[i]++;
        left--;
        r++;
    }

    // 更新中心和右边界 if (i + P[i] > right) {
        center = i;
        right = i + P[i];
    }

    // 记录最大值 if (P[i] > maxLen) {
        maxLen = P[i];
        maxCenter = i;
    }
}

// 提取最长回文子串 int start = (maxCenter - maxLen) / 2;
return s.substr(start, maxLen);
}

```

47. 最优问题总结

最优问题如最大值、最小值、最优解等，解题步骤如下：

1. 开始

→ 输入问题定义（目标函数、约束条件）。

2. 判断问题类型

- 是否具有贪心选择性质？
 - 是 → 使用贪心算法（如 Dijkstra、分数背包）。
 - 否 → 进入下一步。
- 是否具有最优子结构+重叠子问题？
 - 是 → 使用动态规划（如 0-1 背包、LCS）。
 - 否 → 进入下一步。
- 是否可以分治？
 - 是 → 使用分治法（如归并排序、快速排序）。
 - 否 → 尝试其他方法（如回溯、穷举）。

3. 验证算法正确性

- 贪心算法：需证明局部最优能导致全局最优。
- 动态规划：需验证状态转移方程的正确性。

4. 实现与优化

- 贪心：直接按策略实现（如排序+选择）。
- 动态规划：设计 DP 表或记忆化递归。

5. 输出最优解

→ 返回最大值/最小值或最优解。

6. 结束

贪心算法和动态规划算法的对比如图：

对比维度	贪心算法	动态规划
核心思想	每一步选择当前最优解，希望最终得到全局最优解。	通过子问题的最优解来构建当前问题的最优解，需要存储子问题的解。
适用条件	1. 贪心选择性质（Greedy Choice Property）：当前最优选择能导致全局最优。	1. 最优子结构（Optimal Substructure）：问题的最优解包含子问题的最优解。
	2. 无后效性：当前决策不影响未来决策。	2. 重叠子问题（Overlapping Subproblems）：子问题会被重复计算，需要存储。
计算方式	自顶向下（Top-Down），直接做出当前最优选择，不存储子问题的解。	自底向上（Bottom-Up）或带备忘录的自顶向下（Memoization），先解决子问题再解决原问题，存储子问题的解。

48. 贪心问题

遇到以下情况时，贪心算法可能不适用：

- 1. 需要全局信息：当前决策依赖后续信息
 - “必须知道后面所有元素才能决定当前选择”
- 2. 存在后效性：当前选择会影响后续选择空间
 - “选择某元素会永久改变问题结构”
- 3. 无法验证最优性：无法证明局部最优导致全局最优
 - “存在反例证明贪心策略不成立”
- 4. 复杂约束条件：多个相互制约的约束条件
 - “同时满足多个优化目标”

快速判断检查表：

- 问题要求最大/最小/最优解
- 可以分阶段做出局部选择
- 当前选择不影响后续选择空间
- 能定义明确的局部最优标准
- 无法构造出反例

分类	典型特征	贪心策略	
1. 区间调度类	- 涉及时间/空间区间	- 按结束时间排序（最早结束优先）	-
	- 需要选择或安排不冲突的区间	- 按开始时间排序（特定场景）	
2. 分配类	- 有限资源分配	- 小资源优先满足小需求（双指针）	
	- 满足特定约束的分配	- 大资源优先解决大问题	
3. 背包类	- 物品可分割	- 按单位价值排序（价值/重量比最高优先）	
	- 价值与重量关联		

48.1 区间调度类

通过排序转化选择顺序，避免冲突。如最多不重叠区间

```
#include <bits/stdc++.h>
using namespace std;

int maxNonOverlappingIntervals(vector<vector<int>>& intervals) {
    // 按结束时间升序排序
    sort(intervals.begin(), intervals.end(),
        [](const vector<int>& a, const vector<int>& b) {
            return a[1] < b[1];
        });

    int count = 0, last_end = INT_MIN;
    for (const auto& interval : intervals) {
        if (interval[0] >= last_end) {
            ++count;
            last_end = interval[1];
        }
    }
    return count;
}
```

48.2 分配类

双指针匹配需求与资源。如分发饼干

```
#include <bits/stdc++.h>

using namespace std;

int findContentChildren(vector<int>& g, vector<int>& s) {
    sort(g.begin(), g.end());
    sort(s.begin(), s.end());

    int child = 0, cookie = 0;
    while (child < g.size() && cookie < s.size()) {
        if (s[cookie] >= g[child]) ++child;
        ++cookie;
    }
    return child;
}
```

48.3 背包类

单位价值优先。如分数背包

```
#include <bits/stdc++.h>

using namespace std;

struct Item {
    int value;
    int weight;
};

double fractionalKnapsack(int W, vector<Item>& items) {
    // 按价值密度降序排序
    sort(items.begin(), items.end(),
        [](const Item& a, const Item& b) {
            return (double)a.value/a.weight > (double)b.value/b.weight;
        });
}
```

```

double total = 0.0;
for (const auto& item : items) {
    if (W >= item.weight) {
        total += item.value;
        W -= item.weight;
    } else {
        total += W * ((double)item.value/item.weight);
        break;
    }
}
return total;
}

```

48.4 路径/图类

(Kruskal 最小生成树) :

```

#include <bits/stdc++.h>
using namespace std;

struct Edge {
    int u, v, weight;
};

vector<int> parent;

int find(int x) {
    return parent[x] == x ? x : parent[x] = find(parent[x]);
}

int kruskalMST(vector<Edge>& edges, int n) {
    sort(edges.begin(), edges.end(),
        [](const Edge& a, const Edge& b) {

```

```

        return a.weight < b.weight;
    });

    parent.resize(n);
    iota(parent.begin(), parent.end(), 0);

    int total = 0;
    for (const auto& edge : edges) {
        int root_u = find(edge.u);
        int root_v = find(edge.v);
        if (root_u != root_v) {
            parent[root_u] = root_v;
            total += edge.weight;
        }
    }
    return total;
}

```

48.5 字符串处理类

删除 k 位使数字最小

```

#include <bits/stdc++.h>
using namespace std;

string removeKdigits(string num, int k) {
    string result;
    for (char digit : num) {
        while (!result.empty() && k > 0 && result.back() > digit) {
            result.pop_back();
            --k;
        }
        if (!result.empty() || digit != '0') {
            result.push_back(digit);
        }
    }
    if (k > 0) result.erase(result.begin(), result.begin() + k);
    return result.empty() ? "0" : result;
}

```

```

    }
}

while (k-- > 0 && !result.empty()) result.pop_back();

return result.empty() ? "0" : result;
}

```

48.6 序列处理类

LIS 优化（贪心+二分）

```

#include <bits/stdc++.h>
using namespace std;

int lengthOfLIS(vector<int>& nums) {
    vector<int> tails;
    for (int num : nums) {
        auto it = lower_bound(tails.begin(), tails.end(), num);
        if (it == tails.end()) tails.push_back(num);
        else *it = num;
    }
    return tails.size();
}

```

48.7 数学优化类

找零钱问题

```

#include <bits/stdc++.h>
using namespace std;

vector<int> coinChange(vector<int>& coins, int amount) {
    sort(coins.rbegin(), coins.rend());
    vector<int> result;
    for (int coin : coins) {
        while (amount >= coin) {

```

```

        amount -= coin;

        result.push_back(coin);
    }
}

return amount == 0 ? result : vector<int>();
}

```

48.8 区间合并

如合并重叠区间

```

#include <bits/stdc++.h>

using namespace std;

vector<vector<int>> mergeIntervals(vector<vector<int>>& intervals) {
    if (intervals.empty()) return {};

    // 按起始时间排序
    sort(intervals.begin(), intervals.end());

    vector<vector<int>> merged;
    merged.push_back(intervals[0]);
    for (const auto& interval : intervals) {
        if (interval[0] <= merged.back()[1]) {
            merged.back()[1] = max(merged.back()[1], interval[1]);
        } else {
            merged.push_back(interval);
        }
    }
    return merged;
}

```

48.9 特殊约束类贪心问题

带冷却时间的任务调度，求最短完成时间。

```

#include <bits/stdc++.h>

using namespace std;

int leastInterval(vector<char>& tasks, int n) {
    vector<int> count(26, 0);
    for (char task : tasks) {
        ++count[task - 'A'];
    }

    sort(count.begin(), count.end(), greater<int>());

    int max_count = count[0];
    int idle_slots = (max_count - 1) * n;

    for (int i = 1; i < 26 && count[i] > 0; ++i) {
        idle_slots -= min(count[i], max_count - 1);
    }

    return tasks.size() + max(0, idle_slots);
}

```

49. 深搜

```

void dfs(路径, 选择列表) {
    if(最优性剪枝条件) return;
    if (终止条件) {
        存放结果;
        return;
    }

    for (选择 : 选择列表) {
        if (可行性剪枝条件) continue; // 重要优化点
    }
}

```

```

    做选择;

    dfs(路径, 新选择列表);

    撤销选择; // 关键回溯步骤
}
}

```

可行性剪枝：当前路径明显不可能达到目标时提前返回

```
if (当前累计值 + 剩余最大值 < 目标值) return;
```

最优性剪枝：当前解已经差于已知最优解时提前返回

```
if (当前代价 >= 全局最小代价) return;
```

对称性剪枝：避免重复计算对称情况

去重剪枝：使用哈希表记录已访问状态

50. 深搜迷宫模版

问题类型：洪水填充、是否可达、最小步数、路径数量、最小步数路径、最小行走成本、最小行走成本路径

数据存储：地图数组 `mp`，访问标识数组 `vis`，最小步数（成本）数组 `val`，路径数据用 `deque` 模拟栈

需要思考：深搜类型是哪种，是否需要回溯和剪枝？

问题类型	洪水填充	是否可达	最小步数	路径数量	最短路径	最小路径成本	最小成本路径	
回溯	×	×	√	√	√	√	√	
剪枝	×	×	√	×	√	√	√	
边界条件	自然终止	达到终点	到达终点	到达终点	到达终点	到达终点	到达终点	
边界处理	无	exit(0)	打擂台	计数	打擂台，攻擂成功复制栈数据	累加和的最小值	复制栈数据	

实例一：求路径数量、最小步数（未剪枝）

```
#include<bits/stdc++.h>
```

```

using namespace std;

#define ll long long
#define ull unsigned long long

const int N=1e2+5;

ll mp[N][N],vis[N][N];

int n,m,sx,sy,ex,ey,cnt,minx=INT_MAX;

int dx[]={0,1,0,-1},dy[]={1,0,-1,0};

void dfs(int x,int y,int step){

    if(x==ex&&y==ey){

        //cnt++; //路径数量

        if(step<minx) minx=step; //最小步数

        return;

    }

    for(int i=0;i<4;i++){

        int xx=x+dx[i],yy=y+dy[i];

        if(xx>=1&&xx<=n&&yy>=1&&yy<=m&&vis[xx][yy]==0){ //没越界、地图可以走、未
走过+ (可行性剪枝)

            vis[xx][yy]=1;

            dfs(xx,yy,step+mp[xx][yy]);

            vis[xx][yy]=0; //回溯

        }

    }

}

void solve(){

    cin>>n>>m;

    sx=sy=1;

    ex=n;ey=m;

    for(int i=1;i<=n;i++) for(int j=1;j<=m;j++) cin>>mp[i][j];

    vis[sx][sy]=1;

    dfs(sx,sy,mp[sx][sy]);

    cout<<minx;

}

int main(){

```

```

ios::sync_with_stdio(0);

cin.tie(0);cout.tie(0);

//freopen("***.in","r",stdin);
//freopen("***.out","w",stdout);

int T=1;//cin>>T;

while(T--){

    solve();

}

return 0;

}

```

实例二：最小步数、最小路径+剪枝+回溯

```

#include<bits/stdc++.h>

using namespace std;

#define ll long long

#define ull unsigned long long

const int N=1e2+5;

int mp[N][N],vis[N][N],stp[N][N];

int n,m,sx,sy,ex,ey,minx=INT_MAX;

int dx[]={0,1,0,-1},dy[]={1,0,-1,0};

struct node{int x,y;};

deque<node> st,st2;

void dfs(int x,int y,int step){

    if(step>=minx) return ;//最优性剪枝

    if(x==ex&&y==ey){

        if(step<minx) {

            minx=step; //最小步数

            st2=st;

        }

        return;

    }

    for(int i=0;i<4;i++){

        int xx=x+dx[i],yy=y+dy[i];

```

```

if(xx>=1&&xx<=n&&yy>=1&&yy<=m&&mp[xx][yy]==0&&vis[xx][yy]==0&&(stp[xx][yy]==
0||step+1<stp[xx][yy]))
    {
        //没越界 可以走 没走过 可行性剪枝
        vis[xx][yy]=1;//标记已走
        st.push_back({xx,yy});//进栈
        stp[xx][yy]=step+1;//记录当前坐标的最小步数
        dfs(xx,yy,step+1);
        vis[xx][yy]=0;//回溯
        st.pop_back();//出栈
    }
}
}

void solve(){
    cin>>n>>m>>sx>>sy>>ex>>ey;
    for(int i=1;i<=n;i++) for(int j=1;j<=m;j++) cin>>mp[i][j];
    vis[sx][sy]=1;
    st.push_back({sx,sy});
    stp[sx][sy]=0;
    dfs(sx,sy,0);
    cout<<minx<<endl;
    for(int i=0;i<st2.size();i++) {
        cout<<"("<<st2[i].x<<","<<st2[i].y<<") ";
    }
}
}

```

51. 广搜

标准 BFS 模板

```

queue<Node> q;
unordered_set<Node> visited;

q.push(初始状态);
visited.insert(初始状态);

```

```

while (!q.empty()) {
    Node cur = q.front(); q.pop();

    if (到达目标) return 结果;

    for (每个相邻状态 next) {
        if (!visited.count(next)) {
            visited.insert(next);
            q.push(next);
        }
    }
}

```

双向 BFS 实现

```

queue<Node> q1, q2;
unordered_set<Node> visited1, visited2;
// 初始化两个方向的队列
q1.push(起点); visited1.insert(起点);
q2.push(终点); visited2.insert(终点);
while (!q1.empty() && !q2.empty()) {
    // 每次选择较小的队列扩展
    if (q1.size() <= q2.size()) {
        // 正向扩展...
        if (visited2.count(cur)) return 步数;
    } else {
        // 反向扩展...
        if (visited1.count(cur)) return 步数;
    }
}

```

多源 BFS 实现

```

queue<Node> q;

```

```

int dist[N][N];

memset(dist, -1, sizeof dist);

// 多个起点同时入队
for (auto source : sources) {
    q.push(source);
    dist[source.x][source.y] = 0;
}

// 标准 BF

```

52. 广搜迷宫模版

问题类型：洪水填充、是否可达、**最小步数、最小步数路径**。广搜因为找到的第一个就是最小步数的，所以特别适合求解最小步数、最小步数路径的问题。广搜用队列存储搜索过程数据，比较占空间，除非改用循环队列，要不一般不用来求解所有路径数量、最长路径、最小成本及最小成本路径等不能发挥其“找到第一个就是最优的”这个特点的问题。

数据存储：用队列存储搜索过程，注意：除了节点坐标外，还可能还需要存储当前几点步数和前驱节点编号等信息。

需要思考：队列里存储哪些数据，怎么获得最小步数路径。

应用场景：

1. **基本迷宫最短路径查找**：标准的最短路径问题，从起点到终点的最少步数。
2. **多出口迷宫的最优路径选择**：当存在多个出口时，找到最近的出口。
3. **动态障碍物迷宫**：迷宫中的障碍物随时间变化，BFS 需要适应动态环境。
4. **加权迷宫（不同移动成本）**：迷宫中某些区域移动成本更高，可能需要结合优先队列（类似 Dijkstra）。
5. **多目标点路径规划**：需要访问多个特定点的最优顺序。
6. **存在传送门或捷径的迷宫**：利用传送门快速移动，BFS 需要处理这种跳跃。
7. **三维或多层迷宫**：扩展至三维空间中的路径查找。
8. **敌人或移动障碍的规避**：在移动敌人存在的迷宫中，找到安全路径。
9. **迷宫生成算法的验证**：使用 BFS 测试生成的迷宫是否可解。
10. **游戏 AI 中的路径规划**：例如策略游戏中的单位移动。

广搜迷宫找最短路径的代码如下：

```

#include<bits/stdc++.h>

using namespace std;

#define ll int

```

```

const ll N=1e2+55;

struct zb{ll x,y,k;}; //k 是前驱节点的下标

bool mp[N][N];

bool vis[N][N];

ll n,m,res,dx[]={0,1,0,-1},dy[]={1,0,-1,0},sx,sy,ex,ey,head;

deque<zb> q; //为了避免 queue 不能随机访问的问题，可以用 deque 或者数组模拟队列，
            应为要输出结果，所以出队的元素不要删除。

void print(int h){ //递归输出路径结果
    if(h>0) print(q[h].k);
    printf("(%d,%d)->",q[h].x,q[h].y);
}

void bfs(){
    while(head<q.size()){
        //zb t=q.front();q.pop_front();不能用此方法，pop 之后就无法访问前驱节点了
        zb t=q[head];
        for(int i=0;i<4;i++){
            int xx=t.x+dx[i],yy=t.y+dy[i];
            if(xx==ex&&yy==ey){
                print(head);
                printf("(%d,%d)",ex,ey);
                return ;
            }
            if(xx>=1&&xx<=n&&yy>=1&&yy<=m&&mp[xx][yy]==0&&vis[xx][yy]==0){
                vis[xx][yy]=1;
                q.push_back({xx,yy,head});
            }
        }
        head++;
    }

    cout<<"no way";
}

void solve(){
    cin>>n>>m;

```

```

for(int i=1;i<=n;i++) for(int j=1;j<=m;j++) cin>>mp[i][j];

cin>>sx>>sy>>ex>>ey;

vis[sx][sy]=1;

q.push_back({sx,sy,-1}); //起点入队

bfs();

}

```

54. 动态规划

54.1 线性 DP

1. 定义

线性 DP 是指问题的状态可以按照线性顺序（如时间顺序、空间顺序等）进行排列，每个状态只与前面有限个状态相关的动态规划问题。

2. 核心特征

- **线性结构**：问题可以分解为一系列线性排列的阶段
- **状态转移**：当前状态只与前面一个或几个状态有关
- **无后效性**：当前状态一旦确定，后续决策不受之前决策影响

3. 与其它 DP 的区别

特征	线性DP	区间DP	树形DP
状态结构	线性序列	区间范围	树形结构
转移方向	单向（前→后）	由小区间到大区间	自底向上或自顶向下
典型问题	LIS、背包问题	石子合并、矩阵链乘法	树的最大独立集

4. 四步解题法

- **定义状态**：明确 `dp[i]` 或 `dp[i][j]` 表示的含义
- **状态转移**：建立状态之间的关系式（递推公式）
- **边界条件**：确定最小子问题的解（初始化）
- **计算顺序**：确定填表顺序以满足无后效性

5. 代码通用模板

```

// 以线性 DP 为例

int solveDP(vector<int>& input) {
    // 1. 初始化 DP 数组
    vector<int> dp(input.size());
}

```

```

// 2. 设置边界条件
dp[0] = initial_value;

// 3. 状态转移
for (int i = 1; i < input.size(); ++i) {
    dp[i] = transfer_function(dp[...], ...);
}

// 4. 返回结果
return dp.back(); // 或其他特定位置
}

```

6. 线性 DP 的经典模型

单序列模型

特点：只有一个输入序列，状态定义通常为 $dp[i]$ 表示以第 i 个元素结尾的子问题的解。如最长上升子序列(LIS)和最大子数组和等问题。

```

//LIS 问题状态转移方程
dp[i] = max(dp[j]) + 1, 其中  $0 \leq j < i$  且  $nums[j] < nums[i]$ 

//最大子数组和状态转移方程
dp[i] = max(nums[i], dp[i-1] + nums[i])

```

双序列模型

特点：有两个输入序列，状态定义通常为 $dp[i][j]$ 表示处理到第一个序列第 i 个元素和第二个序列第 j 个元素时的解。如最长公共子序列(LCS)和编辑距离问题（给定两个单词，计算将 word1 转换成 word2 所使用的最少操作数（插入、删除、替换））。

```

//LCS 问题状态定义和状态转移方程
dp[i][j]: text1[0..i-1]和 text2[0..j-1]的 LCS 长度
dp[i][j] = dp[i-1][j-1] + 1,          if text1[i-1] == text2[j-1]
          = max(dp[i-1][j], dp[i][j-1]), otherwise

int longestCommonSubsequence(string text1, string text2) {
    vector<vector<int>> dp(text1.size()+1, vector<int>(text2.size()+1, 0));
}

```

```

for (int i = 1; i <= text1.size(); ++i) {
    for (int j = 1; j <= text2.size(); ++j) {
        if (text1[i-1] == text2[j-1]) {
            dp[i][j] = dp[i-1][j-1] + 1;
        } else {
            dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
        }
    }
}

return dp[text1.size()][text2.size()];
}

```

//编辑距离问题状态定义和状态转移方程

$dp[i][j]$: 将 word1[0..i-1]转换为 word2[0..j-1]的最小操作数

$dp[i][j] = dp[i-1][j-1],$ if word1[i-1] == word2[j-1]
 $= \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1,$ otherwise

i

```

int minDistance(string word1, string word2) {
    vector<vector<int>> dp(word1.size()+1, vector<int>(word2.size()+1, 0));

    for (int i = 0; i <= word1.size(); ++i) dp[i][0] = i;
    for (int j = 0; j <= word2.size(); ++j) dp[0][j] = j;

    for (int i = 1; i <= word1.size(); ++i) {
        for (int j = 1; j <= word2.size(); ++j) {
            if (word1[i-1] == word2[j-1]) {
                dp[i][j] = dp[i-1][j-1];
            } else {
                dp[i][j] = min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]}) + 1;
            }
        }
    }
}

```

```
}

return dp[word1.size()][word2.size()];
}
```

背包模型

见背包专题

7. 状态设计技巧

- 前缀型 : `dp[i]`表示考虑前 i 个元素时的解
- 位置型 : `dp[i]`表示以第 i 个元素结尾时的解
- 状态机型 : 增加维度表示不同状态（如买卖股票问题）

8. 转移方程构建

- 分析问题的最优子结构
- 考虑所有可能的转移情况
- 确保转移满足无后效性

9. 边界条件处理

- 空序列情况
- 单元素情况
- 非法状态处理

10. 常见错误

- 状态定义不明确导致转移方程错误
- 边界条件初始化不正确
- 空间优化时覆盖了还需要使用的状态
- 计算顺序不符合状态依赖关系

11. 调试技巧

- 打印 DP 表格观察状态变化
- 用小规模数据手动验证
- 对比暴力解法的结果
- 检查转移条件和边界条件

掌握线性 DP 需要理解+记忆+大量练习，建议从经典模型入手，逐步建立解题直觉和模板库。

54.2 区间 DP

区间动态规划是动态规划的一种特殊形式，主要用于解决涉及区间或子序列的问题。

区间动态规划通常用于解决以下类型的问题：

- 涉及区间合并、分割或操作的问题
- 需要计算区间最优解的问题
- 涉及子序列或子数组的最优解问题

区间 DP 通常有以下特点：

- 状态定义：dp[i][j]表示区间[i,j]的最优解
- 状态转移：通常通过枚举区间内的分割点 k，将问题分解为子区间
- 初始化：对角线或者首行首列上的值通常需要初始化
- 计算顺序：通常按区间长度从小到大计算

区间 DP 的优化技巧：

- 四边形不等式优化：可以将某些区间 DP 的时间复杂度从 $O(n^3)$ 降低到 $O(n^2)$
- 滚动数组：当状态转移只依赖于相邻状态时，可以优化空间复杂度
- 记忆化搜索：有时比自底向上 DP 更容易实现和理解

问题举例

问题定义：

有 N 堆石子排成一排，每堆石子有一定的数量。现在要将这些石子合并成一堆。每次只能合并相邻的两堆石子，合并的代价为这两堆石子的数量之和。找出将所有石子合并成一堆的最小总代价。

示例：

输入：[4, 1, 1, 4]

输出：18

解释：

1. 合并第 2 和第 3 堆：代价 $1+1=2 \rightarrow [4, 2, 4]$
2. 合并第 1 和第 2 堆：代价 $4+2=6 \rightarrow [6, 4]$
3. 合并最后两堆：代价 $6+4=10 \rightarrow [10]$

总代价： $2 + 6 + 10 = 18$

问题分析

1. 关键观察
 - 每次合并都会减少一堆石子
 - 最终需要合并 N-1 次
 - 合并顺序不同会导致总代价不同
 - 需要找到最优的合并顺序
2. 为什么用区间 DP?
 - 问题可以分解为子区间的合并问题
 - 具有最优子结构性质

- 子问题之间有重叠

动态规划解法

1. 状态定义

定义 $dp[i][j]$ 为合并第 i 堆到第 j 堆石子的最小代价。

2. 状态转移方程

对于区间 $[i, j]$ ，我们可以枚举所有可能的分割点 k ($i \leq k < j$)，将区间分为 $[i, k]$ 和 $[k+1, j]$ 两部分：

$$dp[i][j] = \min(dp[i][k] + dp[k+1][j] + sum[i][j]) \text{ 对所有 } k \in [i, j-1]$$

其中 $sum[i][j]$ 是第 i 堆到第 j 堆石子的总数。

3. 初始化

- $dp[i][i] = 0$ (单堆石子不需要合并，代价为 0)
- 其他 $dp[i][j]$ 初始化为一个大数 (如 INT_MAX)

4. 计算顺序

按区间长度从小到大计算：

1. 先计算所有长度为 2 的区间
2. 然后计算长度为 3 的区间
3. 依此类推，直到长度为 N

```
#include <bits/stdc++.h>

using namespace std;

int stoneMerge(const vector<int>& stones) {
    int n = stones.size();

    // 前缀和数组，方便快速计算区间和
    vector<int> prefix(n + 1, 0);
    for (int i = 0; i < n; ++i) {
        prefix[i+1] = prefix[i] + stones[i];
    }

    // DP 表，dp[i][j]表示合并 i 到 j 堆的最小代价
    vector<vector<int>> dp(n, vector<int>(n, 0));
```

```

// 按区间长度从小到大计算
for (int len = 2; len <= n; ++len) {
    for (int i = 0; i <= n - len; ++i) {
        int j = i + len - 1;
        dp[i][j] = INT_MAX;

        // 枚举所有可能的分割点
        for (int k = i; k < j; ++k) {
            int cost = dp[i][k] + dp[k+1][j] + prefix[j+1] - prefix[i];
            if (cost < dp[i][j]) {
                dp[i][j] = cost;
            }
        }
    }
}

return dp[0][n-1];
}

int main() {
    vector<int> stones = {4, 1, 1, 4};
    cout << "Minimum merge cost: " << stoneMerge(stones) << endl;
    return 0;
}

```

54.3 树形 DP

1. 定义

树形 DP 是指在树形结构上进行的动态规划，通过后序遍历或自底向上的方式，将子树的信息传递给父节点，最终得到整棵树的解。

2. 核心特征

- 树形结构：问题建立在树或类似树的结构上
- 递归性质：子树问题与父树问题性质相同

- 无后效性 : 子树一旦处理完成, 其状态不再受父树影响

3. 基本步骤

- 建树 : 根据输入构建树结构 (邻接表表示)
- 状态定义 : 明确每个节点需要维护的状态
- DFS 遍历 : 通过后序遍历获取子树信息
- 状态转移 : 合并子树信息计算当前节点状态
- 结果提取 : 从根节点或所有节点中获取最终结果

4. 通用模版

```
vector<vector<int>> tree; // 树的邻接表表示
```

```
void dfs(int u, int parent) {
```

```
    // 初始化状态
```

```
    dp[u][0] = ...;
```

```
    dp[u][1] = ...;
```

```
    for (int v : tree[u]) {
```

```
        if (v == parent) continue; // 避免回父节点
```

```
        dfs(v, u); // 递归处理子树
```

```
        // 状态转移
```

```
        dp[u][0] += ...;
```

```
        dp[u][1] += ...;
```

```
    }
```

```
}
```

```
int solveTreeDP() {
```

```
    // 1. 建树
```

```
    // 2. 初始化 DP 数组
```

```
    // 3. 从根开始 DFS
```

```
    dfs(root, -1);
```

```
    // 4. 返回结果
```

```
return ...;
}
```

55. 背包（未完）

55.1 01 背包

问题特点

- 每件物品只能选或不选（不能分割，不能重复选）。
- 目标：在不超过背包容量的前提下，使装入物品的总价值最大。
- 典型应用：资源分配、投资组合优化、任务调度等。

问题描述

- 背包容量 W （最大承重）。
- n 件物品，每件物品有：
 - 重量 $w[i]$
 - 价值 $v[i]$

求：选择哪些物品装入背包，使得总重量不超过 W ，且总价值最大。

数据推导

物品\容量			0	1	2	3	4	5	6
无物品	体 积	价 值	0	0	0	0	0	0	0
物品1	2	3	0	0	3	3	3	3	3
物品2	3	4	0	0	3	4	4	7	7
物品3	4	5	0	0	3	4	5	7	8
物品4	5	6	0	0	3	4	5	7	8

状态转移方程

1. 二维 DP

$dp[i][j]$: 前 i 件物品，背包容量 j 时的最大价值。

状态转移：

不选第 i 件物品： $dp[i][j] = dp[i-1][j]$

选第 i 件物品（如果 $j \geq w[i]$ ）： $dp[i][j] = dp[i-1][j-w[i]] + v[i]$

最终方程：

$$dp[i][j] = \begin{cases} \max(dp[i-1][j], dp[i-1][j-w[i]] + v[i]) & \text{if } j \geq w[i] \\ dp[i-1][j] & \text{otherwise} \end{cases}$$

因为有二维 dp 数组，可以通过逆推得到最优解。if(dp[n][W]!=dp[n-1][W]) 就选择了物品 n。

```

// n,W; //物品数量和背包总容量
vector<ll> v,w,select;
void solve()
{
    cin>>n>>W;
    v.resize(n+1);
    w.resize(n+1);
    vector<vector<ll>> dp(n+1,vector<ll>(W+1,0));
    for(int i=1;i<=n;i++){
        cin>>w[i]>>v[i];
        for(int j=1;j<=W;j++){
            if(j>=w[i]) dp[i][j]=max(dp[i][j],dp[i-1][j-w[i]]+v[i]);
            else dp[i][j]=dp[i-1][j];
        }
    }
    cout<<dp[n][W]<<endl;
    //逆推最优解: if(dp[n][W]!=dp[n-1][W]) 就选择了物品 n。
    while(W){
        if(dp[n][W]!=dp[n-1][W]) {
            select.push_back(n);
            W-=w[n];
        }
        n--;
    }
    for(int i=0;i<select.size();i++) cout<<select[i]<<" ";
}
/*
in:
4 8

```

```

2 3
3 4
4 5
5 6
out:
10
4 2
*/

```

2.一维 DP 优化（滚动数组）

- 由于 $dp[i][j]$ 只依赖 $dp[i-1][\leq j]$ ，即只依赖上一行的左上方，可以优化为 $dp[j]$ ，只要逆序遍历即可。
- 关键点：****逆序遍历 j****（防止重复计算）。
- 状态转移： $dp[j]=\max(dp[j], dp[j-w[i]]+v[i])$

因为 dp 数组没有详细的数据，所以无法得到最优解。

```

ll n,W;//物品数量和背包总容量
vector<ll> v,w,select;
void solve()
{
    cin>>n>>W;
    v.resize(n+1);
    w.resize(n+1);
    vector<ll> dp(n+1);
    for(int i=1;i<=n;i++){
        cin>>w[i]>>v[i];
        for(int j=W;j>=w[i];j--){
            dp[j]=max(dp[j],dp[j-w[i]]+v[i]);
        }
    }
    cout<<dp[W]<<endl;
}

```

时间复杂度：一维 DP 是标准解法，适用于大多数情况。

方法	时间复杂度	空间复杂度	适用场景
二维DP	$O(nW)$	$O(nW)$	需要记录完整DP过程
一维DP（滚动数组）	$O(nW)$	$O(W)$	仅求最大值，节省空间

55.2 完全背包

问题特点

- 每件物品可以选无限次（区别于 0-1 背包）
- 目标：在不超过背包容量的前提下，使装入物品的总价值最大
- 典型应用：硬币兑换、资源无限供应场景

问题描述

- 背包容量 W
- n 种物品，每种物品有：
 - 重量 $w[i]$
 - 价值 $v[i]$

求：如何选择物品（每种可选无限次）装入背包，使总价值最大且总重量 $\leq W$

数据推导

物品\容量	0	1	2	3	4	5
初始化	0	0	0	0	0	0
物品1	0	1	2	3	4	5
(w=1,v=1)		(0+1)	(1+1)	(2+1)	(3+1)	(4+1)
物品2	0	1	2	4	5	6
(w=3,v=4)				max(3,0+4)	max(4,1+4)	max(5,2+4)
物品3	0	1	2	4	5	6
(w=4,v=5)					max(5,0+5)	max(6,1+5)

状态转移方程

1. 二维 DP 定义

- $dp[i][j]$: 前 i 种物品，容量 j 时的最大价值
- 状态转移：
 - 不选第 i 种物品： $dp[i][j] = dp[i-1][j]$
 - 选 k 个第 i 种物品 ($k \geq 1$):

$$dp[i][j] = \max(dp[i-1][j-k*w[i]] + k*v[i])$$

- 优化后的方程：

$$dp[i][j] = \begin{cases} \max(dp[i-1][j], dp[i][j-w[i]] + v[i]) & \text{if } j \geq w[i] \\ dp[i-1][j] & \text{otherwise} \end{cases}$$

- 最优解：通过二维 dp 数组，可以分析出一个最优解，从 $dp[n][W]$ 倒推，如果 $dp[i][j] \neq dp[i-1][j]$ 时，可以说明选择了 1 个第 i 个物品，用此方法倒推回去即可得到其中一个最优解。如果 $dp[i][j] = dp[i-1][j]$ ，可以选择 1 个第 i 个物品（注意需要满足选择第 i 个物品的条件）也可以不选择第 i 个物品，所以会产生多组解，通过深搜可以得到所有最优解。

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long
const ll MAXN=1e3+5;
ll dp[MAXN][MAXN],w[MAXN],v[MAXN]; //dp 数组，物品重量，物品价值
ll n,W,cnt;
deque<ll> stk; //用 deque 模拟栈，方便遍历输出结果
void print(){
    cout<<cnt<<":";
    for(int i=0;i<stk.size();i++) cout<<stk[i]<<" ";
    cout<<endl;
}
void dfs(ll x,ll y){
    if(y==0){
        print(); //输出搜索到的结果
        return ;
    }
    if(dp[x][y]!=dp[x-1][y]){ //一定选择 1 个第 x 个物品
        stk.push_back(x); //用栈存储结果
        dfs(x,y-w[x]); //继续深搜
        stk.pop_back(); //回溯
    }else{
        //满足选择第 x 个物品的条件就选择 1 个第 x 个物品
        if(y>=w[x]&&dp[x][y]>=v[x]){
```

```

        stk.push_back(x);//用栈存储结果

        dfs(x,y-w[x]);

        stk.pop_back();//回溯
    }

    //不选择第 x 个物品
    dfs(x-1,y);
}

}

void solve(){
    cin>>n>>W;
    for(int i=1;i<=n;i++) cin>>w[i]>>v[i];
    for(int i=1;i<=n;i++){
        for(int j=1;j<=W;j++){
            if(j<=w[i]){
                dp[i][j]=max(dp[i-1][j],dp[i][j-w[i]]+v[i]);
            }else{
                dp[i][j]=dp[i-1][j];
            }
        }
    }

    //输出 dp 数组
    for(int i=0;i<=n;i++){
        for(int j=0;j<=W;j++){
            cout<<setw(2)<<dp[i][j];
        }

        cout<<endl;
    }

    //输出最优值
    cout<<dp[n][W]<<endl;

    //倒推所有最优解
    dfs(n,W);
}

/*

```

```
3 7
1 1
3 4
4 5
*/
```

2. 一维 DP 优化

- 在二维 dp 数组中，我们发现动态转移方程 $dp[i][j]$ 只与当前项左方和正上方的值有关，我们可以使用两行 $W+1$ 列的二维数组来完成递推过程。再进一步可以用一维数组顺序遍历来保证左侧的数据已经更新之后再算当前的项。所以优化成一维数组 $dp[j]$ ，顺序遍历（允许重复选择）
- 关键转移： $dp[j]=\max(dp[j], dp[j-w[i]]+v[i])$
- 优化之后，因为省略的过程数据，所以无法得到最优解，只能计算最优值。

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long
const ll MAXN=1e3+5,MAXW=1e4+5;
ll dp[MAXW],w[MAXN],v[MAXN]; //dp 数组，物品重量，物品价值
ll n,W,cnt;

void solve(){
    cin>>n>>W;
    for(int i=1;i<=n;i++){
        cin>>w[i]>>v[i];
        for(int j=w[i];j<=W;j++){
            dp[j]=max(dp[j],dp[j-w[i]]+v[i]);
        }
    }
    cout<<dp[W];
}

/*
3 7
1 1
3 4
*/
```

4 5

*/

55.3 多重背包

给定 N 种物品和一个容量为 W 的背包，每种物品 i 有：重量 w_i 价值 v_i 数量限制 s_i （最多能选 s_i 个），求在不超过背包容量的前提下，能获得的最大价值。

解法主要有三种：

A. 三维动态规划

在完全背包的基础上，增加一层选择 i 物品数量 k 的循环，从 0 开始。

```
for (int i = 1; i <= n; ++i) {
    for (int j = 0; j <= W; ++j) {
        for (int k = 0; k <= s[i-1] && k * w[i-1] <= j; ++k) {
            dp[i][j] = max(dp[i][j], dp[i-1][j - k*w[i-1]] + k*v[i-1]);
        }
    }
}
```

B. 用二进制优化划归 01 背包

将每种物品的 s_i 个拆分为 $1, 2, 4, \dots, 2^k, c$ （其中 $c = s_i - (2^{k+1} - 1)$ ）的组合，转化为 01 背包问题

示例：若 $s_i=13$ ，可拆分为：1, 2, 4, 6（因为 $1+2+4+6=13$ ，二 1 2 4 6 四个数组可以加出 1-13 的每个值）

优化原理：

- 任何数字都可以由 2 的幂次组合表示
- 将 $O(S)$ 的枚举优化为 $O(\log S)$

```
int binaryOptimized(int W, vector<int>& w, vector<int>& v, vector<int>& s) {
    vector<pair<int,int>> items; // (weight, value)

    // 二进制拆分
    for (int i = 0; i < w.size(); ++i) {
        int cnt = s[i];
        for (int k = 1; k <= cnt; k *= 2) {
            items.emplace_back(k * w[i], k * v[i]);
        }
    }
}
```

```

        cnt -= k;
    }
    if (cnt > 0)
        items.emplace_back(cnt * w[i], cnt * v[i]);
    }

    // 01 背包求解
    vector<int> dp(W + 1, 0);
    for (auto it : items) {
        for (int j = W; j >= it.first; --j) { // 逆序枚举!
            dp[j] = max(dp[j], dp[j - it.first] + it.second);
        }
    }
    return dp[W];
}

```

55.4 分组背包

给定 N 组物品和一个容量为 W 的背包：

- 每组物品有若干个，同一组内物品只能选一个
- 每个物品 i 有重量 w_i 和价值 v_i
- 求在不超过背包容量的前提下能获得的最大价值

与普通背包的区别

特征	分组背包	01背包
物品选择	每组选0或1个	每个物品选0或1个
决策顺序	先遍历组，再遍历容量	直接遍历物品
状态转移	组内比较	全局比较

关键性质

1. 组间独立性：选择不同组的物品互不影响
2. 组内排他性：同组物品的选择是互斥的

```

#include <bits/stdc++.h>

using namespace std;

```

```

int groupKnapsack(int W, vector<vector<pair<int, int>>>& groups) {
    vector<int> dp(W + 1, 0);

    for (auto group : groups) {        // 遍历每组
        for (int j = W; j >= 0; --j) {  // 逆序遍历容量
            for (auto item : group) {  // 遍历组内物品
                if (j >= item.first) {
                    dp[j] = max(dp[j], dp[j - item.first] + item.second);
                }
            }
        }
    }
    return dp[W];
}

```

```

int main() {
    // 输入数据： 每组物品的(重量, 价值)对
    vector<vector<pair<int, int>>> groups = {
        {{1, 2}, {2, 4}}, // 第 0 组
        {{3, 4}, {1, 5}}, // 第 1 组
        {{2, 3}}          // 第 2 组
    };

    int W = 5;

    // 计算最大价值
    int max_value = groupKnapsack(W, groups);
    cout << "最大价值: " << max_value << endl;

    return 0;
}

```

时间优化

1. 提前排序 : 每组按价值密度(v_i/w_i)降序排列, 可提前 break
2. 剪枝策略 : 记录当前最大价值, 跳过不可能更优的情况

```

int optimizedGroupKnapsack(int W, vector<vector<pair<int, int>>>& groups) {
    vector<int> dp(W + 1, 0);

    for (auto& group : groups) {
        // 按价值密度排序（降序）
        sort(group.begin(), group.end(), [](auto& a, auto& b) {
            return a.second * b.first > b.second * a.first;
        });

        for (int j = W; j >= 0; --j) {
            for (auto& item : group) {
                if (j >= item.first) {
                    if (dp[j - item.first] + item.second > dp[j]) {
                        dp[j] = dp[j - item.first] + item.second;
                        break; // 已找到组内最优
                    }
                }
            }
        }
    }

    return dp[W];
}

```

55.5 二维费用背包

在传统背包问题基础上增加**第二维限制条件**，每个物品有两种不同的费用（如重量和体积），背包也有两种不同的容量限制。

形式化描述：

给定 N 个物品和一个背包，每个物品只能选择一次：

- 每个物品有：重量 w_i 、体积 u_i 、价值 v_i
- 背包有：最大承重 W 、最大容积 U
- 目标：选择物品使得总重量 $\leq W$ ，总体积 $\leq U$ ，且总价值最大

解法	时间复杂度	空间复杂度	适用场景
三维DP	$O(NWU)$	$O(NWU)$	小数据量
二维DP (优化)	$O(NWU)$	$O(W*U)$	常规情况
滚动数组	$O(NWU)$	$O(2WU)$	空间优化

标准解法（三维 DP）

```
int knapsack_3d(int W, int U, vector<int>& w, vector<int>& u, vector<int>& v) {
    int n = w.size();
    vector<vector<vector<int>>> dp(n+1,
        vector<vector<int>>(W+1, vector<int>(U+1, 0)));

    for (int i = 1; i <= n; ++i) {
        for (int j = 0; j <= W; ++j) {
            for (int k = 0; k <= U; ++k) {
                dp[i][j][k] = dp[i-1][j][k]; // 不选
                if (j >= w[i-1] && k >= u[i-1]) {
                    dp[i][j][k] = max(dp[i][j][k], dp[i-1][j-w[i-1]][k-u[i-1]] + v[i-1]);
                }
            }
        }
    }
    return dp[n][W][U];
}
```

空间优化（二维 DP）

优化原理

发现 `dp[i]` 只依赖于 `dp[i-1]`，可压缩第一维

关键点

需要逆序枚举重量和体积，避免覆盖未处理的数据

```
int knapsack_2d(int W, int U, vector<int>& w, vector<int>& u, vector<int>& v) {
    vector<vector<int>> dp(W+1, vector<int>(U+1, 0));

    for (int i = 0; i < w.size(); ++i) {
```

```

    for (int j = W; j >= w[i]; --j) {    // 逆序枚举重量
        for (int k = U; k >= u[i]; --k) { // 逆序枚举体积
            dp[j][k] = max(dp[j][k], dp[j-w[i]][k-u[i]] + v[i]);
        }
    }
}
return dp[W][U];
}

```

常见变种

完全二维背包：每种物品可以选无限次：

```

int knapsack_2d(int W, int U, vector<int>& w, vector<int>& u, vector<int>& v) {
    vector<vector<int>> dp(W+1, vector<int>(U+1, 0));

    for (int i = 0; i < w.size(); ++i) {
        // 只需将逆序改为正序
        for (int j = w[i]; j <= W; ++j) {
            for (int k = u[i]; k <= U; ++k) {
                dp[j][k] = max(dp[j][k], dp[j-w[i]][k-u[i]] + v[i]);
            }
        }
    }
    return dp[W][U];
}

```

分组二维背包：每组物品只能选一个，增加组别循环：

```

for (auto& group : groups) {
    for (int j = W; j >= 0; --j) {
        for (int k = U; k >= 0; --k) {
            for (auto& item : group) {
                if (j >= item.w && k >= item.u) {
                    dp[j][k] = max(dp[j][k], dp[j-item.w][k-item.u] + item.v);
                }
            }
        }
    }
}

```

```

    }
  }
}
}

```

56. 倍增

倍增，即成倍增长。在处理一系列序列数据时，倍增可通过指数级的增长快速覆盖所有数据。

倍增算法采用 2 倍增长的方式，其核心思想是建立以下数组 $Ta[i][j]$ ，表示自 i 向后共 2^j 个数据的某些特性，如最大值等。倍增算法的时间复杂度为 $O(n \log n)$ ，其中倍增的初始化为 $O(n \log n)$ ，查询 $O(1) \sim O(\log n)$

主要用于解决以下类型的问题：

1. 快速查找问题（如最近公共祖先 LCA、区间最值问题）
2. 快速幂计算
3. 路径查询问题
4. 跳跃表实现

57. 求区间最值（RMQ）

倍增思想，DP 预处理 ST 表

ST 表预处理：用动态规划计算所有长度为 2^k 的区间最小值，存储为 $st[i][k]$ （表示区间 $[i, i+2^k-1]$ 的最小值）。

查询：将区间 $[L, R]$ 拆分为两个重叠的 2^k 区间，取最小值。

时间复杂度

预处理： $O(n \log n)$ （计算所有 2^k 区间）

查询： $O(1)$ （只需比较两个区间）

总复杂度： $O(n \log n + q)$ （适合静态数组，查询极快）

关键公式

预处理递推： $st[i][k] = \min(st[i][k-1], st[i+2^{k-1}][k-1])$

查询分解：区间 $[L, R] \Rightarrow \min(st[L][k], st[R-2^k+1][k]), k = \log_2(R-L+1)$

```

vector<vector<int>> buildST(const vector<int>& A) {
    int n = A.size();
    int k = log2(n) + 1;
    vector<vector<int>> st(n, vector<int>(k));

```

```

for (int i = 0; i < n; i++) st[i][0] = A[i]; //第一列是原数组
for (int j = 1; (1 << j) <= n; j++) { //先循环列
    for (int i = 0; i + (1 << j) <= n; i++) { //再循环行
        st[i][j] = min(st[i][j-1], st[i + (1 << (j-1))][j-1]);
    }
}
return st;
}

int queryST(const vector<vector<int>>& st, int L, int R) {
    int k = log2(R - L + 1);
    return min(st[L][k], st[R - (1 << k) + 1][k]);
}

```

58. 逆元

逆元是数论中的一个重要概念，在模运算中扮演着类似"倒数"的角色。下面我将详细解释逆元的定义、性质和应用。

给定整数 a 和模数 m ($m > 1$)，如果存在整数 x 满足： $a * x \equiv 1 \pmod{m}$

则称 x 为 a 在模 m 下的乘法逆元（简称逆元），记作 a^{-1} 或 $\text{inv}(a)$ 。

关键点

1. 存在条件：逆元存在的充要条件是 a 与 m 互质（即 $\text{gcd}(a, m) = 1$ ）
例如：在模 10 下，3 有逆元（ $3 \times 7 = 21 \equiv 1 \pmod{10}$ ），但 2 没有逆元
2. 唯一性：当逆元存在时，在模 m 意义下是唯一的（通常取最小正整数解）
3. 特殊情形：当 m 为质数时，1 到 $m-1$ 的所有整数都有逆元

在模运算中，我们无法直接进行除法运算（ $a/b \pmod{m} \neq (a \pmod{m})/(b \pmod{m})$ ）。逆元允许我们将除法转换为乘法： $a / b \equiv a * b^{-1} \pmod{m}$

实际例子

计算 $12 / 3 \pmod{5}$ ：

1. 先求 3 在模 5 下的逆元： $3 * 2 = 6 \equiv 1 \pmod{5} \Rightarrow 3^{-1} \equiv 2$
2. 然后计算： $12 * 3^{-1} \equiv 12 * 2 = 24 \equiv 4 \pmod{5}$
3. 验证： $12 / 3 = 4 \equiv 4 \pmod{5}$ （正确）

应用场景

1. 组合数学 : 计算大数组合数取模 ($C(n,k) \bmod p$)
2. 密码学 : RSA 等公钥加密算法
3. 算法竞赛 : 解决需要模数除法的问题
4. 多项式运算 : 有限域上的多项式除法

求逆元的方法:

1. 如果模数是质数且只需要计算少量逆元, 费马小定理法最简单。
2. 如果模数不一定是质数, 或者需要更通用的解法, 使用扩展欧几里得算法。
3. 如果需要批量计算大量逆元, 递推法是最佳选择。

在实际编程竞赛中, 扩展欧几里得算法是最常用的方法, 因为它适用范围广且效率足够高。

费马小定理法

当模数 p 为质数时, 根据费马小定理, $a^{(p-1)} \equiv 1 \pmod{p}$, 因此 a 的逆元为 $a^{(p-2)} \bmod p$ 。

```
#include <iostream>

using namespace std;

typedef long long ll;

// 快速幂计算  $a^b \bmod p$ 
ll qpow(ll a, ll b, ll p) {
    ll res = 1;
    while (b) {
        if (b & 1) res = res * a % p;
        a = a * a % p;
        b >>= 1;
    }
    return res;
}

// 费马小定理求逆元
ll inv_fermat(ll a, ll p) {
    return qpow(a, p - 2, p);
}

int main() {
    ll a = 3, p = 1e9 + 7; // p 必须是质数
```

```

    cout << "逆元: " << inv_fermat(a, p) << endl;

    return 0;
}

```

扩展欧几里得算法

扩展欧几里得算法可以求出方程 $ax + py = \gcd(a, p)$ 的解。当 a 和 p 互质时， $\gcd(a, p) = 1$ ，此时 x 就是 a 的逆元。

- 时间复杂度： $O(\log \min(a, p))$
- 优点：适用于任意模数（只要 a 和 p 互质）

```

#include <iostream>

using namespace std;

typedef long long ll;

// 扩展欧几里得算法
ll exgcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    ll d = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return d;
}

// 扩展欧几里得求逆元
ll inv_exgcd(ll a, ll p) {
    ll x, y;
    ll d = exgcd(a, p, x, y);
    if (d != 1) return -1; // 无解
    return (x % p + p) % p; // 保证返回正数
}

```

```

}

int main() {
    ll a = 3, p = 1e9 + 7;
    cout << "逆元: " << inv_exgcd(a, p) << endl;
    return 0;
}

```

递推法（批量计算逆元）

利用递推公式 $\text{inv}[i] = (p - p/i) * \text{inv}[p\%i] \% p$ ，可以批量计算 1 到 n 的所有数的逆元。

- 时间复杂度：O(n)预处理，之后查询 O(1)
- 优点：适合需要大量逆元计算的场景

```

#include <iostream>
#include <vector>
using namespace std;

typedef long long ll;

// 递推法批量计算逆元
vector<ll> inv_recursive(ll n, ll p) {
    vector<ll> inv(n + 1);
    inv[1] = 1;
    for (ll i = 2; i <= n; ++i) {
        inv[i] = (p - p / i) * inv[p % i] % p;
    }
    return inv;
}

int main() {
    ll n = 10, p = 1e9 + 7;
    auto inv = inv_recursive(n, p);
    for (ll i = 1; i <= n; ++i) {

```

```

        cout << "inv[" << i << "] = " << inv[i] << endl;
    }

    return 0;
}

```

59. 表达式计算

后缀表达式计算

从左到右遍历表达式，遇到数字进栈，遇到符号就出栈两个元素做相应运算后将结果进栈，运算时需要注意操作数的先后顺序。遍历结束后，栈里只有一个元素就是运算结果。

代码块

```

void solve(){
    string s;
    stack<ll> stk;
    while(cin>>s){
        if(s.size()==1&&(s[0]=='+'||s[0]=='-'||s[0]=='*'||s[0]=='/'))//运算符
        {
            ll next=stk.top();stk.pop();
            ll pre=stk.top();stk.pop();
            switch (s[0])
            {
                case '+':stk.push(pre+next);break;
                case '-':stk.push(pre-next);break;
                case '*':stk.push(pre*next);break;
                case '/':stk.push(pre/next);break;
            }
        }
        else stk.push(stoll(s));
    }
    cout<<stk.top();
    return;
}

```

前缀表达式计算

笔算时从右到左遍历表达式，其它同后缀表达式。编程时从右到左遍历不方便，所以先把字符串拆分然后倒着遍历，遇到数字就进栈，遇到符号就出栈两个元素计算后再进栈。遍历结束后，数字栈只有一个元素就是计算结果。

```
vector<string> split(string str){
    //将 str 按照空格分隔
    stringstream ss;
    ss<<str;
    vector<string> res;
    string s;
    while(ss>>s){
        res.push_back(s);
    }
    return res;
}

void solve()
{
    string s;
    getline(cin,s);
    vector<string> t=split(s);
    stack<ll> stk;
    //倒序遍历 t，如果是数字就进栈，运算符就出栈两个元素运算后再进栈
    for(int i=t.size()-1;i>=0;i--){
        if(t[i]=="+"||t[i]=="-"||t[i]=="*"||t[i]=="/"){
            ll p=stk.top();stk.pop();
            ll q=stk.top();stk.pop();
            switch(t[i][0]){
                case '+':stk.push(p+q);break;
                case '-':stk.push(p-q);break;
                case '*':stk.push(p*q);break;
                case '/':stk.push(p/q);break;
            }
        }
        else
```

```

    {
        stk.push(stoll(t[i]));
    }
}

cout<<stk.top();
}

```

中缀表达式计算

只含加减的从头依次算到尾。

```

void solve(){
    string s,t,tn;
    getline(cin,s);
    stringstream ss(s);
    stack<ll> stk;
    while(ss>>t){
        if(t=="+"||t=="-"){
            ll p=stk.top();stk.pop();
            ss>>tn;//取下一个运算数
            ll q=stoll(tn);
            if(t=="+") stk.push(p+q);
            else stk.push(p-q);
        }else{
            stk.push(stoll(t));
        }
    }
    cout<<stk.top();

    //或者
    int ans=0,a;
    while(cin>>a) cnt+=a;

    cout<<ans
}

```

只包含加减乘除的，从左到右遍历，遇到数字进栈，遇到加号跳过，遇到减号将下个数字变号进栈，遇到乘号出栈一个元素与乘号右边一个数字相乘后再将结果进栈，遇到除号出栈一个元素与除号右边的数字相除再将结果进栈。遍历结束后，累计栈内数据即计算结果。

```
void solve(){
    ll a,ans=0;
    char op;
    stack<ll> stk;
    cin>>a;
    stk.push(a);
    while(cin>>op>>a){
        if(op=='*'){
            ll p=stk.top();stk.pop();
            stk.push(p*a);
        }else if(op=='/'){
            ll p=stk.top();stk.pop();
            stk.push(p/a);
        }else if(op=='-') stk.push(-a);
        else stk.push(a);
    }
    while(!stk.empty()){
        ans+=stk.top();stk.pop();
    }
    cout<<ans;
}
```

```
void solve(){
    string s,t,tn;
    getline(cin,s);
    stringstream ss(s);
    stack<ll> stk;
    while(ss>>t){
        if(t=="+"||t=="-"){
            ss>>tn;//取下一个运算数
```

```

        ll q=stoll(tn);

        if(t=="-") stk.push(-q);

        else stk.push(q);

    }else if(t=="*"||t=="/"){

        ll p=stk.top();stk.pop();

        ss>>tn;//取下一个运算数

        ll q=stoll(tn);

        if(t=="*") stk.push(p*q);

        else stk.push(p/q);

    }else{

        stk.push(stoll(t));

    }

}

ll ans=0;

//累加站内元素

while(!stk.empty()){

    ans+=stk.top();

    stk.pop();

}

cout<<ans;

}

```

包含加减乘除和括号的：用字符串栈。

直接进栈的情况：

栈当前是左括号、当前是运算符且后面是左括号、运算数。减法转成加相反数，将相反数进栈。

出栈一个元素做运算的情况：

当前是乘号且后面不是左括号、当前是除号且后面不是左括号。出栈一个元素，与当前字符串后面的操作数计算后重新进栈。

特殊情况：

当前是右括号，一直出栈到左括号前面的第一个运算符，做运算后再进栈。

最后累加站内元素即答案。

```

vector<string> split(string str){

    stringstream ss(str);

```

```

vector<string> res;

string s;
while(ss>>s){
    res.push_back(s);
}

return res;
}

bool isNum(string s){
    return (s!="+"&&s!="-"&&s!="*"&&s!="/"&&s!="("&&s!=")");
}

void solve()
{
    string s,t,tn;
    getline(cin,s);
    vector<string> v;
    v=split(s);
    stack<string> stk;
    for(int i=0;i<v.size();i++){
        if(stk.empty()||v[i]=="("||isNum(v[i])||v[i+1]=="("){
            //直接进栈的情况
            stk.push(v[i]);
        }else if(v[i]=="-"){//减法把下个操作数设为相反数
            v[i+1]=to_string(-stoll(v[i+1]));
        }else if(v[i]=="*"){//乘法计算后修改下个操作数
            ll p=stoll(stk.top())*stoll(v[i+1]);
            stk.pop();
            v[i+1]=to_string(p);
        }else if(v[i]=="/"){//除法计算后修改下个操作数
            ll p=stoll(stk.top())/stoll(v[i+1]);
            stk.pop();
            v[i+1]=to_string(p);
        }else if(v[i]=="){
            //右括号，一直计算到左括号，并处理左括号前面的运算符

```

```

    ll p=0;
    while(stk.top()!=""){
        p+=stoll(stk.top());
        stk.pop();
    }
    stk.pop();
    //左括号前面还有运算符
    if(!stk.empty()){
        string k=stk.top();
        stk.pop();
        //分别针对加减乘除进行不同进栈
        if(k=="+") stk.push(to_string(p));
        else if(k=="-") stk.push(to_string(-p));
        else if(k=="*"||k=="/"){
            ll q=stoll(stk.top());
            stk.pop();
            if(k=="*") stk.push(to_string(q*p));
            else if(k=="/") stk.push(to_string(q/p));
        }

    }
}

ll ans=0;
//累加站内元素
while(!stk.empty()){
    ans+=stoll(stk.top());
    stk.pop();
}
cout<<ans;
}

```

中缀表达式转后缀表达式

```

#include<bits/stdc++.h>

using namespace std;

#define ll long long

#define el "\n"

//定义运算符优先级顺序，左括号仅用于入栈
unordered_map<string,int> index={{ "^",4},{ "*",3},{ "/",3},{ "+",2},{ "-",2},{ "(",1}};

//检查是否是运算符
bool isOperator(string s){
    return (s=="+"||s=="-"||s=="*"||s=="/"||s=="^");
}

vector<string> inToPost(vector<string> &strs){
    stack<string> ops;//运算符栈
    vector<string> res;//存后缀结果
    for(string s:strs){
        if(s=="("){
            ops.push(s);
        }else if(isOperator(s)){
            //当前是运算符，把栈顶优先级不小于当前运算符的出栈到结果数组，然后再进栈
            while(!ops.empty()&&index[ops.top()]>=index[s]){
                res.push_back(ops.top());ops.pop();
            }
            ops.push(s);
        }else if(s=="){
            //栈顶弹出直到左括号
            while(!ops.empty()&&ops.top()!=""){
                res.push_back(ops.top());ops.pop();
            }
            ops.pop();//弹出左括号
        }else {
            res.push_back(s);
        }
    }

    //处理运算符栈中剩余的元素

```

```

while(!ops.empty()){
    res.push_back(ops.top());ops.pop();
}

return res;
}

vector<string> split(string s){
    string s1,t;
    //给 s 加上分割空格分割
    for(int i=0;i<s.size();i++){
        if(s[i]=='-'&&(i==0||(s[i-1]!=' ' &&(s[i-1]<'0' || s[i-1]>'9')))){
            //处理负号
            s1=s1+s[i];
        }
        else if(s[i]>='0'&&s[i]<='9') s1+=s[i];
        else s1=s1+" "+s[i]+" ";
    }

    vector<string> res;
    stringstream ss(s1);
    while(ss>>t) res.push_back(t);
    return res;
}

int solve()
{
    string s;
    getline(cin,s);
    vector<string> str=split(s);
    vector<string> ans=inToPost(str);
    for(string s1:ans) cout<<s1<<" ";
}

int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);cout.tie(0);

```

```

//freopen("***.in","r",stdin);
//freopen("***.out","w",stdout);
int T=1;//cin>>T;//多测时，取消注释
while(T--){
    solve();
}
return 0;
}
/*
in:
-2+3*(-6+7)
out:
-2 3 -6 7 + * +
*/

```

表达式树

表达式树是一种二叉树结构，用于表示数学表达式。树的分支节点是运算符，叶子节点是操作数。

已知一个表达式，可以通过这个表达式构建一棵树，树的先序遍历和后续遍历分别对应前缀表达式和后缀表达式，中序遍历得到中缀表达式(需要处理括号)。求值时，深搜，如果当前节点是运算符就计算，否则就直接返回数值。

```

#include <bits/stdc++.h>
using namespace std;
#define vs vector<string>
#define ll long long
#define el '\n'

struct node{
    string data;
    node *left=NULLPTR,*right=NULLPTR;
    node(string s){
        data=s;
    }
    node(string s,node *l,node *r){

```

```

        data=s;left=l;right=r;
    }
};

node *root;

vs split(string s){
    string t;
    vs res;
    stringstream ss(s);
    while(ss>>t) res.push_back(t);
    return res;
}

bool isOperater(string s){
    return (s=="+"||s=="-"||s=="*"||s==" /");
}

void firstView(node *rt){
    cout<<rt->data<<" ";
    if(rt->left) firstView(rt->left);
    if(rt->right) firstView(rt->right);
}

void lastView(node *rt){
    if(rt->left) lastView(rt->left);
    if(rt->right) lastView(rt->right);
    cout<<rt->data<<" ";
}

void middleView(node *rt){
    if(rt->left) cout<<"(",middleView(rt->left);
    cout<<rt->data;
    if(rt->right) middleView(rt->right),cout<<")";
}

ll cala(node *rt){
    string s=rt->data;
    if(isOperater(s)) {

```

```

        if(s=="+") return cala(rt->left)+cala(rt->right);
        else if(s=="-") return cala(rt->left)-cala(rt->right);
        else if(s=="*") return cala(rt->left)*cala(rt->right);
        else if(s=="/") return cala(rt->left)/cala(rt->right);
    }else return stol(s);
}

void solve(){
    string s;
    getline(cin,s);
    vs strs=split(s);
    stack<node *> stk;
    for(string i:strs){
        if(isOperater(i)){
            node *r=stk.top(); stk.pop();
            node *l=stk.top(); stk.pop();
            node *p=new node(i,l,r);
            stk.push(p);
        }else{
            node *p=new node(i);
            stk.push(p);
        }
    }
    root=stk.top();
    cout<<"前缀表达式为: "<<el;
    firstView(root);
    cout<<el;
    cout<<"后缀表达式为: "<<el;
    lastView(root);
    cout<<el;
    cout<<"中缀表达式为: "<<el;
    middleView(root);
    cout<<el;
    cout<<"计算结果为: "<<el;

```

```

        cout<<cala(root)<<endl;
    }

    int main(){
        ios::sync_with_stdio(0);
        cin.tie(0);cout.tie(0);
        //freopen("***.in","r",stdin);
        //freopen("***.out","w",stdout);
        int T=1;//cin>>T;
        while(T--){
            solve();
        }
    }
}

```

60. 树的存储与其深搜与广搜

存储方法	优点	缺点	适用场景
双亲表示法	查找父节点快，结构简单	查找子节点效率低	需要频繁查找父节点的场景
孩子表示法	查找子节点快	查找父节点效率低	需要频繁访问子节点的场景
孩子兄弟表示法	节省空间，统一结构	访问父节点困难	需要节省空间的场景
顺序存储	内存连续，访问快	不适合动态变化	完全二叉树或静态树
邻接表	动态变化灵活	内存开销较大	动态变化的树结构

1. 父亲表示法

因为除了根节点，每个节点只有一个父节点，而根节点没有父节点。可以用 `a[child]=parent` 的方式保存树，适用于频繁查询父节点的情况。如果节点的数值跟下标不一致，可以定义结构体，先序遍历、后序遍历和层次遍历，如下段程序。

```

#include <bits/stdc++.h>
using namespace std;

struct Node {
    char data;
    int parent; // -1 表示无父节点(根节点)，非-1 的值就是父节点的下标
};

```

```

void firstview(vector<Node> &tree ,int root){ //O(n*n)

    cout<<tree[root].data;//先序遍历启用这行，关闭下面后序遍历

    for(int i=0;i<tree.size();i++){

        if(tree[i].parent==root) firstview(tree,i);

    }

    //cout<<tree[root].data;//后序遍历启用这行

    return ;

}

void firstview2(vector<Node> &tree ,int root){ //O(n*n)

    stack<int> stk,res;

    stk.push(root);

    while(!stk.empty()){

        int rt=stk.top();stk.pop();

        cout<<tree[rt].data;//先序遍历启用这行，关闭下面后序遍历

        for(int i=tree.size()-1;i>=0;i--){ //倒着循环

            if(tree[i].parent==rt) stk.push(i);

        }

        //res.push(tree[rt].data);//后序遍历先把当前节点放到栈里

    }

    return ;

}

void levelview(vector<Node> &tree ,int root){ //O(n*n)

    queue<int> q;

    q.push(root);

    while(!q.empty()){

        int rt=q.front();q.pop();

        cout<<tree[rt].data;

        for(int i=0;i<tree.size();i++){//子节点入队

            if(tree[i].parent==rt) q.push(i);

        }

    }

    return ;

}

```

```

int main() {
    // 示例树结构
    //      A
    //     /\
    //    B C D
    //   /\
    //  E  F

    vector<Node> tree = {
        {'A', -1}, // 下标是 0, 父节点是-1, 所以这是跟节点
        {'B', 0},  // 下标是 1, 父节点下标是 0
        {'C', 0},  // 2
        {'D', 0},  // 3
        {'E', 1},  // 4
        {'F', 1}   // 5
    };

    cout<<"树结构数组: "<<endl;
    for (int i = 0; i < tree.size(); ++i) {
        cout <<i<< ": " << tree[i].data
            << ", " << tree[i].parent << endl;
    }

    cout<<"先序遍历: "<<endl;
    firstview(tree,0); //先序遍历递归版
    cout<<endl;
    firstview2(tree,0); //先序遍历非递归版
    cout<<endl;
    cout<<"层次遍历: "<<endl;
    levelview(tree,0); //层次遍历
    cout<<endl;
    return 0;
}

```

2. 孩子表示法

结构体中包含一个多个孩子节点的指针数组，`struct node{int data,node *children[10];}`; 适用于频繁查询子节点的情况。根据需要也可以增加父节点的指针，形成父亲孩子表示法。

```
#include <bits/stdc++.h>

using namespace std;

struct Node {
    char data;
    vector<int> children;//频繁修改增删孩子时，此处可以用 list
};

void firstview(vector<Node> &tree ,int root){ //O(n)
    cout<<tree[root].data;//先序遍历启用这行，关闭下面后序遍历
    for(int i=0;i<tree[root].children.size();i++){
        firstview(tree,tree[root].children[i]);
    }
    //cout<<tree[root].data;//后序遍历启用这行
    return ;
}

void firstview2(vector<Node> &tree ,int root){ //O(n)
    stack<int> stk;
    stk.push(root);
    while(!stk.empty()){
        int rt=stk.top();stk.pop();
        cout<<tree[rt].data;//先序遍历启用这行，关闭下面后序遍历
        for(int i=tree[rt].children.size()-1;i>=0;i--){ //倒着循环
            stk.push(tree[rt].children[i]);
        }
        //后序遍历需要先将 tree[rt].data 存到栈里。
    }
    return ;
}

void levelview(vector<Node> &tree ,int root){ //O(n*n)
    queue<int> q;
```

```

q.push(root);

while(!q.empty()){

    int rt=q.front();q.pop();

    cout<<tree[rt].data;

    for(int i=0;i<tree[rt].children.size();i++){//子节点入队

        q.push(tree[rt].children[i]);

    }

}

return ;

}

```

```

int main() {

    // 示例树结构

    //      A
    //     /|\
    //    B C D
    //   / \
    //  E  F

    vector<Node> tree = {

        {'A', {1,2,3}}, // 下标是 0, 父节点是-1, 所以这是跟节点
        {'B', {4,5}},   // 下标是 1, 父节点下表是 0
        {'C', {}},      // 2
        {'D', {}},      // 3
        {'E', {}},      // 4
        {'F', {}},      // 5
    };

    cout<<"树结构数组: "<<endl;

    for(int i=0;i<tree.size();i++){

        cout<<i<<": "<<tree[i].data<<", {";

        for(int j=0;j<tree[i].children.size();j++) {

            if(j>0) cout<<",";

```

```

        cout<<tree[i].children[j];
    }
    cout<<"}";
    cout<<endl;
}
cout<<"先序遍历: "<<endl;
firstview(tree,0); //先序遍历递归版
cout<<endl;
firstview2(tree,0); //先序遍历非递归版
cout<<endl;
cout<<"层次遍历: "<<endl;
levelview(tree,0); //层次遍历
cout<<endl;
return 0;
}

```

3. 二叉树左右孩子链式存储法

```

#include<bits/stdc++.h>
using namespace std;
struct node{
    int data;
    node *leftChild=NULLptr,*rightChild=NULLptr;
};
node *root;
void makeTree(node *&t){
    int x;cin>>x;
    if(x>0){
        t=new node;
        t->data=x;
        makeTree(t->leftChild);
        makeTree(t->rightChild);
    }
}

```

```

}

void firstView(node *&t){
    cout<<t->data<<" ";
    if(t->leftChild) firstView(t->leftChild);
    if(t->rightChild) firstView(t->rightChild);
}

void lastView(node *&t){
    if(t->leftChild) lastView(t->leftChild);
    if(t->rightChild) lastView(t->rightChild);
    cout<<t->data<<" ";
}

void middleView(node *&t){
    if(t->leftChild) middleView(t->leftChild);
    cout<<t->data<<" ";
    if(t->rightChild) middleView(t->rightChild);
}

void levelView(node *&t){
    node *p[15];
    int front=0,rear=0;
    p[++rear]=t;
    while(front<rear){
        cout<<p[++front]->data<<" ";
        if(p[front]->leftChild) p[++rear]=p[front]->leftChild;
        if(p[front]->rightChild) p[++rear]=p[front]->rightChild;
    }
}

void levelView2(node *&t){ //队列
    queue<node*> q;
    q.push(t);
    while(!q.empty()){
        node *ft=q.front();q.pop();
        cout<<ft->data<<" ";
        if(ft->leftChild) q.push(ft->leftChild);
    }
}

```

```

        if(ft->rightChild) q.push(ft->rightChild);
    }
}

int main(){
    makeTree(root);
    cout<<"先序遍历: "<<endl;
    firstView(root);
    cout<<endl;
    cout<<"后序遍历: "<<endl;
    lastView(root);
    cout<<endl;
    cout<<"中序遍历: "<<endl;
    middleView(root);
    cout<<endl;
    cout<<"层次遍历: "<<endl;
    levelView(root);
    cout<<endl;
    levelView2(root);
    return 0;
}

/*
1 2 5 0 0 6 0 0 4 0 7 8 0 0 9 0 0
*/

```

4. 孩子兄弟表示法

是将普通树转换成二叉树的方式，每个节点的左孩子是普通树形态的第一个子节点，右孩子是普通树形态的兄弟节点。适用于把普通树转化成二叉树，而后可以使用二叉树的算法。

普通树与孩子兄弟二叉树的互转：

a. 根节点保持不变

b. 处理每个节点的子节点：

 第一个子节点作为左孩子

 其余子节点依次作为右孩子（形成兄弟链）

c.递归处理所有子树

```
#include <bits/stdc++.h>

struct TreeNode {
    char data;

    TreeNode* firstChild; // 指向第一个孩子节点
    TreeNode* nextSibling; // 指向下一个兄弟节点

    TreeNode(char val) : data(val), firstChild(nullptr), nextSibling(nullptr) {}
};

// 先序遍历
void preOrderTraversal(TreeNode* root) {
    if (root == nullptr) return;

    std::cout << root->data << " "; // 访问当前节点
    preOrderTraversal(root->firstChild); // 递归遍历第一个孩子
    preOrderTraversal(root->nextSibling); // 递归遍历兄弟节点
}

// 后序遍历
void postOrderTraversal(TreeNode* root) {
    if (root == nullptr) return;

    postOrderTraversal(root->firstChild); // 递归遍历第一个孩子
    std::cout << root->data << " "; // 访问当前节点
    postOrderTraversal(root->nextSibling); // 递归遍历兄弟节点
}

// 层次遍历
void levelOrderTraversal(TreeNode* root) {
    if (root == nullptr) return;
```

```

std::queue<TreeNode*> q;
q.push(root);

while (!q.empty()) {
    TreeNode* node = q.front();
    q.pop();

    std::cout << node->data << " ";

    // 将第一个孩子和所有兄弟节点加入队列
    if (node->firstChild) {
        TreeNode* child = node->firstChild;
        q.push(child);

        // 加入所有兄弟节点
        while (child->nextSibling) {
            q.push(child->nextSibling);
            child = child->nextSibling;
        }
    }
}
}

```

5. 完全二叉树数组表示法

如果完全二叉树的根节点下标是 1，其它节点按层次存储，那么 x 节点的左子节点是 $2x$ ， x 节点的右子节点为 $2x+1$ ，其父节点下标是 $x/2$ ，可以用来确定最后一个非叶子节点的下标。如果完全二叉树的根节点下标是 0，其它按层次顺序存储，那么下标为 x 的节点的左右节点分别是 $2x+1$ 和 $2x+2$ ，其父节点的下标是 $(x-1)/2$ 。

```

#include <bits/stdc++.h>
using namespace std;
void firstView(vector<char> &tree,int rt){
    cout<<tree[rt];
    if(rt*2+1<tree.size()) firstView(tree,rt*2+1);
}

```

```

    if(rt*2+2<tree.size()) firstView(tree,rt*2+2);
}

void firstView2(vector<char> &tree,int rt){
    stack<int> stk;
    stk.push(rt);
    while(!stk.empty()){
        int root=stk.top();stk.pop();
        cout<<tree[root];
        if(2*root+2<tree.size()) stk.push(2*root+2);//先右孩子
        if(2*root+1<tree.size()) stk.push(2*root+1);//后左孩子
    }
}

void lastView(vector<char> &tree,int rt){
    if(rt*2+1<tree.size()) lastView(tree,rt*2+1);
    if(rt*2+2<tree.size()) lastView(tree,rt*2+2);
    cout<<tree[rt];
}

void middleView(vector<char> &tree,int rt){
    if(rt*2+1<tree.size()) lastView(tree,rt*2+1);
    cout<<tree[rt];
    if(rt*2+2<tree.size()) lastView(tree,rt*2+2);
}

void levelView(vector<char> &tree,int rt){
    queue<int> q;
    q.push(rt);
    while(!q.empty()) {
        int root=q.front();q.pop();
        cout<<tree[root];
        if(2*root+1<tree.size()) q.push(2*root+1);
        if(2*root+2<tree.size()) q.push(2*root+2);
    }
}

int main() {

```

```
// 完全二叉树示例
```

```
//      A
```

```
//    /  \
```

```
//   B   C
```

```
//  / \  /
```

```
// D  E F
```

```
vector<char> tree = {'A', 'B', 'C', 'D', 'E', 'F'};
```

```
//从下标 0 开始存放数据，所以孩子下标是  $2*x+1$  和  $2*x+2$ 
```

```
// 打印树结构
```

```
for (int i = 0; i < tree.size(); ++i) {
```

```
    cout << "Node " << i << ": " << tree[i];
```

```
    int parent = (i - 1) / 2;
```

```
    if (i > 0) cout << ", Parent: " << tree[parent];
```

```
    int left = 2 * i + 1;
```

```
    int right = 2 * i + 2;
```

```
    if (left < tree.size()) cout << ", Left: " << tree[left];
```

```
    if (right < tree.size()) cout << ", Right: " << tree[right];
```

```
    cout << endl;
```

```
}
```

```
cout<<"先序遍历(递归版): "<<endl;
```

```
firstView(tree,0);
```

```
cout<<endl;
```

```
cout<<"先序遍历(非递归版): "<<endl;
```

```
firstView2(tree,0);
```

```
cout<<endl;
```

```
cout<<"后序遍历: "<<endl;
```

```
lastView(tree,0);
```

```
cout<<endl;
```

```
cout<<"中序遍历: "<<endl;
```

```
middleView(tree,0);
```

```
cout<<endl;
```

```

    cout<<"层次遍历: "<<endl;

    levelView(tree,0);

    cout<<endl;

    // levelView2(root);

    return 0;

}

```

6. 图的存储方法

因为树是图的一类，所以可以用保存图的方法保存树。如邻接矩阵、邻接表等。在求树的直径等问题时，用图的方式存储树更加方便。

邻接矩阵、邻接表法、链式前向星

6.1. 图的存储

需要分清：图结构的存储（如存有向边、无向边）、权的存储（边的权值，可以与图的结构合并成数对或者结构体）、遍历过程的存储（如 `vis` 数组记录是否访问过还可用于回溯，`inq` 数组记录节点是否在队列中，栈记录深搜的节点，记录前驱节点避免重复造成死循环）。

1. 邻接矩阵

邻接矩阵（二维数组，行数和列数都是点的数量，值为 0 表示无边，值为 1 表示有边，无向图关于主对角线对称），邻接矩阵适合存储稠密图，避免大量的 0 浪费空间。受二维数组容量限制，点数最好不要超过 2000 个。

代码块

```

#include<bits/stdc++.h>

using namespace std;

#define ll long long
#define el "\n"

const ll N=105;

ll a[N][N],n,m,x,y,w;

bool vis[N];

void add(ll x,ll y,ll w){
    a[x][y]=w;//有向图

    //a[y][x]=w;//无向图
}

void dfs(ll x,ll p,ll k){

```

```

cout<<x<<" "<<p<<" "<<k<<el;
for(ll u=1;u<=n;u++){
    ll w=a[x][u];
    if(vis[u]==0&&w>0&&u!=p){
        //未访问过、有边、不是前驱节点
        vis[u]=1;
        dfs(u,x,k+w);
        //vis[u]=0; //需要回溯否
    }
}
}

void bfs(ll x){
    queue<pair<ll,ll>> q;//创建队列，保存节点和路权和
    vector<bool> visit(n,0);//是否访问过
    q.push({x,0});//起点入队
    visit[x]=1;
    while(!q.empty()){
        pair<ll,ll> p=q.front();q.pop();
        cout<<p.first<<" "<<p.second<<el;
        for(ll u=1;u<=n;u++){//遍历每个节点
            if(!visit[u]&&a[p.first][u]>0){
                //未访问过、有边
                q.push({u,a[p.first][u]+p.second});
                visit[u]=1;
            }
        }
    }
}

int main(){
    cin>>n>>m;
    for(int i=1;i<=m;i++){
        cin>>x>>y>>w;
        add(x,y,w);
    }
}

```

```

    }
    for(int i=1;i<=n;i++){
        for(int j=1;j<=n;j++){
            cout<<setw(3)<<a[i][j];
        }
        cout<<endl;
    }
    cout<<"dfs:\n";
    dfs(1,-1,0);//开始节点、父节点、路权和
    cout<<"bfs\n";
    bfs(1);//开始节点
    return 0;
}
/*
4 5
1 2 3
1 3 1
4 1 5
2 4 6
3 4 2
*/

```

2.邻接表

存储邻接表的结构可以根据需要有以下几种：

//vector 数组，数组的下标是边的起点，G 和 W 分别存储边的终点和权重。

```
vector<ll> G[N],W[N];
```

//数据对 vector 数组，数组下标是起点，

//pair<ll,ll>保存边的终点和权重或者权重和终点。

//pair 的好处是直接可以比较

```
vector<pair<ll,ll>> G[N];
```

```

//结构体 vector 数组
struct edge{int to,weight};
vector<edge> G[N];

//二维 vector
vector<vector<int> > g;
cin>>n;
g.resize(n+1);

//哈希 map 好处是节点排序的。
unordered_map<int, vector<int>> graph;

```

```

#include<bits/stdc++.h>
using namespace std;
#define ll long long
#define el "\n"
const ll N=105;
vector<ll> G[N],W[N];
ll n,m,x,y,w;
bool vis[N];
void add(ll x,ll y,ll w){
    G[x].push_back(y);
    W[x].push_back(w);
}
void dfs(ll x,ll v){
    cout<<x<<" "<<v<<el;
    for(ll i=0;i<G[x].size();i++){
        ll u=G[x][i],w=W[x][i];
        if(vis[u]==0){
            vis[u]=1;
            dfs(u,v+w);
            vis[u]=0;
        }
    }
}

```

```

    }
}
int main(){
    cin>>n>>m;
    for(int i=1;i<=m;i++){
        cin>>x>>y>>w;
        add(x,y,w);
    }
    dfs(1,0);
    return 0;
}
/*
4 5
1 2 3
1 3 1
1 4 5
2 4 6
3 4 2
*/

```

3.链式前向星

代码块

```

#include<bits/stdc++.h>
using namespace std;
#define ll long long
#define el "\n"
const ll Ne=105,Nd=105;
ll To[Ne],W[Ne],Next[Ne];//到哪里，权重，下一条边
ll Begin[Nd],e,n,m,x,y,w;//每个节点的链表开始
bool vis[Nd];
void add(ll x,ll y,ll w){ //添加从 x 点到 y 点权重是 w 的边
    To[++e]=y;//新建一条边，并保存边指向 y 点

```

```

        W[e]=w;//设置此边的权重是 w

        Next[e]=Begin[x];//设置此边的下一条边

        Begin[x]=e;//将新建的边作为链表头结点
    }
    void dfs(ll x,ll v)
    {
        cout<<x<<" "<<v<<endl;
        for(ll i=Begin[x];i;i=Next[i]){
            ll u=To[i];
            ll w=W[i];
            if(vis[u]==0){
                vis[u]=1;
                dfs(u,v+w);
                vis[u]=0;
            }
        }
    }
}

int main(){
    cin>>n>>m;
    for(int i=1;i<=m;i++){
        cin>>x>>y>>w;
        add(x,y,w);
    }
    dfs(1,0);
    return 0;
}

/*
4 5
1 2 3
1 3 1
1 4 5
2 4 6
3 4 2

```

*/

62. 树的基本算法

2. 树的直径

树的直径是指树中任意两点间最长路径的长度。

算法一：两次 DFS/BFS 算法

算法原理：

第一次搜索：从任意节点出发，找到距离最远的节点 u

第二次搜索：从节点 u 出发，找到距离最远的节点 v

u 和 v 之间的路径就是树的直径

时间复杂度：

时间复杂度： $O(n)$ ，其中 n 是树的节点数

空间复杂度： $O(n)$

```
#include <bits/stdc++.h>

using namespace std;

// 使用邻接表表示树
using Tree = vector<vector<pair<int, int>>>; // {相邻节点, 边权}

/**
 * 执行 BFS/DFS 搜索，返回距离起始点最远的节点及其距离
 * @param tree 树的邻接表表示
 * @param start 起始节点
 * @return tuple<最远节点, 最大距离>
 */
tuple<int, int> findFarthestNode(const Tree& tree, int start) {
    vector<bool> visited(tree.size(), false);
    queue<pair<int, int>> q; // {节点, 当前距离}
    q.emplace(start, 0);
    visited[start] = true;
```

```

int farthestNode = start;

int maxDist = 0;

while (!q.empty()) {
    auto [node, dist] = q.front();
    q.pop();

    if (dist > maxDist) {
        maxDist = dist;
        farthestNode = node;
    }

    for (const auto& [neighbor, weight] : tree[node]) {
        if (!visited[neighbor]) {
            visited[neighbor] = true;
            q.emplace(neighbor, dist + weight);
        }
    }
}

return {farthestNode, maxDist};
}

```

```

/**

```

```

 * 计算树的直径（两次 BFS/DFS 方法）

```

```

 * @param tree 树的邻接表表示

```

```

 * @return 树的直径长度

```

```

 */

```

```

int treeDiameter(const Tree& tree) {

```

```

    if (tree.empty()) return 0;

```

```

    // 第一次搜索：从节点 0 出发，找到最远节点 u

```

```

    auto [u, _] = findFarthestNode(tree, 0);

```

```

// 第二次搜索：从节点 u 出发，找到最远节点 v
auto [v, diameter] = findFarthestNode(tree, u);

return diameter;
}

int main() {
    // 示例树结构
    Tree tree = {
        {{1, 1}, {2, 1}}, // 节点 0
        {{0, 1}, {3, 1}}, // 节点 1
        {{0, 1}, {4, 1}}, // 节点 2
        {{1, 1}},         // 节点 3
        {{2, 1}, {5, 1}}, // 节点 4
        {{4, 1}}          // 节点 5
    };

    cout << "树的直径为: " << treeDiameter(tree) << endl;
    return 0;
}

```

3. 树的重心

树的重心性质

- 树的重心最多有两个
- 如果树有两个重心，它们必定相邻
- 将树的重心作为根，可以保证所有子树大小不超过 $n/2$
- 重心在树的动态维护和分治算法中有重要应用

解决树的重心问题主要有以下几种算法：

- 暴力 DFS 法：对每个节点计算删除后的最大子树大小，然后选择最小的
- 优化 DFS 法：通过一次 DFS 遍历计算所有必要信息
- 动态规划法：类似优化 DFS 法，但更系统地记录子树信息

对比：

- 暴力 DFS 法时间复杂度 $O(n^2)$ ，不推荐
- 优化 DFS 法和动态规划法都是 $O(n)$ 时间复杂度，但优化 DFS 法实现更简洁
- 动态规划法更系统化，适合更复杂的树问题

优化 DFS 法详解

树的重心是指树中这样的一个节点：删除该节点后，剩下的各个连通块（子树）中节点数的最大值最小。

算法思路

- 任选一个节点作为根进行 DFS 遍历
- 对于每个节点，计算以它为根的子树大小，累加子树大小得到向下的当前树的大小，再与向上方向当前树的大小取较大值。
- 同时记录删除该节点后的最大子树大小
- 最后找出使最大子树大小最小的节点

复杂度分析

- 时间复杂度： $O(n)$ ，只需一次 DFS 遍历
- 空间复杂度： $O(n)$ ，用于存储树结构和辅助数组

```
#include <bits/stdc++.h>

using namespace std;

const int MAXN = 1e5 + 5;
vector<int> tree[MAXN];
int size[MAXN]; // 存储每个节点的子树大小
int max_subtree[MAXN]; // 存储删除该节点后的最大子树大小
int n; // 节点数
vector<int> centroids; // 存储所有重心

void dfs(int u, int parent) {
    size[u] = 1;
    max_subtree[u] = 0;

    for (int v : tree[u]) {
        if (v != parent) {
            dfs(v, u);
            size[u] += size[v];
        }
    }
}
```

```

        max_subtree[u] = max(max_subtree[u], size[v]);
    }
}

// 还要考虑父节点方向的子树
max_subtree[u] = max(max_subtree[u], n - size[u]);
}

void find_centroids() {
    dfs(1, -1); // 假设 1 是根节点

    int min_max_subtree = *min_element(max_subtree + 1, max_subtree + n + 1);

    for (int i = 1; i <= n; ++i) {
        if (max_subtree[i] == min_max_subtree) {
            centroids.push_back(i);
        }
    }
}

int main() {
    cin >> n;
    for (int i = 1; i < n; ++i) {
        int u, v;
        cin >> u >> v;
        tree[u].push_back(v);
        tree[v].push_back(u);
    }

    find_centroids();

    cout << "Tree centroids: ";
    for (int c : centroids) {
        cout << c << " ";
    }
}

```

```

    }

    cout << endl;

    return 0;
}

```

4. LCA

最低公共祖先（Lowest Common Ancestor, LCA）是树结构中两个节点的最近公共祖先节点。LCA 算法广泛应用于树操作、图论、生物信息学等领域。我们介绍 **四种常见的 LCA 算法**。

暴力法（Brute Force）

思路：

从两个节点分别向上回溯到根节点，记录路径，然后比较两条路径，找到最后一个相同的节点。树可以用父亲表示法存储。

适用于单次查询，但多次查询效率低。

时间复杂度：

预处理：无（每次查询都重新计算）。

查询： $O(h)$ （ h 是树的高度）。

适用场景：树较平衡（ $h \approx \log n$ ），查询次数少。

```

#include<bits/stdc++.h>
using namespace std;
struct Node{char data;int parent;};
unordered_map<char,int> mp;
vector<Node> tree;
vector<int> getPath(int nod){
    vector<int>res;
    while(nod>=0){
        res.push_back(nod);
        nod=tree[nod].parent;
    }
    return res;
}

```

```

void solve(){
    int n,m;cin>>n;
    tree.resize(n);
    for(int i=0;i<n;i++){
        cin>>tree[i].data>>tree[i].parent;
        mp[tree[i].data]=i;//用字典记录节点下标，方便通过 data 查询节点下标
    }
    cin>>m;
    while(m--){
        char c,d;cin>>c>>d;
        int nod1=mp[c],nod2=mp[d];//查询节点下标
        vector<int>v1=getPath(nod1),v2=getPath(nod2);//获取路径
        reverse(v1.begin(),v1.end());
        reverse(v2.begin(),v2.end());
        int ansid=0;
        for(int i=0;i<n;i++){
            if(v1[i]!=v2[i]){
                cout<<tree[ansid].data<<endl;
                break;
            }
            ansid=v1[i];
        }
    }
}

int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);cout.tie(0);
    //freopen("P.in","r",stdin);
    //freopen("***.out","w",stdout);
    int T=1;//cin>>T;
    while(T--){

```

```

        solve();
    }
    return 0;
}
/*
输入节点数 n，后面 1~n 行，每行一个字符是第 i (0<=i<n) 个节点数据和父节点编号，根
节点的父节点是-1
有 m 次查询，后面每行两个字符，表示查询两个字符节点的 LCA
10
A -1
B 0
C 0
D 0
E 1
F 1
G 3
H 3
I 5
J 6
5
A B
E F
I J
G J
C E
输出：每行一个即相应的 LCA
*/

```

倍增法 (Binary Lifting)

倍增法 (Binary Lifting) 是一种高效的在线算法，用于求解树结构中两个节点的最近公共祖先 (LCA)。它通过预处理每个节点的 2^k 级祖先，使得每次查询的时间复杂度降至 $O(\log n)$ ，适用于需要频繁查询 LCA 的场景。

思路

预处理每个节点的 2^k 级祖先，利用二进制跳跃快速向上查找。

适用于多次查询，是在线算法（无需提前知道所有查询）。

时间复杂度

预处理时间复杂度： $O(n \log n)$

每个节点需要计算 $\log n$ 个祖先

DFS 遍历整个树需要 $O(n)$ 时间

查询时间复杂度： $O(\log n)$

最多进行 $\log n$ 次跳跃

空间复杂度： $O(n \log n)$

存储 up 表需要 $n \times \log n$ 的空间

算法优势

在线算法：可以随时处理新的查询，不需要预先知道所有查询

高效查询：每次查询只需 $O(\log n)$ 时间

实现简单：代码结构清晰，易于理解和实现

核心思路

预处理：计算每个节点 u 的 2^k 级祖先（即 u 向上跳 2^k 步到达的节点）。

查询优化：利用二进制跳跃快速调整两个查询节点的深度，先将两个节点调整到相同深度，然后同步向上跳跃找到 LCA。

关键点

二进制跳跃：利用 2^k 的指数增长特性，减少跳跃次数（类似快速幂的思想）。

动态规划预处理： $up[u][k] = up[up[u][k-1]][k-1]$ （即 u 的 2^k 级祖先是 u 的 2^{k-1} 级祖先的 2^{k-1} 级祖先）。因为要从根节点开始逐个预处理，所以最好用孩子表示法或者邻接表存储数据。

算法步骤

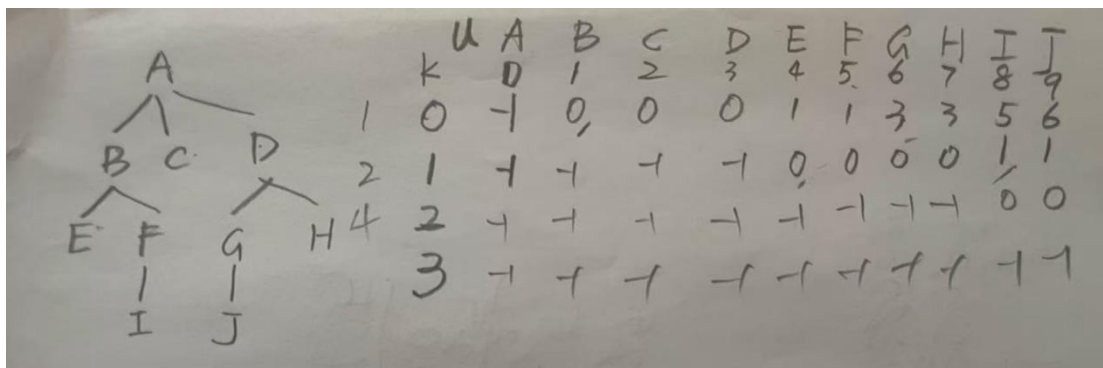
（1）预处理阶段

计算每个节点的深度 $depth[u]$ （DFS 或 BFS 遍历）。

动态规划计算 $up[u][k]$ ：

$up[u][0] = parent[u]$ （直接父节点）。

$up[u][k] = up[up[u][k-1]][k-1]$ （递归计算）。



(2) 查询阶段

调整节点深度：将较深的节点 u 向上跳至与 v 同一深度。

从 $k = \log_2(\text{depth}[u] - \text{depth}[v])$ 开始尝试跳跃。

同步向上跳跃：

若 $\text{up}[u][k] \neq \text{up}[v][k]$ ，则同时跳跃 2^k 步。

最终 u 和 v 的父节点即为 LCA。

```
#include<bits/stdc++.h>

using namespace std;

const int MAXN = 100005; // 最大节点数
const int LOG = 20;      // log2(MAXN)向上取整

vector<int> tree[MAXN];    // 树的邻接表表示
int up[MAXN][LOG], maxk;   // up[u][k]表示 u 的 2^k 级祖先
int depth[MAXN];          // 每个节点的深度

void dfs(int u, int parent) {
    up[u][0] = parent; // 直接父节点
    // 计算 2^k 级祖先
    for (int k = 1; k < maxk; ++k) {
        if (up[u][k-1] != -1) {
            up[u][k] = up[up[u][k-1]][k-1];
        } else {
            up[u][k] = -1;
        }
    }
}
```

```

// 递归处理子节点
for (int v : tree[u]) {
    if (v != parent) {
        depth[v] = depth[u] + 1;
        dfs(v, u);
    }
}
}

void preprocess(int root, int n) { // 预处理
    // 初始化
    for (int i = 0; i < n; ++i) {
        for (int k = 0; k < maxk; ++k) {
            up[i][k] = -1;
        }
    }
    depth[root] = 0;
    dfs(root, -1); // root 的父节点设为-1
}

int lca(int u, int v) { // 查询 u 和 v 的 LCA
    // 确保 u 是较深的节点
    if (depth[u] < depth[v]) swap(u, v);

    // 将 u 跳到与 v 同一深度
    for (int k = maxk-1; k >= 0; --k) {
        if (depth[u] - (1 << k) >= depth[v]) {
            u = up[u][k];
        }
    }

    // 如果此时 u 和 v 相同，直接返回
    if (u == v) return u;
}

```

```

// 同步向上跳跃
for (int k = maxk-1; k >= 0; --k) {
    if (up[u][k] != up[v][k]) {
        u = up[u][k];
        v = up[v][k];
    }
}

// 最终父节点就是 LCA
return up[u][0];
}

void solve(){
    // 示例树结构，用邻接表存无向图的方式
    //      0
    //     /|\
    //    1 2 3
    //   /\ /\
    //  4 5 6 7
    //   /  \
    //  8   9
    tree[0] = {1,2,3};
    tree[1] = {0,4,5};
    tree[2] = {0};
    tree[3] = {0,6,7};
    tree[4] = {1};
    tree[5] = {1,8};
    tree[6] = {3,9};
    tree[7] = {3};
    tree[8] = {5};
    tree[9] = {6};

    int n = 10; // 节点总数

    int root = 0;

```

```

    maxk=log(n)+1;

    // 预处理
    preprocess(root, n);

    // 查询示例
    cout << "LCA(6,7) = " << lca(6,7) << endl; // 应该输出 2
    cout << "LCA(4,8) = " << lca(4,8) << endl; // 应该输出 0
    cout << "LCA(8,9) = " << lca(8,9) << endl; // 应该输出 0

    return;
}

int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);cout.tie(0);
    //freopen("P.in","r",stdin);
    //freopen("***.out","w",stdout);
    int T=1;//cin>>T;
    while(T--){
        solve();
    }
    return 0;
}

```

如果节点数据跟下标不一致，可以用以下方式建立两个字典表，基础的 LCA 程序不用变

```

unordered_map<char,int> mpint;//从字符到下标的字典
char mpchar[105];//从下标到字符的字典
void solve(){
    int n=0;
    mpchar[n]='A';//分别创建字典
    mpint['A']=n;
    n++;
    if('B'==mpchar[mpint['B']]){ //B 节点已经存在

```

```

        ....
    }
    else
    {
        mpchar[n]='B';//B 节点加入字典
        mpint['B']=n;
        n++;
    }
}

```

完成程序如下：

代码块

```

#include<bits/stdc++.h>
using namespace std;
#define ll long long
#define el endl
vector<vector<int>>> tree,up;//分别存树（无向）和 up 表
unordered_map<char,int> c2Int;//存节点数据的下标，方便查询
vector<char> i2Char;//存节点数据
vector<int> toRoot,depth;//roRoot 是桶数组，用来找根节点下标。depth 保存节点深度
int e,n,m,id,root,maxk;//分别表示边数、节点数、查询数等。
char c,d;
void dic(char c){
    if(c!=i2Char[c2Int[c]]){//这个 if 是去除子->父类型的边
        i2Char[id]=c;
        c2Int[c]=id;
        id++;
    }
}
void dfs(int u,int parent){
    up[u][0]=parent;
    //计算 u 节点的 2^k 级祖先
    for(int k=1;k<maxk;k++){

```

```

        if(up[u][k-1]!=-1) up[u][k]=up[up[u][k-1]][k-1];
        else up[u][k]=-1;
    }
    //递归处理子节点
    for(int v:tree[u]){
        if(v!=parent){
            depth[v]=depth[u]+1;
            dfs(v,u);
        }
    }
}

void prep(){//预处理
    for(int i=0;i<n;i++){
        {
            for(int j=0;j<maxk;j++) up[i].push_back(-1);
        }
        depth[root]=0;
        dfs(root,-1);
    }
}

int lca(int u,int v){
    if(depth[u]<depth[v]) swap(u,v);//确保 u 更深

    if(depth[u]!=depth[v]){
        //将 u 和 v 调到统一高度
        for(int k=maxk-1;k>=0;k--){
            if(depth[u]-(1<=k)>=depth[v]){
                u=up[u][k];
            }
        }
    }

    if(u==v) return u;//u 和 v 相同，本身就是 lca
    for(int k=maxk-1;k>=0;k--){//同步上升，不同就上移
        if(up[u][k]!=up[v][k]){

```

```

        u=up[u][k];
        v=up[v][k];
    }
}
return up[u][0];
}

void solve(){
    cin>>e;n=e+1;
    i2Char.resize(n);
    tree.resize(n);
    toRoot.resize(n);
    depth.resize(n);
    up.resize(n);
    maxk=log(n)+1;
    for(int i=0;i<e;i++){
        cin>>c>>d;
        dic(c);dic(d);
        int idc=c2Int[c],idd=c2Int[d];
        tree[idc].push_back(idd);
        tree[idd].push_back(idc);
        toRoot[idc]=1;
    }
    for(int i=0;i<n;i++) { //查找树根
        if(toRoot[i]==0) {
            root=i;
            break;
        }
    }
    prep();//预处理
    // for(int i=0;i<n;i++)
    // {
    //     for(int j=0;j<maxk;j++)
    //     {

```

```

//    cout<<up[i][j]<<" ";
// }
// cout<<el;
// }
int m;cin>>m;
while(m--){
    cin>>c>>d;
    cout<<"an:"<<i2Char[lca(c2Int[c],c2Int[d])]<<el;
}
}
int main(){
    ios::sync_with_stdio(0);
    cin.tie(0);cout.tie(0);
    // freopen("***.in","r",stdin);
    // freopen("***.out","w",stdout);
    int T=1;//cin>>T;
    while(T--){
        solve();
    }
    return 0;
}

```

/*

输入：第一行两个整数 e 和 m，表示有 e 条边和 m 次查询,节点数 n=e+1

后面 n 行，每行两个字符以空格间隔，表示第一个字符节点的父节点是第二个字符节点。

再后面 m 行，每行两个字符以空格间隔，表示查询第一个字符节点与第二个字符节点的

LCA

9

B A

C A

D A

E B

F B

G D

```
H D
I F
J G
5
A B
E F
I J
G J
C E
*/
```

Tarjan 离线算法 (Tarjan's Offline LCA)

思路

使用 并查集 (Union-Find) 和 DFS 遍历，在遍历过程中处理查询。

适用于批量查询 (离线算法)。

时间复杂度

预处理: $O(n + q)$ (n 是节点数, q 是查询数)。

查询: $O(1)$ (所有查询在 DFS 过程中处理)。

适用场景: 适用于静态树 (查询已知)。

RMQ 转换法 (Euler Tour + RMQ)

思路

通过 欧拉遍历 (Euler Tour) 将树转换为线性结构，然后用 RMQ (Range Minimum Query) 计算 LCA。

适用于多次查询 (在线算法)。

时间复杂度

预处理:

欧拉遍历: $O(n)$

RMQ 预处理: $O(n \log n)$ (稀疏表) 或 $O(n)$ (线段树)。

查询: $O(1)$ (稀疏表) 或 $O(\log n)$ (线段树)。

适用场景: 适用于频繁查询。

63. 二叉搜索树

二叉搜索树是一棵二叉树同时满足每个父节点的左孩子都小于父节点，右孩子都不小于父节点。

```
#include<bits/stdc++.h>

using namespace std;

struct node{
    int data;
    node *left=NULLptr,*right=NULLptr;
};

node *root;

int a[15]={5,1,7,4,6,2,9,7,3,6},n=10;

void tianjia(node *&r,int x)
{
    if(r)
        if(x<r->data) tianjia(r->left,x);
        else tianjia(r->right,x);
    else
        r=new node,r->data=x;
}

void middleView(node *&r)
{
    if(r->left) middleView(r->left);
    cout<<r->data<<' ';
    if(r->right) middleView(r->right);
}

int bfs(node *&r,int x)
{
    node *p[1005];
    int front=0,rear=0;
    p[++rear]=r;
    while(front<rear)
    {
```

```

        node *q=p[++front];

        if(q->data==x) return 1;

        if(q->left) p[++rear]=q->left;

        if(q->right) p[++rear]=q->right;

    }

    return 0;
}

int search(node *&r,int x)
{
    if(r)
    {
        if(r->data==x) return 1;

        else if(x<r->data) return search(r->left,x);

        else return search(r->right,x);

    }

    else return 0;
}

int jishu(node *&r,int x)
{
    if(r)
    {
        if(r->data==x) return 1+jishu(r->right,x);

        else if(x<r->data) return jishu(r->left,x);

        else return jishu(r->right,x);

    }

    else return 0;
}

int main()
{
    for(int i=0;i<n;i++) tianjia(root,a[i]);

    middleView(root);

    int x;cin>>x;

    cout<<bfs(root,x);

```

```
cin>>x;

cout<<jishu(root,x);

return 0;
}
```

64. 字典树 (Trie 树/前缀树)

一种树形数据结构，专门用于高效存储和检索字符串集合。它的核心思想是 共享公共前缀，从而减少存储空间并提高查询效率。

字典树的每个节点包含：

子节点指针 (children)：通常用数组或哈希表存储。

结束标记 (isEndOfWord)：标记从根到当前节点的路径是否构成一个完整单词。

核心特性：

前缀共享：例如 "apple" 和 "app" 共享前缀 "app"。

快速查找：查找时间复杂度为 $O(m)$ (m 是单词长度)。

空间换时间：相比哈希表，Trie 可能占用更多内存，但能高效支持前缀查询。

```
#include <bits/stdc++.h>

using namespace std;

// Trie 节点结构
struct TrieNode {
    unordered_map<char, TrieNode*> children;
    bool isEndOfWord = false;
};

// 插入单词
void insert(TrieNode* root, const string& word) {
    TrieNode* current = root;
    for (char c : word) {
        if (current->children.find(c) == current->children.end()) {
            current->children[c] = new TrieNode();
        }
    }
}
```

```

    }

    current = current->children[c];
}

current->isEndOfWord = true;
}

// 搜索完整单词
bool search(TrieNode* root, const string& word) {
    TrieNode* current = root;
    for (char c : word) {
        if (current->children.find(c) == current->children.end()) {
            return false;
        }
        current = current->children[c];
    }
    return current->isEndOfWord;
}

```

```

// 检查前缀是否存在
bool startsWith(TrieNode* root, const string& prefix) {
    TrieNode* current = root;
    for (char c : prefix) {
        if (current->children.find(c) == current->children.end()) {
            return false;
        }
        current = current->children[c];
    }
    return true;
}

```

```

// 打印所有单词 (DFS 遍历)
void printTrie(TrieNode* node, string currentWord) {
    if (node->isEndOfWord) {

```

```

        cout << currentWord << endl;
    }
    for (auto& pair : node->children) {
        printTrie(pair.second, currentWord + pair.first);
    }
}

int main() {
    TrieNode* root = new TrieNode();
    insert(root, "apple");
    insert(root, "app");
    insert(root, "banana");

    cout << "Search 'apple': " << (search(root, "apple") ? "Found" : "Not Found") <<
endl;
    cout << "Search 'app': " << (search(root, "app") ? "Found" : "Not Found") << endl;
    cout << "Search 'orange': " << (search(root, "orange") ? "Found" : "Not Found") <<
endl;

    cout << "\nPrefix 'app' exists: " << (startsWith(root, "app") ? "Yes" : "No") << endl;
    cout << "Prefix 'ora' exists: " << (startsWith(root, "ora") ? "Yes" : "No") << endl;

    cout << "\nAll words in Trie:" << endl;
    printTrie(root, "");

    return 0;
}

```

65. 堆与堆排序

堆是一种完全二叉树，其符合每个父节点不小于或者不大于其子节点的特性。

堆排序适用于 Top K 问题、实时取最值、优先队列实现等场景。

堆的基本操作：

存储：用数组从下标 1 开始存储， i 节点的左孩子是 $i*2$ ，右孩子是 $i*2+1$ ，父节点是 $i/2$ ，最后一个非

叶子节点是 $n/2$ 。

up 操作：从某个节点开始，依次向上检查跟父节点的关系，直到关系正确或者到达堆顶。

```
void up(int u){
    while(u/2 && h[u/2]> h[u]){
        swap(h[u],h[u/2]);
        u /= 2;
    }
}
```

down 操作：从某个节点开始，依次向下检查跟子节点的关系，直到关系正确或者越界。如果没有插入节点和修改节点操作，只用 down 操作就可以完成堆排序，只是 TOP K 时堆会更大数量是 n 不是 K 。

```
void down(int u)
{
    int t = u;
    if(u*2<= size && h[u*2]< h[t]) t = u*2;
    if(u*2 + 1<= size && h[u*2+1]< h[t]) t = u*2 + 1;
    if(t != u){
        swap(h[u],h[t]);
        down(t);
    }
}
```

插入数据：heap[++n]=t; up(n);

取堆顶：heap[1];

删除堆顶：heap[1]=heap[n--]; down(1);

删除堆中元素：heap[x]=heap[n--]; up(x); down(x);

修改堆顶：heap[1]=x; down(1);

修改堆中元素：heap[x]=y; up(x); down(x);

堆排序若不需要中途插入数据时，可以直接输入完数组后（下标 $1-n$ ），从最后一个非叶子节点（ $n/2$ ）做 down 操作，依次往前处理每个非叶子节点，即可完成建堆操作。取数据时，将堆顶跟最后一个元素交换，然后对堆顶做 down 操作（注意 down 操作的数据范围逐渐减小）。

Top K：求 Top K 大，就是维护一个 K 个元素的小顶堆，如果后面的元素比堆顶大就替换掉堆顶，并做堆调整。时间复杂度是 $O(n\log K)$

算法	平均时间复杂度	空间复杂度	稳定性	适用场景
堆排序	$O(n \log n)$	$O(1)$	不稳定	大规模数据、Top K
快速排序	$O(n \log n)$	$O(\log n)$	不稳定	通用排序、小内存
归并排序	$O(n \log n)$	$O(n)$	稳定	外部排序、稳定性要求高

代码块

```
#include<bits/stdc++.h>

using namespace std;

#define ll long long
#define el "\n"

ll n=10,a[15]={6,2,4,9,5,1,5,4,7,2};

void heapify(ll m,ll root){//就是 down 操作
    ll largest=root;//默认 a[root]最大，当擂主
    ll left=root*2+1,right=root*2+2;//计算左右孩子的下标
    if(left<m&& a[left]>a[largest]) largest=left;//左孩子打擂台
    if(right<m&& a[right]>a[largest]) largest=right;//右孩子打擂台
    if(largest!=root){//如果擂主更换了
        swap(a[root],a[largest]);//交换 root 节点和新擂主节点
        heapify(m,largest);//递归调整子树
    }
}

void heapSort(){
    for(ll i=n/2-1;i>=0;i--){//倒序遍历每个非叶子结点
        heapify(n,i);//建堆
    }
    for(ll i=n-1;i>=0;i--){
        swap(a[0],a[i]);//交换堆顶和最后一个元素（乱序范围内的）
        heapify(i,0);//调整剩余元素为大顶堆
    }
}

void solve(){
    heapSort();
}
```

```

    for(ll i=0;i<n;i++) cout<<a[i]<<" ";
}

```

Top K 问题：把前 K 个元素建小顶堆，然后从第 k+1 的元素开始，把比堆顶大的元素换到堆顶然后做堆调整，遍历到最后一个元素。从第 k 个元素到第 1 个元素分别与堆顶交换，并做堆调整，使数组的前 k 个元素形成从大到小的序列。

Top K 问题，也可以用优先队列解决，总体思路是：用优先队列建小顶堆。遍历数组元素，如果堆元素数量小于 K，就直接入队，否则如果比堆顶大就弹出堆顶并入队。因为是小顶堆，从大到小输出时，需要先讲堆顶依次放进结果数组，倒数输出结果数组。

代码块

```

#include<bits/stdc++.h>

using namespace std;

#define ll long long
#define el "\n"

ll n=10,a[15]={6,2,4,9,5,1,5,4,7,2},k=4;

void heapify(ll m,ll root){ //小顶堆
    ll minest=root;
    ll left=root*2+1,right=root*2+2;
    if(left<m&& a[left]<a[minest]) minest=left;
    if(right<m&& a[right]<a[minest]) minest=right;
    if(minest!=root) {
        swap(a[minest],a[root]);
        heapify(m,minest);
    }
}

void solve(){
    for(ll i=k/2-1;i>=0;i--) heapify(k,i); //将前 k 个元素建堆
    for(ll i=k;i<n;i++){ //从第 k+1 个元素开始，比堆顶大的元素进入堆并做调整
        if(a[i]>a[0]){
            swap(a[i],a[0]);
            heapify(k,0);
        }
    }

    for(ll i=k-1;i>=0;i--){ //依次把堆顶换到最后面，调整堆，形成从大到小的序列

```

```

        swap(a[0],a[i]);
        heapify(i,0);
    }
    for(ll i=0;i<k;i++) //输出前 k 个元素
        cout<<a[i]<<" ";
}

```

用优先队列实现如下：

代码块

```

#include<bits/stdc++.h>
using namespace std;
#define ll long long
#define el "\n"
ll n=10,a[15]={6,2,4,9,5,1,5,4,7,2},k=4,res[15];
priority_queue<ll,vector<ll>,greater<ll> > pq;//greater 比较器对应小根堆
void solve(){
    for(ll i=0;i<n;i++){
        if(i<k) pq.push(a[i]); //前 k 个直接入堆
        else {
            if(pq.top()<a[i]){ //大于堆顶的元素，弹出堆顶并入堆
                pq.pop();
                pq.push(a[i]);
            }
        }
    }
    for(ll i=0;i<k;i++){ //取堆顶放入结果数组
        res[i]=pq.top();
        pq.pop();
    }
    for(ll i=k-1;i>=0;i--)cout<<res[i]<<" "; //倒着输出结果数组
}

```

用堆排序解决对结构体的数据的 TOP K 问题时，需要注意两处：

1.用 priority_queue 时，默认用 less 比较器得到大顶堆，对应重载小于号，希望得到小顶堆就要在重

载小于号是将关系规则颠倒。

2.自写 up 函数时，数组元素从 1 开始，用 $i/2$ 得到父节点编号。

如信友队 <https://www.xinyoudui.com/ac/contest/27600000C000006000006F6/problem/1613>

两种实现方式分别是：

用 priority_queue 的：

代码块

```
#include<bits/stdc++.h>

using namespace std;

#define ll long long

struct cjd{
    ll id,avg,fch;
    bool operator <(const cjd &b)const{//下面三处规则颠倒一下
        if(avg!=b.avg) return avg>b.avg;
        else if(fch!=b.fch) return fch<b.fch;
        return id<b.id;
    }
    void out(){
        cout<<id<<":"<<avg<<":"<<fch<<endl;
    }
};

ll n,m,k,t,avg,fch;

priority_queue<cjd> pq;

//默认用 less 比较器得到大顶堆，对应重载小于号，要得到小顶堆小于号比较规则需要颠倒

int main(){
    cin>>n>>m>>k;
    for(ll i=1;i<=n;i++){
        avg=0;fch=0;
        for(int j=0;j<k;j++){
            cin>>t;
            avg+=t;
            fch+=t*t;
        }
    }
}
```

```

    }

    avg/=k;
    fch=(fch-k*avg*avg)/k;
    cjd cj={i,avg,fch};
    if(pq.size()<m) pq.push(cj);
    else {
        if(cj<pq.top()){//只重载了<, 所以只能用<, 注意<规则颠倒了
            pq.pop();
            pq.push(cj);
        }
    }
}

vector<cjd> vt;
while(!pq.empty()){
    vt.push_back(pq.top());
    pq.pop();
}
reverse(vt.begin(),vt.end());
for(int i=0;i<vt.size();i++)
    cout<<vt[i].id<<" ";
return 0;
}

```

自写堆排序的:

代码块

```

#include<bits/stdc++.h>

using namespace std;

#define ll long long

struct cjd{
    ll id,avg,fch;

    bool operator <(const cjd &b)const{
        //直接按照从小到大排序的方式写, 我们取 1-K 个元素建堆。
    }
}

```

```

        if(avg!=b.avg) return avg<b.avg;
        else if(fch!=b.fch) return fch>b.fch;
        return id>b.id;
    }
};

cjd res[100];
ll n,m,k,t,avg,fch,cnt;
void up(int i){
    //用循环实现
    // while(i/2&&res[i]<res[i/2]){
    //     swap(res[i],res[i/2]);
    //     i/=2;
    // }
    //用递归实现
    if(i/2&&res[i]<res[i/2]){
        swap(res[i],res[i/2]);
        up(i/2);
    }
}

void down(int i){
    //用循环实现
    // while(1){
    //     //先比左孩子，再比右孩子
    //     int k=i;
    //     if(2*i<=cnt&&res[2*i]<res[k]) k=2*i;
    //     if(2*i+1<=cnt&&res[2*i+1]<res[k]) k=2*i+1;
    //     if(k!=i){
    //         swap(res[k],res[i]);
    //         i=k;
    //     }
    //     else break;

    //先找较小的孩子，再跟父亲比。

```

```
//      int lc=2*i,rc=2*i+1,minc=lc;
//      if(rc<=cnt&&res[rc]<res[lc]) minc=rc;
//      if(minc<=cnt&&res[minc]<res[i]){
//          swap(res[minc],res[i]);
//          i=minc;
//      }
//      else break;
//  }
```

//递归实现

```
int k=i;
if(i*2<=cnt&&res[i*2]<res[k]) k=2*i;
if(i*2+1<=cnt&&res[i*2+1]<res[k]) k=2*i+1;
if(k!=i) {
    swap(res[i],res[k]);
    down(k);
}
}
```

```
int main()
{
    cin>>n>>m>>k;
    for(ll i=1;i<=n;i++){
        avg=0;fch=0;
        for(int j=0;j<k;j++){
            cin>>t;
            avg+=t;
            fch+=t*t;
        }
        avg/=k;
        fch=(fch-k*avg*avg)/k;
        //cout<<avg<<":"<<fch<<endl;
        cjd cj={i,avg,fch};
```

```

        if(cnt<m){
            res[++cnt]=cj;
            up(cnt);
            //res[1].out();
        }
        else if(res[1]<cj){
            res[1]=cj;
            down(1);
            //res[1].out();
        }
    }
    while(cnt>1){
        swap(res[1],res[cnt]);
        cnt--;
        down(1);
    }
    for(int i=1;i<=m;i++){
        //res[i].out();
        cout<<res[i].id<<" ";
    }
    return 0;
}

```

66. 优先队列

优先队列的应用场景除了堆排序（原地排序，不需要额外存储空间）、Top K、等问题外，还特别适合要在频繁插入和删除数据并同时输出最值和中间值的情况（实时数据流统计）。如：实时取异常比赛的前 10% 的分数线（对顶堆）。

闹钟的功能也可以用优先队列实现的，新建闹钟时入小根堆，删除时用数组标记（非 stl 实现的可以删除），每秒取堆顶，如果时间匹配就触发闹钟并弹出重复取堆顶处理，堆顶时间小于当前时间就弹出。

67. 拓扑排序

拓扑排序可以用于有向无环图(DAG)按方向排序，实际问题中的方向可以是先后顺序、依赖关系或者因果关系等。

拓扑排序算法还可以用于解决有向无环图中两点间的最短路问题。

拓扑排序的结果并不唯一。时间复杂度 $O(V+E)$

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long
const ll N=1e5+5;
ll n,m,x,y,in[N],out[N],res[N];
vector<vector<ll> > mp(N);
queue<ll> q;
void solve(){
    cin>>n>>m;
    for(int i=0;i<m;i++)
    {
        cin>>x>>y;
        out[x]++;
        in[y]++;
        mp[x].push_back(y);
    }
    for(int i=1;i<=n;i++)
    {
        if(!in[i])
        {
            res[i]=1;
            q.push(i);
        }
    }
    while(q.size())
    {
        ll ft=q.front();
        q.pop();
        for(int i=0;i<mp[ft].size();i++)
        {
            ll next=mp[ft][i];
```

```

        in[next]--;
        //if(res[ft]+1>res[next]) res[next]=res[ft]+1;
        if(!in[next])
        {
            res[next]=res[ft]+1;
            q.push(next);
        }
    }
}

for(int i=1;i<=n;i++) cout<<res[i]<<"\n";
}

int main(){
    ios::sync_with_stdio(0);
    cin.tie(0);cout.tie(0);
    //freopen("xxx.in","r",stdin);
    //freopen("***.out","w",stdout);
    int T=1;//cin>>T;
    while(T--){
        solve();
    }
    return 0;
}

```

68. 笛卡尔树

每个节点是一个数值对 (**key**: **value**) , **key** 是序号, **value** 是数值, 可以分别对应数组的下标和元素值。笛卡尔树是同时满足以下两个性质的二叉树:

1. **key** 满足二叉排序树的性质 (即每个节点的左子节点都小于此节点, 右子节点都不小于此节点)
2. **value** 满足堆的性质 (如小根堆: 每个父节点都小于其子节点)

笛卡尔树的构建: 单调递增栈法:

1. 遍历数组 (*i* 是 **key**, *a[i]* 是 **value**)
2. 对每个数组元素
 - a. 创建节点
 - b. 维护递增栈 (元素是节点指针): 把小于当前元素值的出栈, 最后一个出栈的作为当前节

点的左孩子。栈为空时直接进栈同时更新树根为当前节点。栈不为空时，当前节点作为栈顶元素的右孩子。

笛卡尔树可以高效解决 RMQ（区间最值问题），构建和 RMQ 代码如下：

```
// 笛卡尔树节点结构
struct Node {
    int value;
    int index;    // 在原数组中的位置
    Node* left=nullptr;
    Node* right=nullptr;
    Node(int v,int id){value=v;index=id;}
};

// 使用单调栈构建笛卡尔树（小根堆）
Node* buildCartesianTree(const vector<int>& arr) {
    stack<Node*> st;
    Node* root = nullptr;

    for (int i = 0; i < arr.size(); ++i) {
        Node* curr = new Node(arr[i], i);

        // 弹出栈中比当前值大的节点
        while (!st.empty() && st.top()->value > arr[i]) {
            curr->left = st.top();
            st.pop();
        }

        // 连接新节点到栈顶节点的右孩子
        if (!st.empty()) {
            st.top()->right = curr;
        } else {
            root = curr; // 栈为空时当前节点成为根
        }
    }
}
```

```

        st.push(curr);
    }
    return root;
}

// 在笛卡尔树中查询区间最小值
int queryMin(Node* root, int start, int end) {
    if (!root) return INT_MAX;
    if (root->index >= start && root->index <= end) {
        return root->value;
    }
    if (root->index > end) {
        return queryMin(root->left, start, end);
    }
    return queryMin(root->right, start, end);
}

```

69. 图的基本算法

1. 相关概念

1.1 连通分量

连通分量：在无向图中，相互通连的最大子图。

强连通分量：在有向图中，若一些点可以相互通达的最大子图。单个孤立的节点也是强连通分量。

1.2 树边与回边

树边是 DFS 遍历过程中首次访问某节点时经过的边，它们构成了 DFS 树(或 DFS 森林)的骨架。其特点是：

- a. 通过树边发现的节点都是未被访问过的
- b. 在代码中表现为 `if(!dfn[v])` 的条件分支
- c. 在无向图中，所有树边都是双向的

回边指向 DFS 树中已访问的祖先（除父亲）节点的边，它们揭示了图中存在的环路。其特点是：

- a. 目标节点必须已被访问且在栈中 （对于有向图）
- b. 在代码中表现为 `else if(inStack[v])`（有向图）或 `else if(v != parent[u])`（无向图）
- c. 在无向图中，所有非树边都是回边

- d. 形成环路的关键证据，有回边就可以认为图中存在回路。

1.3 割点与割边

割点：在无向图中，如果删除该顶点及其关联的边后，图的连通分量数量增加，则该顶点称为割点。

割边：在无向图中，割边（也称为桥）是图中的一条边，如果删除这条边，图的连通分量数量会增加。

1.4 双连通分量

边双连通分量：无向图中删除任意一条边后仍保持连通的极大子图

点双连通分量：无向图中删除任意一个顶点后仍保持连通的极大子图

关键性质：

- 边双连通图中没有桥（割边）
- 点双连通图中没有割点
- 两个边双连通分量之间由桥连接，所以发现桥（ $low[v] > dfn[u]$ ）时，就将 dfs 栈出栈到 u 即边双连通分量。

边双连通分量

```
void tarjan(int u, int parent) {
    dfn[u] = low[u] = ++idx;
    stk.push(u);

    for (int v : G[u]) {
        if (v == parent) continue; // 跳过父节点

        if (!dfn[v]) {
            tarjan(v, u);
            low[u] = min(low[u], low[v]);

            // 发现桥时提取边双
            if (low[v] > dfn[u]) {
                vector<int> ebcc;
                while (true) {
                    int x = stk.top(); stk.pop();
                    ebcc.push_back(x);
                    if (x == v) break;
                }
            }
        }
    }
}
```

```

        ebccs.push_back(ebcc);
    }
} else {
    low[u] = min(low[u], dfn[v]); // 回边更新
}
}
}
}

```

点双连通分量

```

void tarjan(int u, int parent) {
    dfn[u] = low[u] = ++idx;
    stk.push(u);
    int children = 0;

    for (int v : G[u]) {
        if (v == parent) continue;

        if (!dfn[v]) {
            children++;
            tarjan(v, u);
            low[u] = min(low[u], low[v]);

            // 发现割点时提取点双
            if (low[v] >= dfn[u]) {
                vector<int> bcc;
                while (true) {
                    int x = stk.top(); stk.pop();
                    bcc.push_back(x);
                    if (x == v) break;
                }
                bcc.push_back(u); // 割点属于多个点双
                bccs.push_back(bcc);
            }
        }
    }
}

```

```

    } else {
        low[u] = min(low[u], dfn[v]);
    }
}
}

```

2. 判断是否有环（或者判断图是否是树）

用邻接表存储图，首先弄事判断无向环还是有向环。

思路一：用 bfs 或者 dfs 遍历图，如果当前节点未被访问过，就标记已访问继续搜索，如果当前节点已经访问过，并且不是前驱节点（无向图）那么就存在环。

思路二：用并查集，在遍历图的过程中不断合并集合，如果遇到两个点没有合并之前就已经属于一个集合那么就存在环。

```

#include <bits/stdc++.h>
using namespace std;

bool hasCycleDFS(int node, int parent,
                const unordered_map<int, vector<int>>& graph,
                vector<bool>& visited) {
    visited[node] = true;

    for (int neighbor : graph.at(node)) {
        if (!visited[neighbor]) {
            if (hasCycleDFS(neighbor, node, graph, visited)) {
                return true;
            }
        }
        else if (neighbor != parent) {
            return true;
        }
    }

    return false;
}

```

```

bool isTreeDFS(const unordered_map<int, vector<int>>& graph) {
    if (graph.empty()) return true;

    vector<bool> visited(graph.size(), false);

    // 检查是否有环
    if (hasCycleDFS(graph.begin()->first, -1, graph, visited)) {
        return false;
    }

    // 检查是否所有节点都被访问过（连通性）
    for (const auto& pair : graph) {
        if (!visited[pair.first]) {
            return false;
        }
    }

    return true;
}

```

```

#include <bits/stdc++.h>
using namespace std;

bool isTreeBFS(const unordered_map<int, vector<int>>& graph) {
    if (graph.empty()) return true;

    vector<bool> visited(graph.size(), false);
    unordered_map<int, int> parent;
    queue<int> q;

    int start = graph.begin()->first;
    q.push(start);

```

```

visited[start] = true;
parent[start] = -1;

while (!q.empty()) {
    int current = q.front();
    q.pop();

    for (int neighbor : graph.at(current)) {
        if (!visited[neighbor]) {
            visited[neighbor] = true;
            parent[neighbor] = current;
            q.push(neighbor);
        }
        else if (neighbor != parent[current]) {
            return false; // 发现环
        }
    }
}

// 检查连通性
for (const auto& pair : graph) {
    if (!visited[pair.first]) {
        return false;
    }
}

return true;
}

```

```

#include <bits/stdc++.h>

using namespace std;

int findRoot(vector<int>& parent, int node) {

```

```

    if (parent[node] != node) {
        parent[node] = findRoot(parent, parent[node]);
    }
    return parent[node];
}

bool unionNodes(vector<int>& parent, vector<int>& rank, int x, int y) {
    int rootX = findRoot(parent, x);
    int rootY = findRoot(parent, y);

    if (rootX == rootY) {
        return false; // 已经在同一集合，说明有环
    }

    if (rank[rootX] > rank[rootY]) {
        parent[rootY] = rootX;
    }
    else if (rank[rootX] < rank[rootY]) {
        parent[rootX] = rootY;
    }
    else {
        parent[rootY] = rootX;
        rank[rootX]++;
    }

    return true;
}

bool isTreeUF(const unordered_map<int, vector<int>>& graph) {
    if (graph.empty()) return true;

    // 初始化并查集
    int n = graph.size();

```

```

vector<int> parent(n);
vector<int> rank(n, 0);
for (int i = 0; i < n; ++i) {
    parent[i] = i;
}

// 处理所有边
for (const auto& pair : graph) {
    int u = pair.first;
    for (int v : pair.second) {
        if (!unionNodes(parent, rank, u, v)) {
            return false; // 发现环
        }
    }
}

// 检查连通性
int root = findRoot(parent, 0);
for (int i = 1; i < n; ++i) {
    if (findRoot(parent, i) != root) {
        return false;
    }
}

return true;
}

```

```

int main() {
    // 构建一个无环图（树）
    unordered_map<int, vector<int>> tree = {
        {0, {1, 2}},
        {1, {0, 3, 4}},
        {2, {0, 5}},
    }
}

```

```

    {3, {1}},
    {4, {1}},
    {5, {2}}
};

// 构建一个有环图
unordered_map<int, vector<int>> cyclicGraph = {
    {0, {1, 2}},
    {1, {0, 2}},
    {2, {0, 1}}
};

cout << "DFS - Tree: " << isTreeDFS(tree) << endl;
cout << "DFS - Cyclic: " << isTreeDFS(cyclicGraph) << endl;

cout << "BFS - Tree: " << isTreeBFS(tree) << endl;
cout << "BFS - Cyclic: " << isTreeBFS(cyclicGraph) << endl;

cout << "UF - Tree: " << isTreeUF(tree) << endl;
cout << "UF - Cyclic: " << isTreeUF(cyclicGraph) << endl;

return 0;
}

```

3. 判断图的连通性

广搜或者深搜都可以，从某个点按方向搜索一遍并标记为已访问。标记完成后，如果有点没有被访问过，那么就是未连通的，否则就是连通的。

```

#include <bits/stdc++.h>

using namespace std;

bool isConnected(const vector<vector<int>>& adj) {
    int n = adj.size();

```

```

vector<bool> visited(n, false);
queue<int> q;

q.push(0);
visited[0] = true;

while (!q.empty()) {
    int node = q.front();
    q.pop();

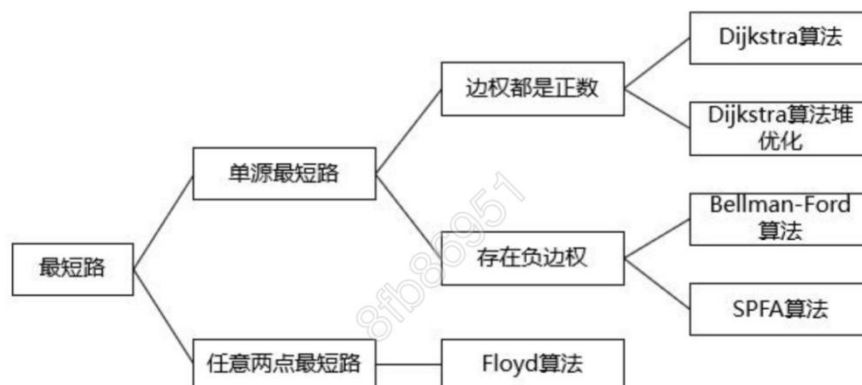
    for (int neighbor : adj[node]) {
        if (!visited[neighbor]) {
            visited[neighbor] = true;
            q.push(neighbor);
        }
    }
}

for (bool v : visited) {
    if (!v) return false;
}

return true;
}

```

70. 最短路算法总结



1. 对比表格

算法	算法思想	适用场景	时间复杂度	空间复杂度	特点
Dijkstra	贪心策略，每次扩展当前距离起点最近的节点	无负权边的单源最短路	$O((V+E)\log V)$	$O(V)$	优先队列优化，不能处理负权边
Bellman-Ford	动态规划思想，通过V-1次松弛所有边来找到最短路径	含负权边的单源最短路	$O(VE)$	$O(V)$	可检测负权环，适合稀疏图
SPFA	Bellman-Ford的队列优化，只松弛那些距离发生变化的节点	Bellman-Ford的优化版本	平均 $O(E)$ 最坏 $O(VE)$	$O(V)$	实际效率常优于Bellman-Ford
Floyd-Warshall	动态规划，通过中间节点k逐步优化所有节点对的最短距离	所有节点对的最短路径	$O(V^3)$	$O(V^2)$	代码简洁，适合稠密小图
拓扑排序+DAG	基于节点入度的线性排序，按拓扑序松弛边	有向无环图的单源最短路	$O(V+E)$	$O(V)$	线性时间复杂度，需先拓扑排序
A*	启发式搜索，结合实际距离g(n)和启发函数h(n)来指导搜索方向	知道目标位置的启发式搜索	取决于启发函数	$O(V)$	路径规划常用，需设计合适h(n)
Johnson	通过势能函数重新赋权，结合Bellman-Ford和Dijkstra的优势	稀疏图的多源最短路	$O(VE\log V)$	$O(V^2)$	适合含负权边的稀疏图

2. 算法思想详解

1. Dijkstra 的贪心思想：

- 维护一个优先队列存储待处理节点
- 每次选择当前距离起点最近的节点进行扩展
- 因为无负权边，所以局部最优等于全局最优

2. Bellman-Ford 的动态规划思想：

- 定义 $dp[k][v]$ 为最多经过 k 条边到达 v 的最短距离
- 通过松弛操作逐步优化： $dp[k][v] = \min(dp[k-1][v], dp[k-1][u] + w(u,v))$
- $V-1$ 次迭代后保证收敛

3. SPFA 的队列优化思想：

- 只有被松弛成功的节点才会影响其他节点
- 使用队列记录距离变化的节点，避免不必要的松弛操作
- 类似 BFS 的思想但可以重复入队

4. Floyd-Warshall 的 DP 状态转移：

- 定义 $dp[k][i][j]$ 为只经过 $1..k$ 节点时 i 到 j 的最短路径
- 状态转移方程： $dp[k][i][j] = \min(dp[k-1][i][j], dp[k-1][i][k] + dp[k-1][k][j])$
- 通过滚动数组优化空间

5. 拓扑排序的依赖解析思想：

- 通过不断移除入度为 0 的节点来建立线性顺序
- 保证处理每个节点时其所有前驱节点都已被处理
- 本质是图的依赖关系解析

6. A* 的启发式搜索思想：

- 评估函数 $f(n)=g(n)+h(n)$ 平衡实际代价和预估代价
- 可采纳性保证($h(n) \leq \text{真实距离}$)确保最优解

- 好的启发函数能显著减少搜索空间

7. Johnson 的势能转换思想：

- 通过势能函数 $h(v)$ 重新赋权： $w'(u,v)=w(u,v)+h(u)-h(v)$
- 保证新图中无负权边，从而可以应用 Dijkstra
- 最后通过势能差还原原始距离

3. 选择策略总结

1. 根据图特征选择：

- 稠密图：Floyd-Warshall（多源）或 Dijkstra（单源）
- 稀疏图：SPFA（有负权）或 Johnson（多源有负权）
- DAG：必选拓扑排序+DAG 最短路

2. 根据需求选择：

- 需要检测负环：Bellman-Ford 或 SPFA
- 知道目标位置：优先考虑 A^*
- 多源查询：Floyd-Warshall（小图）或 Johnson（大稀疏图）

3. 根据性能选择：

- 最坏情况保证：Dijkstra
- 平均性能：SPFA
- 理论最优：Johnson（稀疏多源负权）

4. Floyd 算法

多源最短路，可以负边权，不可以有负权环， $O(N*N*N)$ ，稠密图的多源最短路问题最合适

算法原理：

Floyd-Warshall 算法采用动态规划思想：

1. 定义 $dist[i][j]$ 为 i 到 j 的最短路径
2. 对于每个中间节点 k ，检查 $dist[i][j] > dist[i][k] + dist[k][j]$
3. 更新所有节点对的距离

时间复杂度

- 时间复杂度： $O(V^3)$
- 空间复杂度： $O(V^2)$

```
#include<bits/stdc++.h>
```

```

using namespace std;

#define ll long long
#define el "\n"

const ll N=1e3+5;

ll dis[N][N]; //邻接矩阵

ll n,m,x,y,w,start;

void solve(){

    cin>>n>>m>>start; //n 个点， m 条边， 从 start 开始

    for(int i=1;i<=n;i++) for(int j=1;j<=n;j++)

        if(i!=j) dis[i][j]=LONG_LONG_MAX;

    for(int i=1;i<=m;i++){

        cin>>x>>y>>w; //从 x 到 y 权重是 w 的边

        dis[x][y]=w; //有向图只这一行

        dis[y][x]=w; //无向图加上反向的情况

    }

    for(int k=1;k<=n;k++){

        for(int i=1;i<=n;i++){

            for(int j=1;j<=n;j++){

                if(dis[i][k]!=LONG_LONG_MAX && dis[k][j]!=LONG_LONG_MAX &&

dis[i][j]>dis[i][k]+dis[k][j]){

                    dis[i][j]=dis[i][k]+dis[k][j];

                }

            }

        }

    }

    for(int i=1;i<=n;i++) {

        for(int j=1;j<=n;j++) cout<<dis[i][j]<<" ";

        cout<<endl;

    }

}

int main(){

    ios::sync_with_stdio(0);

    cin.tie(0);cout.tie(0);

```

```

//freopen("***.in","r",stdin);
//freopen("***.out","w",stdout);
int T=1;//cin>>T;
while(T--){
    solve();
}
return 0;
}
/*
6 10 1
1 2 4
1 3 6
2 3 1
2 4 5
3 4 4
3 5 3
4 5 2
4 6 4
5 6 2
2 5 1
*/

```

5. Dijkstra 算法

算法原理:

Dijkstra 算法由荷兰计算机科学家 Edsger Dijkstra 于 1956 年提出，采用贪心策略，适用于无负边权的单源最短路问题。

1. 将起点距离设为 0，其他节点距离设为 ∞
2. 每次从未处理的节点中选择距离最小的节点 u ：可以用优先队列优化，队列中存的是 $\{dist[i], i\}$ 的数对。每个点的数据可以入队多次，所以对堆顶 $first >$ 当前 $dis[i]$ 直接跳过。
3. 对 u 的所有邻居 v 进行松弛操作： $dis[v] = \min(dis[v], dis[u] + weight(u,v))$
4. 重复直到所有节点都被处理

时间复杂度分析:

- 普通数组实现： $O(V^2)$

- 二叉堆实现: $O((V+E)\log V)$
- 斐波那契堆实现: $O(E + V\log V)$

代码块

```
#include<bits/stdc++.h>

using namespace std;

#define ll long long
#define el "\n"
#define prl pair<ll,ll>

vector<vector<prl>> G;//邻接表

ll n,m,x,y,w,start;

void solve(){
    cin>>n>>m>>start;//n 个点， m 条边， 从 start 开始

    G.resize(n+1);

    vector<ll> dis(n+1,LLONG_LONG_MAX);//存每个点的最短距离，默认权重无穷大
    dis[start]=0;//起点最短为 0

    for(int i=1;i<=m;i++){
        cin>>x>>y>>w;//从 x 到 y 权重是 w 的边
        G[x].push_back({y,w});//有向图只这一行
        G[y].push_back({x,w});//无向图加上反向的情况
    }

    for(int i=0;i<n-1;i++){//重复 n-1 次松弛操作
        for(int j=1;j<=n;j++){//遍历每个点的每条边
            if(dis[j]<LLONG_LONG_MAX){
                for(const auto pr:G[j]){
                    ll u=pr.first,w=pr.second;
                    if(dis[j]+w<dis[u]) //松弛操作
                        dis[u]=dis[j]+w;
                }
            }
        }
    }

    //如果松弛 n-1 次之后，还可以继续松弛，那么就存在负权环
```

```

for(int i=1;i<=n;i++){
    if(dis[i]!=LONG_LONG_MAX){
        for(const auto pr:G[i]){
            ll u=pr.first,w=pr.second;
            if(dis[u]>dis[i]+w){
                cout<<"存在负权环";
                return ;
            }
        }
    }
}

for(int i=1;i<=n;i++) cout<<dis[i]<<" ";
}

int main(){
    ios::sync_with_stdio(0);
    cin.tie(0);cout.tie(0);
    //freopen("***.in","r",stdin);
    //freopen("***.out","w",stdout);
    int T=1;//cin>>T;
    while(T--){
        solve();
    }
    return 0;
}

/*
6 10 1
1 2 4
1 3 6
2 3 1
2 4 5
3 4 4
3 5 3
4 5 2

```

```
4 6 4
5 6 2
2 5 1
*/
```

6. Bellman-Ford 算法

算法原理：

Bellman-Ford 算法通过对所有边进行（点数 $v-1$ ）次松弛操作来找到最短路径。适用于有负权边和检测负权环的问题。

1. 初始化所有节点距离为 ∞ ，起点为 0
2. 对每条边 (u,v) 进行松弛操作
3. 重复 $V-1$ 次
4. 最后检查是否存在负权环

时间复杂度：

- 时间复杂度： $O(VE)$
- 空间复杂度： $O(V)$

代码块

```
#include<bits/stdc++.h>

using namespace std;

#define ll long long
#define el "\n"
#define prl pair<ll,ll>

vector<vector<prl>> G;//邻接表

ll n,m,x,y,w,start;

void solve(){
    cin>>n>>m>>start;//n 个点， m 条边， 从 start 开始
    G.resize(n+1);
    vector<ll> dis(n+1,LLONG_LONG_MAX);//存每个点的最短距离，默认权重无穷大
    dis[start]=0;//起点最短为 0
    for(int i=1;i<=m;i++){
        cin>>x>>y>>w;//从 x 到 y 权重是 w 的边
        G[x].push_back({y,w});//有向图只这一行
```

```

        G[y].push_back({x,w}); //无向图加上反向的情况
    }
    for(int i=0;i<n-1;i++){ //重复 n-1 次松弛操作
        for(int j=1;j<=n;j++){ //遍历每个点的每条边
            if(dis[j]<LONG_LONG_MAX){
                for(const auto pr:G[j]){
                    ll u=pr.first,w=pr.second;
                    if(dis[j]+w<dis[u]) //松弛操作
                        dis[u]=dis[j]+w;
                }
            }
        }
    }
    //如果松弛 n-1 次之后，还可以继续松弛，那么就存在负权环
    for(int i=1;i<=n;i++){
        if(dis[i]!=LONG_LONG_MAX){
            for(const auto pr:G[i]){
                ll u=pr.first,w=pr.second;
                if(dis[u]>dis[i]+w){
                    cout<<"存在负权环";
                    return ;
                }
            }
        }
    }
    for(int i=1;i<=n;i++) cout<<dis[i]<<" ";
}

int main(){
    ios::sync_with_stdio(0);
    cin.tie(0);cout.tie(0);
    //freopen("***.in","r",stdin);
    //freopen("***.out","w",stdout);
    int T=1;//cin>>T;

```

```

while(T--){
    solve();
}
return 0;
}
/*
6 10 1
1 2 4
1 3 6
2 3 1
2 4 5
3 4 4
3 5 3
4 5 2
4 6 4
5 6 2
2 5 1

0 4 5 7 5 7
*/

```

7. SPFA 算法

算法原理：

SPFA(Shortest Path Faster Algorithm)是 Bellman-Ford 的队列优化版本。适用于稀疏图且含负权边的情况，注意菊花图会卡 TLE。

1. 使用队列存储需要松弛的节点
2. 每次从队列取出节点 u ，松弛其所有邻居 v
3. 如果 v 的距离被更新且不在队列中，则加入队列
4. 直到队列为空

时间复杂度：

- 平均时间复杂度： $O(E)$
- 最坏时间复杂度： $O(VE)$

代码块

```
#include<bits/stdc++.h>

using namespace std;

#define ll long long
#define el "\n"
#define prl pair<ll,ll>

vector<vector<prl>> G;//邻接表

ll n,m,x,y,w,start;

void solve(){

    cin>>n>>m>>start;//n 个点， m 条边， 从 start 开始

    G.resize(n+1);

    for(int i=1;i<=m;i++){

        cin>>x>>y>>w;//从 x 到 y 权重是 w 的边

        G[x].push_back({y,w});//有向图只这一行

        G[y].push_back({x,w});//无向图加上反向的情况

    }

    vector<ll> dis(n+1,LLONG_LONG_MAX);//存每个点的最短距离，默认权重无穷大

    vector<bool> inque(n+1,0);

    vector<ll> count(n+1,0);

    dis[start]=0;//起点最短为 0

    queue<ll> q;

    q.push(start);

    inque[start]=1;

    count[start]++;

    while(!q.empty()){

        ll u=q.front();q.pop();

        inque[u]=0;

        for(auto [v,w]:G[u]){

            if(dis[u]+w<dis[v]){

                dis[v]=dis[u]+w;

                if(!inque[v]){

                    q.push(v);

                }

            }

        }

    }

}
```

```

        count[v]++;
        if(count[v]>n){
            cout<<"有负权环";
            return ;
        }
    }

}

}

}

for(int i=1;i<=n;i++) cout<<dis[i]<<" ";
}

int main(){
    ios::sync_with_stdio(0);
    cin.tie(0);cout.tie(0);
    //freopen("***.in","r",stdin);
    //freopen("***.out","w",stdout);
    int T=1;//cin>>T;
    while(T--){
        solve();
    }
    return 0;
}

/*
6 10 1
1 2 4
1 3 6
2 3 1
2 4 5
3 4 4
3 5 3
4 5 2
4 6 4

```

5 6 2
2 5 1
*/

71. 最小生成树算法总结

最小生成树(Minimum Spanning Tree, MST)是图论中的一个重要概念，指在一个连通无向图中，找到一个边的子集，使得所有顶点都连通，并且边的权重之和最小。

算法	时间复杂度	空间复杂度	适用场景	特点
Prim算法	$O(E \log V)$	$O(V)$	稠密图	基于顶点扩展，适合邻接矩阵
Kruskal算法	$O(E \log E)$	$O(E)$	稀疏图	基于边排序，适合邻接表
Boruvka算法	$O(E \log V)$	$O(V)$	并行计算	分阶段处理，适合分布式

1. Prim 算法

算法原理

从图中任选一个顶点作为起始点，加入 MST 集合
不断选择连接 MST 集合与非 MST 集合的最小权重边
将该边加入 MST，并将对应的顶点加入 MST 集合
重复上述过程直到所有顶点都加入 MST

算法特点

贪心算法：每次选择当前最优的局部解
基于顶点扩展：逐步扩大 MST 的顶点集合
适合稠密图：在边数较多的图中表现良好，用邻接表存储图
时间复杂度：使用优先队列优化后为 $O(E \log V)$

关键数据结构

优先队列(最小堆)：存储连接 MST 集合与非 MST 集合的边，按权重排序
标记数组：记录顶点是否已加入 MST
键值数组：存储每个顶点连接到 MST 的最小边权重
父节点数组：记录 MST 中每个顶点的父节点

空间复杂度

邻接表存储图： $O(V + E)$
辅助数据结构 (key、parent、inMST)： $O(V)$

优先队列：最坏情况下 $O(E)$

总空间复杂度： $O(V + E)$

Prim 算法特别适合

稠密图（边数接近完全图）

需要从特定点开始构建 MST 的情况

图以邻接矩阵形式存储的情况

需要逐步扩展 MST 的场景

```
#include <bits/stdc++.h>

using namespace std;

using Edge = pair<int, int>; // pair<weight, vertex>

// 全局变量存储图状态和 MST 信息
vector<vector<Edge>> graph;
vector<int> key;
vector<int> parent;
vector<bool> inMST;
priority_queue<Edge, vector<Edge>, greater<Edge>> pq;

// 初始化 Prim 算法数据结构
void initPrim(int V) {
    key.assign(V, INT_MAX);
    parent.assign(V, -1);
    inMST.assign(V, false);
    graph.resize(V);
}

// 构建初始 MST
void buildMST(int startVertex) {
    pq.push(make_pair(0, startVertex));
    key[startVertex] = 0;
```

```

while (!pq.empty()) {
    int u = pq.top().second;
    pq.pop();

    if (inMST[u]) continue;
    inMST[u] = true;

    for (const auto& edge : graph[u]) {
        int weight = edge.first;
        int v = edge.second;

        if (!inMST[v] && weight < key[v]) {
            key[v] = weight;
            parent[v] = u;
            pq.push(make_pair(key[v], v));
        }
    }
}

// 添加新边并增量更新 MST
void addEdgeAndUpdate(int u, int v, int weight) {
    // 添加双向边
    graph[u].emplace_back(weight, v);
    graph[v].emplace_back(weight, u);

    // 检查新边是否可能改进 MST
    if ((inMST[u] && !inMST[v] && weight < key[v]) ||
        (inMST[v] && !inMST[u] && weight < key[u])) {

        if (inMST[u]) {
            key[v] = weight;
            parent[v] = u;

```

```

        pq.push(make_pair(key[v], v));
    } else {
        key[u] = weight;
        parent[u] = v;
        pq.push(make_pair(key[u], u));
    }

    // 继续 Prim 算法过程
    while (!pq.empty()) {
        int x = pq.top().second;
        pq.pop();

        if (inMST[x]) continue;
        inMST[x] = true;

        for (const auto& edge : graph[x]) {
            int w = edge.first;
            int y = edge.second;

            if (!inMST[y] && w < key[y]) {
                key[y] = w;
                parent[y] = x;
                pq.push(make_pair(key[y], y));
            }
        }
    }
}

```

// 打印当前 MST

```

void printMST() {
    int V = graph.size();

    cout << "Current Minimum Spanning Tree Edges:\n";
}

```

```

int totalWeight = 0;
for (int i = 1; i < V; ++i) {
    if (parent[i] != -1) {
        cout << parent[i] << " -- " << i << " : " << key[i] << "\n";
        totalWeight += key[i];
    }
}
cout << "Total weight: " << totalWeight << endl;
}

```

```

int main() {
    const int V = 5;
    initPrim(V);

    // 初始边
    addEdgeAndUpdate(0, 1, 2);
    addEdgeAndUpdate(0, 3, 6);
    addEdgeAndUpdate(1, 2, 3);
    addEdgeAndUpdate(1, 3, 8);
    addEdgeAndUpdate(1, 4, 5);
    addEdgeAndUpdate(2, 4, 7);
    addEdgeAndUpdate(3, 4, 9);

    // 构建初始 MST
    buildMST(0);
    printMST();

    // 增量添加新边
    cout << "\n 添加新边 2 3 1\n";
    addEdgeAndUpdate(2, 3, 1);
    printMST();

    cout << "\n 添加新边 0 4 4\n";
}

```

```
    addEdgeAndUpdate(0, 4, 4);  
  
    printMST();  
  
    return 0;  
}
```

2. Kruskal 算法

Kruskal 算法也是一种贪心算法，按照边的权重从小到大排序，依次选择不形成环的边加入 MST。

算法原理

将所有边按权重从小到大排序

依次选择权重最小的边，如果这条边连接的两个顶点不在同一个连通分量中（即不形成环），则将其加入最小生成树

重复步骤 2 直到选择了 $V-1$ 条边（ V 为顶点数）

关键数据结构

并查集(Disjoint Set Union)

Kruskal 算法的核心在于高效判断两个顶点是否在同一个连通分量中，这需要使用并查集数据结构：

Find 操作：查找元素所属的集合（根节点）

Union 操作：合并两个集合

并查集优化技术

路径压缩：在 Find 操作中，将查找路径上的所有节点直接指向根节点

按秩合并：在 Union 操作中，将较小的树合并到较大的树下

实现步骤：

将所有边按权重从小到大排序

初始化并查集数据结构

按顺序处理每条边：

 如果边的两个顶点不在同一集合(不形成环)，则加入 MST

 合并这两个顶点的集合

```
#include <bits/stdc++.h>  
  
using namespace std;  
  
#define ll long long  
  
#define el '\n'  
  
struct edge{
```

```

    ll u,v,w;//端点 1, 端点 2, 边权

    bool operator <(const edge &b)const{

        return w<b.w;

    }

};

ll n,weight;//n 个节点, 最小生成树边权和 weight
vector<edge> edges,mst;//图的边集, 最小生成树边集
vector<ll> parent,depth;//并查集和并查子集深度
ll findParent(vector<ll> &parent,ll u){

    if(parent[u]!=u)

        parent[u]=findParent(parent,parent[u]);//路径压缩

    return parent[u];

}

void kruskalMST(){

    sort(edges.begin(),edges.end());

    for(edge e:edges){

        ll rootu=parent[e.u],rootv=parent[e.v];

        //按秩合并, 将小树合并到大树下面, 以降低整体高度

        if(rootu!=rootv){

            mst.push_back(e);

            weight+=e.w;

            if(depth[rootu]<depth[rootv])

                parent[rootu]=rootv;

            else if(depth[rootu]>depth[rootv])

                parent[rootv]=rootu;

            else{

                parent[rootu]=rootv;

                depth[rootu]++;

            }

        }

    }

    //如果选中了 n-1 条边, 就完成了

    if(mst.size()==n-1) break;

}

```

```

//输出
for(edge e:mst){
    cout<<e.u<<" "<<e.v<<" "<<e.w<<endl;
}
}

void solve()
{
    n=5;
    //初始化并查集
    parent.resize(n);
    depth.resize(n);
    for(int i=0;i<n;i++) parent[i]=i;
    //读入边数据
    edges={
        {0, 1, 2},
        {0, 3, 6},
        {1, 2, 3},
        {1, 3, 8},
        {1, 4, 5},
        {2, 4, 7},
        {3, 4, 9}
    };
    kruskalMST();
}

int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);cout.tie(0);
    //freopen("P.in","r",stdin);
    //freopen("***.out","w",stdout);
    int T=1;//cin>>T;
    while(T--){
        solve();
    }
}

```

```

    }
    return 0;
}

```

72. 图的 dfn 序和 low 数组

dfn 序与 low 数组是什么

dfn 序是 dfs 时访问节点的序号，low 数组是每个节点在 DFS 遍历过程中能回溯到的最早访问的祖先节点。

dfn[u]: 节点 u 的 DFS 访问序号（时间戳）

low[v]: 节点 v 能追溯到的最早访问的祖先节点（最小 dfn 值）

dfn 序和 low 数组的应用场景特征

在强连通分量、割点问题、割边问题、LCA 等问题中有重要应用。

low 数组具有以下特性：若存在 u 到 v 的边

low 越小的节点越接近 dfs 的根

孤立节点的 low 值等于其 dfn 值，这类节点自成一个强连通分量

low 值相同的节点一定属于同一个强连通分量(SCC)，分量的根满足条件 $\text{low}[u] == \text{dfn}[u]$

$\text{low}[v] \leq \text{dfn}[u]$ 时，说明存在包含(u,v)的环路

$\text{low}[v] == \text{dfn}[u]$ 表示 v 通过单一路径回到 u

$\text{low}[v] > \text{dfn}[u]$ 确保边(u,v)是连接两个分量的唯一路径，即割边

$\text{low}[v] \geq \text{dfn}[u] \ \&\& \ u \neq \text{root}$ 反映子树 v 无法绕过 u 到达更高层级节点，u 是割点。如果根节点有多个子树，那么根节点也是割点。

适合增量计算，当添加新边时，可以局部更新 low 值

```

#include<bits/stdc++.h>
using namespace std;
#define ll long long
#define el '\n'
vector<vector<ll>>> G;
ll n,m,from,to,counter=1;
vector<bool> vis;
vector<ll> dfn,low;
void dfsn(ll u,ll parent){

```

```

vis[u]=1;
low[u]=dfn[u]=counter++;
for(ll v:G[u]){
    if(!vis[v]){
        dfsn(v,u);
        low[u]=min(low[u],low[v]);
    }else if(v!=parent){
        low[u]=min(low[u],dfn[v]);
    }
}
}
}
void solve(){
    cin>>n>>m;
    G.resize(n+1);
    vis.assign(n+1,0);
    dfn.resize(n+1);
    low.resize(n+1);
    while(m--){
        cin>>from>>to;
        G[from].emplace_back(to);
        G[to].emplace_back(from);
    }
    for(int i=1;i<=n;i++){
        if(!vis[i]){
            dfsn(i,-1);
        }
    }
    for(int i=1;i<=n;i++) cout<<dfn[i]<<" ";
    cout<<el;
    for(int i=1;i<=n;i++) cout<<low[i]<<" ";
}
int main(){

```

```

ios::sync_with_stdio(0);

cin.tie(0);cout.tie(0);

// freopen("***.in","r",stdin);
// freopen("***.out","w",stdout);

int T=1;//cin>>T;

while(T--){

    solve();

}

return 0;

}

/*
in:
8 8
1 2
2 4
2 5
5 8
1 3
3 6
3 7
2 8
out:
1 2 6 3 4 7 8 5
1 2 6 3 2 7 8 2
*/

```

1. 强连通分量

low 值相同的节点属于一个强连通分量，所以可以桶一下。

也可以用栈存储 dfs 过程，在出现 $\text{low}[u] == \text{dfv}[u]$ 时输出栈顶元素到 u 做为一个强连通分量。

Tarjan 算法求强连通分量

```

#include <bits/stdc++.h>

using namespace std;

```

```

const int MAXN = 10010;
vector<int> G[MAXN];
int dfn[MAXN], low[MAXN], idx = 0;
bool inStack[MAXN];
stack<int> st;
vector<vector<int>> sccs;

void tarjan(int u) {
    dfn[u] = low[u] = ++idx;
    st.push(u);
    inStack[u] = true;

    for (int v : G[u]) {
        if (!dfn[v]) {
            tarjan(v);
            low[u] = min(low[u], low[v]);
        } else if (inStack[v]) {
            low[u] = min(low[u], dfn[v]);
        }
    }

    if (dfn[u] == low[u]) {
        vector<int> scc;
        while (true) {
            int v = st.top();
            st.pop();
            inStack[v] = false;
            scc.push_back(v);
            if (v == u) break;
        }
        sccs.push_back(scc);
    }
}

```

```

}

void findSCCs(int n) {
    fill(dfn, dfn + n + 1, 0);
    fill(low, low + n + 1, 0);
    fill(inStack, inStack + n + 1, false);
    idx = 0;
    sccs.clear();

    for (int i = 1; i <= n; ++i) {
        if (!dfn[i]) {
            tarjan(i);
        }
    }
}

```

2. 割点

根节点如果有多个子树，那么根节点是割点。

非根节点且 $low[v] \geq dfn[u]$ ， u 是割点

```

#include<bits/stdc++.h>
using namespace std;
#define ll long long
#define el '\n'
ll n,m,from,to,counter=1;
vector<vector<ll>> G;
vector<bool> vis,isCut;
vector<ll> dfn,low;
//根节点如果有多个子树，那么根节点是割点。
//非根节点且 low[v]>=dfn[u], u 是割点
void tarjan(ll u,ll parent){
    vis[u]=1;
    low[u]=dfn[u]=counter++;

```

```

int cnt=0;
for(ll v:G[u]){
    if(v==parent) continue;
    if(!vis[v]){
        tarjan(v,u);
        cnt++;
        low[u]=min(low[u],low[v]);
        //处理非根节点割点
        if(parent!=-1&&low[v]>=dfn[u]) isCut[u]=1;
    }else{
        low[u]=min(low[u],dfn[v]);
    }
}
//处理根节点割点
if(parent==-1&&cnt>1) isCut[u]=1;
}

void solve(){
    cin>>n>>m;
    G.resize(n+1);
    vis.assign(n+1,0);
    isCut.assign(n+1,0);
    dfn.resize(n+1);
    low.resize(n+1);
    for(int i=0;i<m;i++){
        cin>>from>>to;
        G[from].emplace_back(to);
        G[to].emplace_back(from);
    }
    tarjan(1,-1);
    for(int i=1;i<=n;i++) cout<<dfn[i]<<" ";
    cout<<el;
    for(int i=1;i<=n;i++) cout<<low[i]<<" ";
    cout<<el;
}

```

```

        for(int i=1;i<=n;i++) cout<<isCut[i]<<" ";
    }
int main(){
    ios::sync_with_stdio(0);
    cin.tie(0);cout.tie(0);
    // freopen("***.in","r",stdin);
    // freopen("***.out","w",stdout);
    int T=1;//cin>>T;
    while(T--){
        solve();
    }
    return 0;
}
/*
8 8
1 2
2 4
2 5
5 8
1 3
3 6
3 7
2 8
out:
1 2 6 3 4 7 8 5
1 2 6 3 2 7 8 2
1 1 1 0 0 0 0 0
*/

```

3. 割边

当 $low[v] > dfn[u]$ 是 $u \rightarrow v$ 是一条割边。

```
#include<bits/stdc++.h>
```

```

using namespace std;

#define ll long long
#define el '\n'

ll n, m, from, to, counter = 1;

vector<vector<ll>> G;    // 图的邻接表表示
vector<bool> vis, isCut; // 访问标记和割点标记
vector<ll> dfn, low;    // DFS 序和 low 值
stack<ll> stk;          // 用于 SCC 计算的栈
vector<vector<ll>> sccs; // 存储所有强连通分量
vector<pair<ll, ll>> bridges; // 存储所有割边（桥）

void init() {
    counter = 1;
    G.clear();
    vis.clear();
    isCut.clear();
    dfn.clear();
    low.clear();
    while(!stk.empty()) stk.pop();
    sccs.clear();
    bridges.clear(); // 清空割边集合
}

void tarjan(ll u, ll parent) {
    vis[u] = 1;
    low[u] = dfn[u] = counter++;
    stk.push(u);
    int cnt = 0; // 记录子树数量

    for(ll v : G[u]) {
        if(v == parent) continue; // 跳过父节点
    }
}

```

```

if(!vis[v]) {
    cnt++;
    tarjan(v, u);
    low[u] = min(low[u], low[v]);

    // 割点判断条件
    if(parent != -1 && low[v] >= dfn[u])
        isCut[u] = 1;

    // 割边（桥）判断条件
    if(low[v] > dfn[u])
        bridges.emplace_back(min(u, v), max(u, v));

} else {
    low[u] = min(low[u], dfn[v]);
}
}

// 根节点割点判断
if(parent == -1 && cnt > 1)
    isCut[u] = 1;

// 强连通分量计算
if(low[u] == dfn[u]) {
    vector<ll> scc;
    while(1) {
        ll v = stk.top();
        stk.pop();
        scc.push_back(v);
        if(v == u) break;
    }
    sccs.push_back(scc);
}

```

```
}
```

```
void solve() {
```

```
    init();
```

```
    cin >> n >> m;
```

```
    G.resize(n+1);
```

```
    vis.assign(n+1, 0);
```

```
    isCut.assign(n+1, 0);
```

```
    dfn.resize(n+1);
```

```
    low.resize(n+1);
```

```
    for(int i = 0; i < m; i++) {
```

```
        cin >> from >> to;
```

```
        G[from].emplace_back(to);
```

```
        G[to].emplace_back(from);
```

```
    }
```

```
    tarjan(1, -1);
```

```
    // 输出 dfn 序
```

```
    for(int i = 1; i <= n; i++) cout << dfn[i] << " ";
```

```
    cout << el;
```

```
    // 输出 low 值
```

```
    for(int i = 1; i <= n; i++) cout << low[i] << " ";
```

```
    cout << el;
```

```
    // 输出割点
```

```
    for(int i = 1; i <= n; i++) cout << isCut[i] << " ";
```

```
    cout << el;
```

```
    // 输出强连通分量
```

```
    cout << "sccs:" << el;
```

```

    for(int i = 0; i < sccs.size(); i++) {
        for(int j = 0; j < sccs[i].size(); j++)
            cout << sccs[i][j] << " ";
        cout << el;
    }

    // 输出割边 (桥)
    cout << "bridges:" << el;
    for(auto& bridge : bridges) {
        cout << bridge.first << " " << bridge.second << el;
    }
}

```

```

int main() {
    ios::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);

    int T = 1; // cin >> T;
    while(T--) {
        solve();
    }
    return 0;
}

```

```

/*

```

```

in:

```

```

8 8

```

```

1 2

```

```

2 4

```

```

2 5

```

```

5 8

```

```

1 3

```

```

3 6

```

```

3 7

```

```

8 2
out:
1 2 6 3 4 7 8 5
1 2 6 3 2 7 8 2
1 1 1 0 0 0 0 0
sccs:
4
8 5 2
6
7
3
1
bridges:
2 4
1 2
3 6
3 7
1 3
*/

```

4. 欧拉回路

1. 欧拉回路存在条件：
 - 有向图：每个节点入度=出度
 - 无向图：所有节点度数为偶数
2. Hierholzer 算法：
 - 使用 DFS 遍历图，当没有未访问的边时，将节点加入路径。
 - DFN 序确保我们按照深度优先的顺序遍历，最后反转路径得到欧拉回路。
3. 正确性：DFS 保证了每条边只访问一次，且通过栈结构保证了回路的连续性。

```

#include <bits/stdc++.h>

using namespace std;

```

```

const int MAXN = 10010;

vector<pair<int, int>> G[MAXN]; // 邻接表, 存储(to, edgeIndex)

int dfn[MAXN], idx = 0;

vector<bool> visitedEdge;

vector<int> eulerPath;

void dfsEuler(int u) {
    dfn[u] = ++idx;
    for (auto it : G[u]) {
        v=it.first; edgeIdx=it.second;
        if (!visitedEdge[edgeIdx]) {
            visitedEdge[edgeIdx] = true;
            dfsEuler(v);
        }
    }
    eulerPath.push_back(u);
}

vector<int> findEulerPath(int n, int m, int start) {
    fill(dfn, dfn + n + 1, 0);
    visitedEdge.assign(m, false);
    eulerPath.clear();
    idx = 0;

    dfsEuler(start);
    reverse(eulerPath.begin(), eulerPath.end());
    return eulerPath;
}

```

73. 树状数组

树状数组是一种高效的数据结构，主要用于解决前缀和查询和单点更新问题。它能够在 $O(\log n)$ 时间内完成单点更新和前缀查询，比普通数组的前缀和查询(更新 $O(n)$ ，查询 $O(1)$)或普通数组的单点操作(更新 $O(1)$ ，查询 $O(n)$)更高效。

树状数组的核心思想是利用二进制表示的特性来高效维护前缀和：

1. 二进制索引：每个索引 i 可以表示为 2 的幂次方的和，树状数组利用这一点将前缀和分解为多个子段的和
2. lowbit 函数：计算一个数的最低有效位(最右边的 1)，这是树状数组操作的关键
3. 分层存储：数组元素被组织成多级结构，每层覆盖不同长度的区间

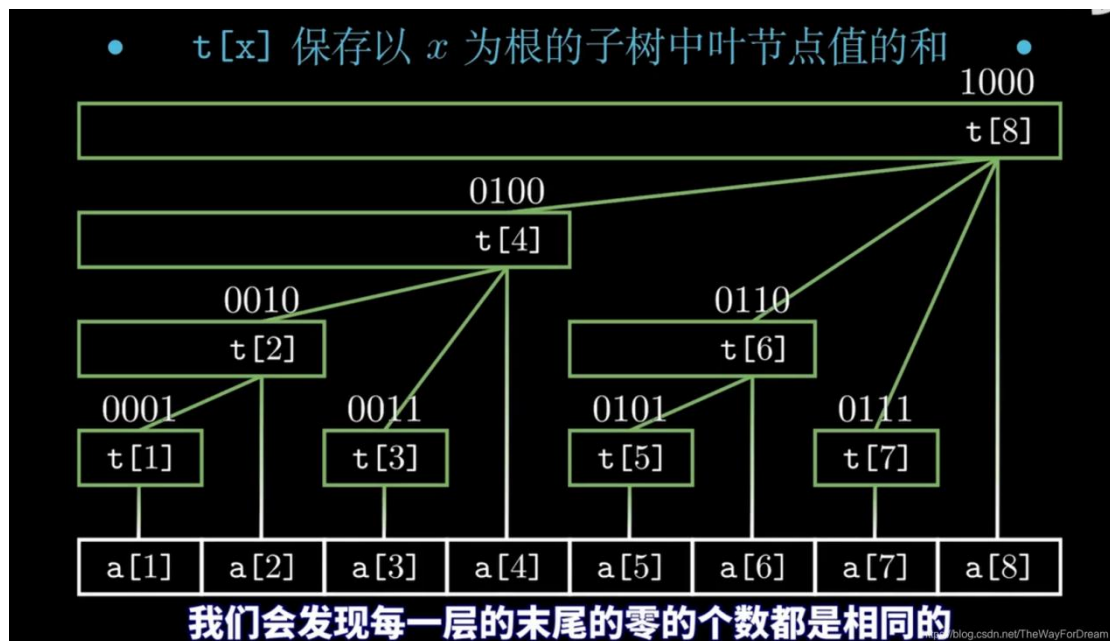
lowbit 函数

C

```
int lowbit(int x) {return x & -x;}
```

这个函数返回 x 的最低有效位。例如：

- $\text{lowbit}(6) = 2$ ($6=110_2$)
- $\text{lowbit}(8) = 8$ ($8=1000_2$)



推演 $i+\text{lowbit}(i)$ 和 $i-\text{lowbit}(i)$ 的变化规律。

更新操作

当更新位置 i 的值时，我们需要更新所有覆盖 i 的区间：

1. 从 i 开始
2. 每次加上 $\text{lowbit}(i)$ ，直到超过数组长度
3. 在每个位置更新值

查询操作

查询前缀和[1..i]:

1. 从 i 开始
2. 每次减去 `lowbit(i)`，直到为 0
3. 累加所有经过位置的值

实现思想

1. 初始化 : 创建一个数组(索引从 1 开始)
2. 单点更新 : 更新指定位置的值，并更新所有相关区间
3. 前缀查询 : 计算从 1 到指定位置的所有元素的和
4. 区间查询 : 通过两次前缀查询相减得到区间和

```
C
#include<bits/stdc++.h>

using namespace std;

#define ll long long
#define el '\n'

const ll MAXN=500005;

ll n,m,a[MAXN],ts[MAXN]; //a 原数组，ts 树状区间和数组

ll lowbit(ll x){ return x&-x; }

void updateSum(ll x,ll y){ //把原数组第 i 项增加 y
    while(x<=n){
        ts[x]+=y;
        x+=lowbit(x);
    }
    // for(;x<=n;x+=lowbit(x)) ts[x]+=y;
}

ll querySum(ll x){ //查询前 x 项的和
    ll res=0;
    while(x>0){
        res+=ts[x];
        x-=lowbit(x);
    }
    // for(;x>0;x-=lowbit(x)) res+=ts[x];

    return res;
```

```

}

int solve(){
    cin>>n>>m;
    for(int i=1;i<=n;i++){
        cin>>a[i];
        updateSum(i,a[i]);
    }
    for(int i=0;i<m;i++){
        ll opt,x,y;
        cin>>opt>>x>>y;
        if(opt==1) updateSum(x,y);
        else cout<<querySum(y)-querySum(x-1)<<el;
    }
}

```

复杂度分析

- 空间复杂度 : $O(n)$
- 时间复杂度 :
 - 构建: $O(n \log n)$
 - 单点更新: $O(\log n)$
 - 前缀查询: $O(\log n)$
 - 区间查询: $O(\log n)$

应用场景

1. 频繁更新和查询前缀和的问题

- a. 单点修改单点查询 ok
- b. 单点修改区间查询 ok
- c. 区间修改单点查询: 利用差分数组可以实现快速区间修改, 差分数组的前缀和可以还原原数组得到第 i 项。所以用树状数组维护差分数组 $D[]$, 区间修改 $(+k)$ 就是修改差分数组的 $D[L]$ 和 $D[R+1]$ 项分别 $+k$ 和 $-k$ 时间复杂度 $O(1)$, 差分数组的前缀和可以方便得用树状数组求解, 时间复杂度 $O(\log n)$ 。
- d. 区间修改区间查询, 用线段树。

2. 逆序对计数

将原数组 $a[]$ 离散化得到数组 $b[]$ ，然后倒序遍历 b 数组，用树状数组维护 b 数组做计数数组 $ts[]$ 的过程 $O(n\log n)$ ，用树状数组维护 ts 数组，然后倒序累计即可得到逆序对数量 $O(n\log n)$ 。相比归并排序，树状数组求逆序对支持在线查询。

```
#include<bits/stdc++.h>

using namespace std;

#define ll long long
#define el endl

const ll MAXN=500005;

//a 原数组，ts 树状数组
ll n,m,a[MAXN],ts[MAXN],cnt;

set<ll> st;

unordered_map<ll,ll> dic;

//init_set 去重排序集合
void init_set(){
    st.clear();
    for(int x:a) st.insert(x);
}

//init_map 离散化并返回字典
void init_map(){
    dic.clear();
    ll rank=1;
    for(int x:st) dic[x]=rank++;
    fill(ts+1,ts+st.size()+1,0);
}

ll lowbit(ll x){ return x&-x; }

void update(ll x,ll y){
    for(;x<=n;x+=lowbit(x)) ts[x]+=y;
}

ll query(ll x){
    ll res=0;
    for(;x>0;x-=lowbit(x)) res+=ts[x];
    return res;
}
```

```

void makeTree(){
    cnt=0;
    for(int i=n-1;i>=0;i--){
        ll rank=dic[a[i]];
        cnt+=query(rank-1);
        update(rank,1);
    }
}

void insert(ll x){
    n++;
    if(st.find(x)==st.end()){//新增的 x 以前没有出现
        if(x>*st.rbegin()){//x 比原最大值还大，直接加到后面，逆序对数量不变
            st.insert(x);
            dic[x]=st.size();
            update(dic[x],1);
        }else{//x 排序在中间的时候，需要重新更新离散化字典
            st.insert(x);
            init_map();
            makeTree();
        }
    }else{//x 在 a[]中已经有了，x 的 n-1 个数对中，减掉<=x 的元素数量 就是新增的数量
        cnt+=n-1-query(dic[x]);
        update(dic[x],1);
    }
}

void solve(){
    cin>>n;
    for(int i=0;i<n;i++) cin>>a[i];
    init_set();
    init_map();
    //倒序维护对 dic[a[i]]的计数树状数组
    makeTree();
    cout<<cnt;
}

```

```

    ll k; cin>>k; insert(k);

    cout<<cnt;
}

int main(){
    // ios::sync_with_stdio(nullptr);
    // cin.tie(nullptr);cin.tie(nullptr);

    int T=1;//cin>>;
    while(T--){
        solve();
    }
}

```

5. 区间统计问题

3. 需要高效单点更新和区间查询的场景

如果要查询区间最大值，做以下调整：

1. 每个节点存储对应区间的最大值而非和
2. 查询时需要合并多个区间的结果
3. 更新操作只能增大值（减小值需要特殊处理）

```

#include<bits/stdc++.h>

using namespace std;

#define ll long long

#define el '\n'

const ll MAXN=5e5+5;

ll n,m,a[MAXN],t[MAXN];

void update(ll x,ll y){//只能增大
    if(y<a[x]) return ;
    a[x]=y;
    for(;x<=n;x+=x&-x) t[x]=max(t[x],y);
}

//查询 a[1~x]的最大值

ll query_prefix(ll x){

```

```

    ll res=LONG_LONG_MIN;

    for(;x>0;x-=x&-x){
        res=max(res,t[x]);
    }

    return res;
}

//查询 a[x~y]的最大值
ll query_range(ll x,ll y){
    ll res=LONG_LONG_MIN;
    while(x<=y){
        if(y-(y&-y)+1>=x){//能用树状数组优先用 y-(y&-y)+1 是 t[y]覆盖范围的最小下标
            res=max(res,t[y]);
            y-=y&-y;
        }else{//剩余的用原始数组
            res=max(res,a[y--]);
        }
    }

    return res;
}

void solve(){
    cin>>n;
    fill(t,t+n+1, LONG_LONG_MIN);
    for(int i=1;i<=n;i++) {
        cin>>a[i];
        update(i,a[i]);
    }

    cin>>m;
    for(int i=1;i<=m;i++){
        ll opt,x,y;
        cin>>opt>>x>>y;
        if(opt==1){
            update(x,y);
        }else{

```

```
        cout<<query_range(x,y)<<el;
    }
}
}

int main(){
    // ios::sync_with_stdio(0);
    // cin.tie(0);cout.tie(0);
    // freopen("***.in","r",stdin);
    // freopen("***.out","w",stdout);
    int T=1;//cin>>T;
    while(T--){
        solve();
    }
    return 0;
}
```